Practical Exam Practice Solutions

The HRMS scenario: A_1 to B_3

COS 110 | Swift Tuition | 2026 edition

Model solutions for the practice questions. Try each question first. Each solution is the one file you would upload. It compiles with g++ -std=c++98 -Wall -Werror and runs clean under valgrind with the given files. Other correct answers are possible.

Contents

- **A_1 Exceptions**
  1. A1-01. Exceptions: Employee::setIDNum (length and gender) (100)
  2. A1-02. Exceptions: Employee::setIDNum (digits, month, day and gender) (100)
  3. A1-03. Exceptions: Employee::setIDNum (the age must match the birth year) (100)
  4. A1-04. Exceptions: Employee::extractIntFromString (100)
  5. A1-05. Exceptions: Team::operator[] throws for a bad index or an empty slot (100)
  6. A1-06. Exceptions: Department::operator[] with two error messages (100)
  7. A1-07. Exceptions: Team::addMember (NULL, already in the team, full) (100)
  8. A1-08. Exceptions: Team::assignIDs catches each error and continues (100)
  9. A1-09. Exceptions: Team::setMemberID catches two exception types (100)
  10. A1-10. Exceptions: Team::hire frees memory before it throws again (100)
- **A_2 Operator overloading**
  1. A2-01. Operator overloading: operator<< for Employee (100)
  2. A2-02. Operator overloading: operator<< for Team (skips empty slots) (100)
  3. A2-03. Operator overloading: operator<< for Department (virtual calls) (100)
  4. A2-04. Operator overloading: template operator<< for List<T> (100)
  5. A2-05. Operator overloading: Team::operator[] (const) returning const Employee& (100)
  6. A2-06. Operator overloading: Department::operator[] returning Team*& (100)
  7. A2-07. Operator overloading: Employee::operator== and operator!= (100)
  8. A2-08. Operator overloading: Department operator< and operator> by cost (100)
  9. A2-09. Operator overloading: Team::operator+= grows the array by one (100)
  10. A2-10. Operator overloading: Team::operator+ returns a merged team (100)
  11. A2-11. Operator overloading: Employee prefix and postfix operator++ (a birthday) (100)
  12. A2-12. Operator overloading: Team::operator() finds a surname (100)
- **A_3 Assignment operator**
  1. A3-01. Assignment operator: Employee::operator= (100)
  2. A3-02. Assignment operator: Team::operator= (100)
  3. A3-03. Assignment operator: Team::operator= with a compact copy (100)
  4. A3-04. Assignment operator: Department::operator= (100)
  5. A3-05. Assignment operator: Sales::operator= in a derived class (100)
  6. A3-06. Assignment operator: List<T>::operator= (100)
  7. A3-07. Copy constructor: Team::Team(const Team&) (100)
- **B_1 Recursion**
  1. B1-01. Recursion: Employee::extractIntFromString (100)
  2. B1-02. Recursion: Employee::isAllDigits (100)
  3. B1-03. Recursion: Team::countMembers (100)
  4. B1-04. Recursion: Department::totalEmployees (100)
  5. B1-05. Recursion: List<T>::countNodes (100)
  6. B1-06. Recursion: List<T>::printReverse (100)
  7. B1-07. Recursion: List<T>::contains (100)
  8. B1-08. Recursion: Company::totalCost (100)
  9. B1-09. Recursion: Employee::checksum (100)
  10. B1-10. Recursion: Team::oldest (100)
- **B_2 Linked lists**
  1. B2-01. Linked lists: List<T>::append (100)
  2. B2-02. Linked lists: List<T>::insertAt (100)
  3. B2-03. Linked lists: List<T>::remove by value (100)
  4. B2-04. Linked lists: List<T>::removeAt (100)
  5. B2-05. Linked lists: List<T>::reverse (100)
  6. B2-06. Linked lists: List<T>::sort by relinking (100)

7. B2-07. Linked lists: List<T>::indexOf (100)
8. B2-08. Linked lists: List<T>::operator[] (100)
9. B2-09. Linked lists: List<T>::clear (100)
10. B2-10. Linked lists: Company::removeDepartment (ownership when removing from a list) (100)
11. B2-11. Linked lists: List<T> copy constructor (100)

- **B_3 Template specialization**
  1. B3-01. Template specialization: printItem<Department*> (100)
  2. B3-02. Template specialization: largest<Employee> (100)
  3. B3-03. Template specialization: isEqual<Department*> (100)
  4. B3-04. Template specialization: List<Department*>::printAll (100)
  5. B3-05. Template specialization: List<Department*>::clear (100)
  6. B3-06. Template specialization: totalCost<Department*> (100)
  7. B3-07. Template specialization: addition<Employee, Employee> throws (100)

A_1 Exceptions: solutions

A1-01. Exceptions: `Employee::setIDNum` (length and gender) (100)

File: A_1.cpp

```
#include "Employee.h"

void Employee::setIDNum(string idNum) {
    if ((int)idNum.length() != 13) {
        throw Exception("ID number must have 13 digits");
    }
    // extractIntFromString throws "Not a digit" itself; that exception is not caught here.
    int genderDigits = extractIntFromString(idNum, 6, 10);
    char idGender = 'F';
    if (genderDigits >= 5000) {
        idGender = 'M';
    }
    if (idGender != gender) {
        throw Exception("ID number does not match gender");
    }
    this->idNum = idNum;
}
```

Expected output of the given main.cpp

```
Thandi Mokoena, 29, F, ID: 9702140123081
Caught: ID number must have 13 digits
Caught: ID number does not match gender
Sipho Dlamini, 35, M, ID: none
Sipho Dlamini, 35, M, ID: 9105225800087
```

What the function must do

The function `setIDNum` receives a new ID number as a `string`. It checks two things in a fixed order: the length is 13, and the gender digits match the employee's `gender`. If both checks pass, the function stores the ID number in the member `idNum`. If a check fails, the function throws an `Exception` and leaves the old `idNum` unchanged.

How the solution works

```
void Employee::setIDNum(string idNum) {
```

The parameter has the same name as the member `idNum`. Inside the function, the plain name `idNum` means the parameter. The member is written `this->idNum`. The word `this` is a pointer to the object on which the function was called.

Step 1: the length check.

```
    if ((int)idNum.length() != 13) {
        throw Exception("ID number must have 13 digits");
    }
```

- The function `length()` returns an unsigned number. The cast `(int)` turns it into an `int`, so both sides of the comparison have the same type.
- The condition is true when the string is shorter or longer than 13 characters. This includes the empty string.
- A `throw` stops the function at once. The line `this->idNum = idNum;` never runs, so the old value stays.
- This check comes first. The next step reads index 6 to 9. If the string has fewer than 10 characters, `extractIntFromString` would throw `Exception("Invalid range")` instead. The question asks for the length message in that case.

Step 2: read the gender digits.

```
    int genderDigits = extractIntFromString(idNum, 6, 10);
```

The given static function reads index 6, 7, 8 and 9 and returns them as one number. The end index 10 is not included. If one of these four characters is not a digit, `extractIntFromString` throws `Exception("Not a digit")`. The solution has no `try` and no `catch`, so the exception goes to the caller. The question says the function must not catch its own exceptions.

Step 3: turn the number into a letter.

```
    char idGender = 'F';
    if (genderDigits >= 5000) {
        idGender = 'M';
    }
```

The letter starts as `'F'`. The `if` condition checks the boundary of the male range. The value 5000 is male, so the test uses `>=` and not `>`. The value 4999 stays female.

Step 4: compare with the employee.

```
    if (idGender != gender) {
        throw Exception("ID number does not match gender");
    }
```

The name `gender` is the member of the employee. If the letters differ, the function throws and does not store the ID number.

Step 5: store the value.

```
    this->idNum = idNum;
```

This line comes last. The function changes the object only after both checks pass. If the line came first, a failed call would still change the object.

No memory is allocated in this function, so there is nothing to free. The `string` objects manage their own memory.

Trace

The call is `sipho->setIDNum("9105220800087")`. The employee Sipho has `gender` equal to `'M'` and `idNum` equal to `""`.

Step	Code	Values	Result
1	length check	length is 13	condition false, continue
2	`extractIntFromString(idNum, 6, 10)`	characters at index 6 to 9 are `0800`	`genderDigits` is 800
3	`genderDigits >= 5000`	800 >= 5000 is false	`idGender` stays `'F'`
4	`idGender != gender`	`'F'` != `'M'` is true	throw `Exception("ID number does not match gender")`
5	store	not reached	`idNum` stays `""`

The `catch (Exception& e)` in `main` receives the exception. A catch by reference uses the thrown object itself and does not copy it. The line `cout << "Caught: " << e.what() << endl;` prints the message. The later `cout << *sipho` prints `ID: none`, because `operator<<` prints `none` when `idNum` is empty.

The first call in `main` uses the ID `"910522580008"`, which has 12 characters. Step 1 throws. The last call uses `"9105225800087"`, whose gender digits are `5800`. Step 3 gives `'M'`, step 4 passes, and step 5 stores the ID.

The output of `main.cpp` is:

```
Thandi Mokoena, 29, F, ID: 9702140123081
Caught: ID number must have 13 digits
Caught: ID number does not match gender
Sipho Dlamini, 35, M, ID: none
Sipho Dlamini, 35, M, ID: 9105225800087
```

Common mistakes

- **Use `>` instead of `>=` for 5000.** The ID `8001015000081` belongs to a male employee. With `>`, the function reads 5000 as female and throws. The edge test rejected this change because its output differed.
- **Store the ID before the checks.** If the first line is `this->idNum = idNum;`, a failed call still changes the object. The fourth output line would be `Sipho Dlamini, 35, M, ID: 9105220800087`, the ID that failed the gender check, and not `ID: none`. The main test rejected this change.
- **Write `idNum = idNum;` at the end.** Inside the function, both names mean the parameter. The member never changes. The compiler gives no warning for this line. The first and the last output lines would show `ID: none`.
- **Catch `Exception` inside `setIDNum`.** The caller then never sees `Not a digit`. The edge test expects the line `Caught: Not a digit`.
- **Check the gender before the length.** For a string with fewer than 10 characters, such as the empty string, `extractIntFromString` cannot read index 6 to 9 and throws `Invalid range`. The expected message is `ID number must have 13 digits`. The edge test prints `Caught: Invalid range` for the empty string and rejects this change.

A1-02. Exceptions: `Employee::setIDNum` (digits, month, day and gender) (100)

File: A_1.cpp

```
#include "Employee.h"

void Employee::setIDNum(string idNum) {
    if ((int)idNum.length() != 13) {
        throw Exception("ID number must have 13 digits");
    }
    for (int i = 0; i < (int)idNum.length(); i++) {
        if (idNum[i] < '0' || idNum[i] > '9') {
            throw Exception("ID number must contain only digits");
        }
    }
    int month = extractIntFromString(idNum, 2, 4);
    if (month < 1 || month > 12) {
        throw Exception("Invalid birth month");
    }
    int day = extractIntFromString(idNum, 4, 6);
    if (day < 1 || day > 31) {
        throw Exception("Invalid birth day");
    }
    char idGender = 'F';
    if (extractIntFromString(idNum, 6, 10) >= 5000) {
        idGender = 'M';
    }
    if (idGender != gender) {
        throw Exception("ID number does not match gender");
    }
    this->idNum = idNum;
}
```

Expected output of the given main.cpp

```
Accepted: Thandi Mokoena, 29, F, ID: 9702140123081
Rejected 91052258000: ID number must have 13 digits
Rejected 9105225800087: ID number must contain only digits
Rejected 9113225800087: Invalid birth month
Rejected 9105005800087: Invalid birth day
Rejected 9105220800087: ID number does not match gender
Accepted: Sipho Dlamini, 35, M, ID: 9105225800087
```

What the function must do

The function `setIDNum` checks a new ID number with five checks in a fixed order: length, digits, month, day and gender. The first check that fails stops the function with an `Exception` and a message for that check. If all five checks pass, the function stores the ID number in `idNum`. If a check fails, `idNum` keeps its old value.

How the solution works

```
void Employee::setIDNum(string idNum) {
```

The parameter `idNum` has the same name as the member `idNum`. Inside the function, the plain name means the parameter. The member is written `this->idNum`. The word `this` is a pointer to the object on which the function was called.

Check 1: the length.

```
    if ((int)idNum.length() != 13) {
        throw Exception("ID number must have 13 digits");
    }
```

The function `length()` returns an unsigned number. The cast `(int)` makes both sides of the comparison `int`. Here the other side is the fixed number 13, so g++ gives no warning even without the cast. The cast is required in check 2, where the loop compares the variable `i` with the length. There, `-Wall -Werror` turns the signed/unsigned warning into an error. The condition is true for a string that is too short or too long. A `throw` stops the function at once. This check comes first, so the later steps can read index 0 to 12 without leaving the string.

Check 2: every character is a digit.

```
    for (int i = 0; i < (int)idNum.length(); i++) {
        if (idNum[i] < '0' || idNum[i] > '9') {
            throw Exception("ID number must contain only digits");
        }
    }
```

The loop visits all 13 characters. The characters `'0'` to `'9'` have consecutive codes. A character below `'0'` or above `'9'` is not a digit. This includes letters, a space and `'-'`. The loop compares characters one by one because a 13-digit number does not fit in an `int`.

This check comes before the month, the day and the gender. Those steps call `extractIntFromString`, and the helper throws `Exception("Not a digit")` for a non-digit. The question requires the message `ID number must contain only digits` for any non-digit in the whole string. For example, the letter `O` at index 2 must give this message and not the helper's message.

Check 3: the month.

```
    int month = extractIntFromString(idNum, 2, 4);
    if (month < 1 || month > 12) {
        throw Exception("Invalid birth month");
    }
```

The helper reads index 2 and 3 (the end index 4 is not included) and returns them as a number. The condition is true for 0 and for 13 or more. A valid month is 1 to 12. The two parts are joined with `||`, so one true part is enough to throw.

Check 4: the day.

```
    int day = extractIntFromString(idNum, 4, 6);
    if (day < 1 || day > 31) {
        throw Exception("Invalid birth day");
    }
```

The helper reads index 4 and 5. A valid day is 1 to 31. The value 0 is invalid, so the test is `day < 1` and not `day < 0`. The function does not check how many days the month has, so day 30 in month 02 is accepted.

Check 5: the gender.

```
    char idGender = 'F';
    if (extractIntFromString(idNum, 6, 10) >= 5000) {
        idGender = 'M';
    }
    if (idGender != gender) {
        throw Exception("ID number does not match gender");
    }
```

The letter starts as `'F'`. The gender digits at index 6 to 9 form a number. A value of 5000 or more means `'M'`. The second `if` compares the letter with the member `gender`. A difference makes the function throw.

Store the value.

```
    this->idNum = idNum;
```

This line comes last. The function changes the object only after all five checks pass.

The function never catches an exception and prints nothing. The function allocates no memory.

Trace

The call is `tryID(sipho, "9105220800087")`. Sipho has `gender` equal to `'M'`.

Step	Code	Values	Result
1	length check	length is 13	continue

Step	Code	Values	Result
2	digit loop, `i` from 0 to 12	all 13 characters are between `'0'` and `'9'`	no throw
3	`month`	characters at index 2 and 3 are `05`	`month` is 5
4	month test	5 < 1 is false, 5 > 12 is false	continue
5	`day`	characters at index 4 and 5 are `22`	`day` is 22
6	day test	22 < 1 is false, 22 > 31 is false	continue
7	gender digits	characters at index 6 to 9 are `0800`	value is 800, `idGender` stays `'F'`
8	`idGender != gender`	`'F'` != `'M'` is true	throw `Exception("ID number does not match gender")`

The function `tryID` catches the exception by reference and prints `Rejected 9105220800087: ID number does not match gender`. The ID number is not stored. The last call, `"9105225800087"`, has gender digits `5800`, so `idGender` becomes `'M'`, and the function stores the value.

The call with `"91O5225800087"` stops in step 2 at `i` equal to 2, where the character is the letter `O`. The call with `"9113225800087"` stops in step 4, because the month is 13.

The output of `main.cpp` is:

```
Accepted: Thandi Mokoena, 29, F, ID: 9702140123081
Rejected 91052258000: ID number must have 13 digits
Rejected 91O5225800087: ID number must contain only digits
Rejected 9113225800087: Invalid birth month
Rejected 9105005800087: Invalid birth day
Rejected 9105220800087: ID number does not match gender
Accepted: Sipho Dlamini, 35, M, ID: 9105225800087
```

Common mistakes

- **Check the gender before the day.** An ID with a bad day and a gender mismatch then reports the gender message. The edge test uses `9101320800087` and expects `Invalid birth day`. The edge test rejected this change.
- **Accept day 0.** If the test is `day < 0`, the ID `9105005800087` passes the day check. The main output then changes. The main test rejected this change.
- **Skip the digit loop.** The helper then throws `Not a digit` for the ID `91O5225800087`. The question requires `ID number must contain only digits`.
- **Check only the characters that the helper reads.** The ID `910230580008X` has the letter at index 12. No helper call reads index 12, so the function would accept the ID. The edge test expects a rejection.
- **Compare `i` with `idNum.length()` without a cast.** The compiler warns about a signed/unsigned comparison. Under `-Wall -Werror` the build fails, and the mark is 0.
- **Reject day 30 in month 02.** The question says the function does not check how many days a month has. The edge test expects `9102305800087` to be accepted.

A1-03. Exceptions: `Employee::setIDNum` (the age must match the birth year) (100)

File: A_1.cpp

```
#include "Employee.h"

void Employee::setIDNum(string idNum) {
    if ((int)idNum.length() != 13) {
        throw Exception("ID number must have 13 digits");
    }
    int yy = extractIntFromString(idNum, 0, 2);
    int birthYear = 1900 + yy;
    if (yy <= 26) {
        birthYear = 2000 + yy;
    }
    if (age != 2026 - birthYear) {
        throw Exception("ID number does not match age");
    }
    char idGender = 'F';
    if (extractIntFromString(idNum, 6, 10) >= 5000) {
        idGender = 'M';
    }
    if (idGender != gender) {
        throw Exception("ID number does not match gender");
    }
    this->idNum = idNum;
}
```

Expected output of the given main.cpp

```
Accepted: Thandi Mokoena, 29, F, ID: 9702140123081
Accepted: Sipho Dlamini, 35, M, ID: 9105225800087
Rejected 8503305800089: ID number does not match gender
Accepted: Anna Botha, 41, F, ID: 8503300800089
Rejected 9001015800083: ID number does not match age
Accepted: Bongani Zulu, 30, M, ID: 9601015800083
```

What the function must do

The function `setIDNum` checks a new ID number against the employee. It checks the length, then the age that the birth year gives, then the gender digits. If all three checks pass, the function stores the ID number in `idNum`. If one check fails, the function throws an `Exception` with the message for that check and leaves `idNum` unchanged.

How the solution works

```
void Employee::setIDNum(string idNum) {
```

The parameter `idNum` has the same name as the member `idNum`. Inside the function, the plain name means the parameter. The member is written `this->idNum`. The word `this` is a pointer to the object on which the function was called.

Check 1: the length.

```
    if ((int)idNum.length() != 13) {
        throw Exception("ID number must have 13 digits");
    }
```

The function `length()` returns an unsigned number. The cast `(int)` makes both sides of the comparison `int`. The condition is true for any string that does not have 13 characters. This check comes first because the next steps read index 0 and 1, and later index 6 to 9. A string with fewer than 10 characters would make `extractIntFromString` throw `Invalid range` instead of the expected message.

Check 2: the age.

```
    int yy = extractIntFromString(idNum, 0, 2);
    int birthYear = 1900 + yy;
    if (yy <= 26) {
        birthYear = 2000 + yy;
```

```
    }
    if (age != 2026 - birthYear) {
        throw Exception("ID number does not match age");
    }
```

- The call reads index 0 and 1 (the end index 2 is not included) and returns the two digits as a number. If one of them is not a digit, the helper throws `Not a digit`. The solution does not catch it, so it goes to the caller.
- The variable `birthYear` starts as the 1900s year.
- The `if` condition `yy <= 26` follows the rule in the question. A value of 26 or less belongs to the 2000s. The value 26 must be included, so the test uses `<=` and not `<`. The value 27 gives 1927.
- The condition `age != 2026 - birthYear` compares the stored `age` of the employee with the age that the ID number gives. The year 2026 is the current year in this question. A difference means the ID number does not fit the employee, so the function throws.

Check 3: the gender.

```
    char idGender = 'F';
    if (extractIntFromString(idNum, 6, 10) >= 5000) {
        idGender = 'M';
    }
    if (idGender != gender) {
        throw Exception("ID number does not match gender");
    }
```

The letter starts as `'F'`. The gender digits at index 6 to 9 form a number. A value of 5000 or more means `'M'`. The second `if` compares this letter with the member `gender`. A difference makes the function throw.

Store the value.

```
    this->idNum = idNum;
```

This line comes last. The function changes the object only after all three checks pass.

The order of the checks matters. An ID that is wrong for both age and gender must report the age message. The edge test checks this order. The function allocates no memory, so there is nothing to free.

Trace

The call is `tryID(anna, "8503305800089")`. The employee Anna has `age` 41 and `gender` `'F'`.

Step	Code	Values	Result
1	length check	length is 13	continue
2	`extractIntFromString(idNum, 0, 2)`	characters `85`	`yy` is 85
3	`birthYear = 1900 + yy`	1900 + 85	`birthYear` is 1985
4	`yy <= 26`	85 <= 26 is false	`birthYear` stays 1985
5	`age != 2026 - birthYear`	41 != 41 is false	continue
6	`extractIntFromString(idNum, 6, 10)`	characters `5800`	value is 5800
7	`>= 5000`	5800 >= 5000 is true	`idGender` is `'M'`
8	`idGender != gender`	`'M'` != `'F'` is true	throw `Exception("ID number does not match gender")`

The function `tryID` catches the exception by reference and prints `Rejected 8503305800089: ID number does not match gender`. The next call uses `"8503300800089"`. Its gender digits are `0800`, so `idGender` stays `'F'` and the ID number is stored.

The output of `main.cpp` is:

```
Accepted: Thandi Mokoena, 29, F, ID: 9702140123081
Accepted: Sipho Dlamini, 35, M, ID: 9105225800087
Rejected 8503305800089: ID number does not match gender
```

```
Accepted: Anna Botha, 41, F, ID: 8503300800089
Rejected 9001015800083: ID number does not match age
Accepted: Bongani Zulu, 30, M, ID: 9601015800083
```

Common mistakes

- **Write `yy < 26`.** The year `26` then gives 1926 and age 100. An employee with age 0 and the ID `2601010001080` is rejected. The edge test rejected this change because its output differed.
- **Check the gender before the age.** An ID that is wrong in both ways then reports `ID number does not match gender`. The edge output expects `ID number does not match age` for `9901010001080`. The edge test rejected this change.
- **Use the wrong current year or the wrong direction.** The age is `2026 - birthYear`. If the code uses `birthYear - 2026` or another year, correct IDs are rejected and the output differs.
- **Catch the exception from `extractIntFromString`.** The caller then never receives `Not a digit`. The edge test expects the line `Rejected 0A01015001080: Not a digit`.
- **Store `idNum` before the checks.** A failed call then changes the object, and the printed ID is wrong. The last three lines of the edge output show the IDs of failed calls.

A1-04. Exceptions: `Employee::extractIntFromString` (100)

File: A_1.cpp

```
#include "Employee.h"

int Employee::extractIntFromString(string str, int start, int end) {
    if (start < 0 || start >= end || end > (int)str.length()) {
        throw Exception("Invalid range");
    }
    int result = 0;
    for (int i = start; i < end; i++) {
        if (str[i] < '0' || str[i] > '9') {
            throw Exception("Not a digit");
        }
        result = result * 10 + (str[i] - '0');
    }
    return result;
}
```

Expected output of the given main.cpp

```
"9702140123081" [0, 2) = 97
"9702140123081" [2, 4) = 2
"9702140123081" [6, 10) = 123
"12a4" [0, 4): Not a digit
"1234" [2, 2): Invalid range
Thandi Mokoena, 29, F, ID: 9702140123081
Sipho Dlamini, 35, M, ID: 9105225800087
```

What the function must do

The function `extractIntFromString` reads a part of a string and returns the number that the digits make. The part starts at index `start` and stops before index `end`. If the range is not valid, the function throws `Exception("Invalid range")`. If a character in the range is not a digit, the function throws `Exception("Not a digit")`. The function is `static`, so a caller can write `Employee::extractIntFromString(...)` without an `Employee` object.

How the solution works

```
int Employee::extractIntFromString(string str, int start, int end) {
```

The word `static` appears only in the class declaration in `Employee.h`. The definition in `A_1.cpp` does not repeat it.

Step 1: check the range.

```
    if (start < 0 || start >= end || end > (int)str.length()) {
        throw Exception("Invalid range");
    }
```

The three conditions are joined with `||`. The whole condition is true if at least one part is true.

- `start < 0` protects against a negative index. The call `str[-1]` reads memory outside the string.
- `start >= end` protects against an empty range or a reversed range, such as 3 to 1 or 2 to 2. The question requires `Invalid range` for these.
- `end > (int)str.length()` protects against a range that goes past the last character. The valid values of `end` are up to and including the length, because `end` itself is not read. The function `length()` returns an unsigned number. The cast `(int)` avoids a signed/unsigned comparison warning, which `-Wall -Werror` turns into an error.

This check comes before the loop. The loop reads `str[i]`. Without the check, the loop could read characters that do not exist. The check also decides which message the function gives. The input `"abc"` with range 0 to 5 gives `Invalid range` and not `Not a digit`.

Step 2: build the number.

```
    int result = 0;
```

```
    for (int i = start; i < end; i++) {
```

The variable `result` starts at 0. The loop visits index `start` up to `end - 1`. The condition `i < end` excludes `end`, as the question requires.

Step 3: check each character.

```
        if (str[i] < '0' || str[i] > '9') {
            throw Exception("Not a digit");
        }
```

The characters `'0'` to `'9'` have consecutive codes. A character below `'0'` or above `'9'` is not a digit. This includes letters, a space and `'-'`. The function checks only the characters in the range, so characters outside the range have no effect. The `throw` leaves the loop and the function at once.

Step 4: add the digit.

```
        result = result * 10 + (str[i] - '0');
```

The expression `str[i] - '0'` gives the value of the digit. For example, `'7' - '0'` is 7. Multiplying `result` by 10 moves the earlier digits one place to the left. Leading zeros add nothing, so `"0042"` from index 1 to 4 gives 42.

```
    return result;
```

The function returns the number. A range has at most 9 digits, so the value fits in an `int`.

The function allocates no memory, so nothing needs to be freed.

Trace

The call is `tryExtract("12a4", 0, 4)`.

Step	Code	Values	Result
1	range check	`start` 0, `end` 4, length 4	0 < 0 false, 0 >= 4 false, 4 > 4 false, continue
2	`result = 0`		`result` is 0
3	`i = 0`	`'1'` is a digit	`result = 0 * 10 + 1`, so 1
4	`i = 1`	`'2'` is a digit	`result = 1 * 10 + 2`, so 12
5	`i = 2`	`'a'` is greater than `'9'`	throw `Exception("Not a digit")`

The function never reaches `i = 3` and never returns. The `catch (Exception& e)` in `tryExtract` prints the message.

A successful call is `tryExtract("9702140123081", 6, 10)`. The characters at index 6 to 9 are `0`, `1`, `2`, `3`. The values of `result` are 0, 1, 12, 123. The function returns 123.

The call `tryExtract("1234", 2, 2)` fails at step 1, because `start >= end` is true.

The output of `main.cpp` is:

```
"9702140123081" [0, 2) = 97
"9702140123081" [2, 4) = 2
"9702140123081" [6, 10) = 123
"12a4" [0, 4): Not a digit
"1234" [2, 2): Invalid range
Thandi Mokoena, 29, F, ID: 9702140123081
Sipho Dlamini, 35, M, ID: 9105225800087
```

Common mistakes

- **Treat `end` as included.** If the loop uses `i <= end`, the function reads one more character. The call from 0 to 2 would give a number with three digits. The main test rejected this change because its output differed.
- **Return 0 for an empty range.** If the range check allows `start == end`, the call `tryExtract("1234", 2, 2)` prints a value and not `Invalid range`. The main test rejected this change.

- **Compare an `int` with `str.length()` without a cast.** The comparison of a signed number with an unsigned number gives a warning. Under `-Wall -Werror` the warning is an error, and the program does not compile. The mark is 0.
- **Check the digits before the range.** For `"abc"` with range 0 to 5, the loop meets the letter `'a'` first and throws `Not a digit`. The question requires `Invalid range`. For `"1234"` with range 0 to 5, the loop reads `str[4]`, which is after the last character. In C++98 this read is undefined behaviour. With g++ the read gives the end character `'\0'`, so the function throws `Not a digit` and not `Invalid range`. The edge test rejected this change because its output differed.
- **Write `static` again in `A_1.cpp`.** The compiler reports an error, because `static` is allowed only inside the class declaration.

A1-05. Exceptions: `Team::operator[]` throws for a bad index or an empty slot (100)

File: A_1.cpp

```
#include "Team.h"

const Employee& Team::operator[](int index) {
    // Check the range first, so that members[index] is never read outside the array.
    if (index < 0 || index >= size) {
        throw Exception("Index out of bounds");
    }
    if (members[index] == NULL) {
        throw Exception("No employee at this index");
    }
    return *members[index];
}
```

Expected output of the given main.cpp

```
[0] Thandi Mokoena, 29, F, ID: none
[1] Sipho Dlamini, 35, M, ID: none
[2] Caught: No employee at this index
[3] Caught: Index out of bounds
[-1] Caught: Index out of bounds
```

What the function must do

The operator `operator[]` of `Team` returns the employee stored in slot `index` of the array `members`. The return type is a reference, so the function has no way to return `NULL`. If the index is outside the array, the function throws `Exception("Index out of bounds")`. If the slot is empty, the function throws `Exception("No employee at this index")`.

How the solution works

```
const Employee& Team::operator[](int index) {
```

The return type `const Employee&` is a reference to an `Employee` that the caller cannot change. A reference is another name for an existing object. The function returns the employee that is already in the team. The function makes no copy.

The member `members` has the type `Employee**`. It points to an array of `Employee*` pointers. Each pointer is either the address of an employee or `NULL`. The slot of an empty position holds `NULL`. The member `size` is the number of slots.

Check 1: the range.

```
    if (index < 0 || index >= size) {
        throw Exception("Index out of bounds");
    }
```

- `index < 0` is true for a negative index.
- `index >= size` is true when the index is `size` or larger. The valid indexes are 0 to `size - 1`, so the index `size` is already outside the array.
- The two parts are joined with `||`. One true part is enough to throw.
- For a team with `size` 0, every index fails this check.

Check 2: the slot.

```
    if (members[index] == NULL) {
        throw Exception("No employee at this index");
    }
```

The condition is true when the slot exists but holds no employee. An empty slot occurs at the end of a team that is not full. It also occurs after `removeMember`, which deletes the employee and sets the slot to `NULL`. If the function returned `*members[index]` for a `NULL` pointer, the program would use the address 0 and crash.

The order of the two checks matters. The condition `members[index]` reads memory. With a bad index, this read is outside the array. Valgrind reports an invalid read. The range check must run first, so the function reads `members[index]` only for a valid index.

Return the employee.

```
    return *members[index];
```

The expression `members[index]` is a pointer. The star `*` follows the pointer and gives the employee. The function returns this employee as a reference. The team still owns the employee and deletes it in `~Team()`. The function allocates and deletes nothing.

The function throws the `Exception` object by value with `throw Exception(...)`. The caller catches it with `catch (Exception& ex)`. The `&` makes the catch use the thrown object without a copy.

Trace

The team in `main.cpp` has `size` 3. The calls to `addMember` fill slots 0 and 1. Slot 2 is `NULL`.

Call	`index < 0`	`index >= size`	`members[index]`	Result
`show(*team, 0)`	false	0 >= 3 false	address of Thandi	return Thandi
`show(*team, 1)`	false	1 >= 3 false	address of Sipho	return Sipho
`show(*team, 2)`	false	2 >= 3 false	`NULL`	throw `No employee at this index`
`show(*team, 3)`	false	3 >= 3 true	not read	throw `Index out of bounds`
`show(*team, -1)`	true	not evaluated	not read	throw `Index out of bounds`

For `-1`, the first part of `||` is true, so C++ does not evaluate the second part. The function `show` prints the employee when the call returns and prints `Caught:` plus the message when the call throws.

The output of `main.cpp` is:

```
[0] Thandi Mokoena, 29, F, ID: none
[1] Sipho Dlamini, 35, M, ID: none
[2] Caught: No employee at this index
[3] Caught: Index out of bounds
[-1] Caught: Index out of bounds
```

Common mistakes

- **Check the slot before the range.** For index 3 in a team of size 3, the code reads `members[3]`, which is outside the array. For index -1, it reads `members[-1]`, which is before the array. Valgrind reports `Invalid read of size 8` for both calls. The value read there is undefined. In the test run it was 0, so both lines printed `No employee at this index` instead of `Index out of bounds`. The main test rejected this change.
- **Write `index > size` in the range check.** The index `size` then passes the check and `members[size]` is read outside the array. Valgrind reports `Invalid read of size 8`. In the test run the line for index 3 printed `No employee at this index` instead of `Index out of bounds`. The main test rejected this change.
- **Skip the `NULL` check.** For an empty slot, `*members[index]` follows a `NULL` pointer. When the caller prints the employee, valgrind reports `Invalid read of size 8` and the program stops with a segmentation fault (SIGSEGV). The mark is 0.
- **Write `return members[index];`.** This returns a pointer, and the return type is a reference. The compiler reports an error.
- **Write `throw new Exception(...)`.** This throws a pointer. The clause `catch (Exception& ex)` does not match a pointer, so the exception is not caught. The program stops with `terminate called after throwing an instance of 'Exception*'`, and the mark is 0.
- **Use one message for both errors.** The question requires two different messages. With only `Index out of bounds`, the line for index 2 is wrong. With only `No employee at this index`, the lines for index 3 and -1 are wrong.

A1-06. Exceptions: `Department::operator[]` with two error messages (100)

File: A_1.cpp

```
#include "Department.h"

Team*& Department::operator[](int index) {
    if (index < 0) {
        throw Exception("Negative index");
    }
    if (index >= size) {
        throw Exception("Index too large");
    }
    // An empty slot is not an error: the caller gets a reference to the NULL pointer.
    return teams[index];
}
```

Expected output of the given main.cpp

```
Slot 1 is empty
Sales (cost R45000)
Team:
  Thandi Mokoena, 29, F, ID: none
  Sipho Dlamini, 35, M, ID: none
Team:
  Anna Botha, 41, F, ID: none
Caught: Negative index
Caught: Index too large
```

What the function must do

The operator `operator[]` of `Department` gives the caller access to slot `index` of the array `teams`. The return type `Team*&` is a reference to the pointer in the slot. The caller can read the pointer or store a new pointer in the slot. For a negative index, the function throws `Exception("Negative index")`. For an index that is `size` or larger, the function throws `Exception("Index too large")`. An empty slot is not an error.

How the solution works

```
Team*& Department::operator[](int index) {
```

The member `teams` has the type `Team**`. It points to an array of `Team*` pointers. The type `Team*` is a pointer to a `Team`. The type `Team*&` is a reference to such a pointer. A reference is another name for an existing variable. If the function returns a reference to `teams[index]`, the caller works on the slot itself and not on a copy.

This is why the statement `(*sales)[2] = new Team(1);` works. The call `(*sales)[2]` returns the slot. The assignment then stores the new address in the slot. If the function returned a plain `Team*`, the result would be a temporary copy. A copy cannot receive an assignment, so the compiler would report an error for this statement.

Check 1: a negative index.

```
    if (index < 0) {
        throw Exception("Negative index");
    }
```

A negative index is not a valid position in the array. The function throws before it reads or returns any slot. A `throw` stops the function and sends the `Exception` to the nearest matching `catch` in the caller.

Check 2: an index that is too large.

```
    if (index >= size) {
        throw Exception("Index too large");
    }
```

The valid indexes are 0 to `size - 1`. The test uses `>=` because the index `size` is already outside the array. For a department with `size` 0, every non-negative index fails this check. The two checks use two separate `if` statements because the two cases have two

different messages.

Return the slot.

```
    return teams[index];
```

The index is valid here, so the read is inside the array. The function does not check for `NULL`. The question says that an empty slot is not an error, and the caller may need to test it with `(*sales)[1] == NULL` or fill it.

The function allocates nothing and deletes nothing. The department still owns every team in its slots and deletes them in `~Department()`.

If an assignment such as `(*sales)[3] = extra;` throws, the assignment never happens. The department does not own `extra`. The caller must delete `extra`, and `main.cpp` does this in the `catch` block. Without that `delete`, valgrind reports a leak.

Trace

The department is `new Sales(3)`. It has `size` 3, and all three slots start as `NULL`. The slot 0 gets `north` through `addTeam`.

Step	Code in `main.cpp`	Check 1	Check 2	Effect
1	`(*sales)[2] = new Team(1);`	2 < 0 false	2 >= 3 false	returns slot 2, the assignment stores the new team
2	`(*sales)[2]->addMember(...)`	false	false	returns slot 2, which now holds a team, and adds Anna
3	`(*sales)[1] == NULL`	false	1 >= 3 false	returns slot 1, which is `NULL`, so the output is `Slot 1 is empty`
4	`(*sales)[-1];`	-1 < 0 true	not reached	throw `Negative index`
5	`(*sales)[3] = extra;`	3 < 0 false	3 >= 3 true	throw `Index too large`, `extra` is not stored

In step 5, the `catch` block prints the message and runs `delete extra;`. The final `delete sales;` frees the teams in slots 0 and 2. The cost line shows R45000, because the class `Sales` charges R15000 for each of the three employees.

The output of `main.cpp` is:

```
Slot 1 is empty
Sales (cost R45000)
Team:
  Thandi Mokoena, 29, F, ID: none
  Sipho Dlamini, 35, M, ID: none
Team:
  Anna Botha, 41, F, ID: none
Caught: Negative index
Caught: Index too large
```

Common mistakes

- **Use `index > size`.** The index `size` then passes the check, and the function returns `teams[size]`, which is outside the array. The statement `(*sales)[3] = extra;` then writes outside the array. Valgrind reports `Invalid write of size 8`. The program prints `Extra team added`, and the department never deletes `extra`, so valgrind also reports a leak (`definitely lost`). The main test rejected this change.
- **Throw for a `NULL` slot.** The call `(*sales)[2] = new Team(1);` needs the empty slot 2. If the function throws for `NULL`, nothing in `main` catches the exception at that statement. The program stops with `terminate called after throwing an instance of 'Exception'` before it prints anything, and the mark is 0. The main test rejected this change.
- **Return a local copy.** Code such as `Team* tmp = teams[index]; return tmp;` returns a reference to a variable that ends when the function ends. The compiler reports `reference to local variable returned`, and `-Werror` turns this warning into an error.
- **Return `*teams[index]` or print the team.** The return type is `Team*&`, so the function must return the pointer slot. The line `return *teams[index];` gives the compile error `invalid initialization of reference of type 'Team*&' from expression of type 'Team'`. The function must not print anything.

- **Use one message for both errors.** The question requires `Negative index` for a negative index and `Index too large` for a large index. With one message, one of the two lines `Caught: ...` in the output would be wrong.

A1-07. Exceptions: `Team::addMember` (NULL, already in the team, full) (100)

File: A_1.cpp

```
#include "Team.h"

void Team::addMember(Employee* employee) {
    if (employee == NULL) {
        throw Exception("Employee is NULL");
    }
    // Look at every slot first: the same pointer may sit after an empty slot.
    for (int i = 0; i < size; i++) {
        if (members[i] == employee) {
            throw Exception("Employee is already in the team");
        }
    }
    for (int i = 0; i < size; i++) {
        if (members[i] == NULL) {
            members[i] = employee;
            return;
        }
    }
    throw Exception("Team is full");
}
```

Expected output of the given main.cpp

```
Caught: Employee is already in the team
Caught: Team is full
Caught: Employee is NULL
Members: 2
Team:
  Thandi Mokoena, 29, F, ID: none
  Sipho Dlamini, 35, M, ID: none
```

What the function must do

The function `addMember` puts an `Employee*` into the first empty slot of the array `members`. After a successful call, the team owns the employee and deletes it in `~Team()`. The function throws an `Exception` in three cases: the pointer is `NULL`, the same pointer is already in the team, or no slot is empty. After a throw, the array is unchanged and the caller still owns the employee.

How the solution works

The member `members` has the type `Employee**`. It points to an array of `Employee*` pointers. A pointer holds the address of an object. An empty slot holds `NULL`. The member `size` is the number of slots.

Check 1: a NULL pointer.

```
    if (employee == NULL) {
        throw Exception("Employee is NULL");
    }
```

A `NULL` pointer points to no employee, so the question requires an exception for it. This check comes first, as the question requires. When the team is full and the pointer is `NULL`, the message must be `Employee is NULL` and not `Team is full`.

Check 2: the same pointer in any slot.

```
    for (int i = 0; i < size; i++) {
        if (members[i] == employee) {
            throw Exception("Employee is already in the team");
        }
    }
```

The loop visits every slot from 0 to `size - 1`. The condition `members[i] == employee` compares two addresses. It is true when the slot holds the same employee object. The question defines "already in the team" as the same address, not the same name or ID number.

The loop must visit every slot and must not stop at an empty slot. An empty slot can appear before the employee, because `removeMember` sets a slot to `NULL`. If the same pointer were stored twice, `~Team()` would run `delete` on the same address twice. This is a double delete, and valgrind reports it.

The duplicate check runs before the search for a free slot. The function must not store the pointer and then find out that it is a duplicate.

Check 3 and the store.

```
    for (int i = 0; i < size; i++) {
        if (members[i] == NULL) {
            members[i] = employee;
            return;
        }
    }
    throw Exception("Team is full");
```

The second loop looks for the lowest index with a `NULL` slot. The condition `members[i] == NULL` is true for an empty slot. The statement `members[i] = employee;` stores the pointer. The team now owns the employee. The `return` ends the function at once, so the function does not continue to the `throw`.

If the loop ends without a store, no slot was empty. The last line throws `Team is full`. For a team of size 0, the loop does not run, and the function throws `Team is full`.

Memory: the function allocates nothing and deletes nothing. It stores a pointer that the caller created with `new`. After a throw, `members` has not changed. The caller must delete the employee, as `main.cpp` does for `lebo`. Without that `delete`, valgrind reports a leak.

Trace

The call sequence is from `main.cpp`. The team has `size` 2. The arrays show the slots 0 and 1.

Call	Check 1	Check 2	Free slot search	Slots after the call
`addMember(thandi)`	not `NULL`	slots are `NULL`, no match	slot 0 is `NULL`, store	`[thandi, NULL]`
`addMember(thandi)`	not `NULL`	slot 0 matches	not reached	`[thandi, NULL]`, throw `Employee is already in the team`
`addMember(sipho)`	not `NULL`	no match	slot 0 is used, slot 1 is `NULL`, store	`[thandi, sipho]`
`addMember(lebo)`	not `NULL`	no match	no `NULL` slot, loop ends	`[thandi, sipho]`, throw `Team is full`
`addMember(NULL)`	`NULL`, throw	not reached	not reached	`[thandi, sipho]`, throw `Employee is NULL`

After the call with `lebo` throws, `main` runs `delete lebo;` in the `catch` block. The program prints `Members: 2` and the team. Then `delete team;` runs `~Team()`, which deletes `thandi` and `sipho`.

The output of `main.cpp` is:

```
Caught: Employee is already in the team
Caught: Team is full
Caught: Employee is NULL
Members: 2
Team:
  Thandi Mokoena, 29, F, ID: none
  Sipho Dlamini, 35, M, ID: none
```

Common mistakes

- **Stop the duplicate search at the first empty slot.** The edge test removes the employee at slot 1 and then adds `lebo`, which sits in slot 2. The search stops at slot 1 and misses `lebo`. The function stores `lebo` a second time in slot 1, so the team prints Lebo twice, and `~Team()` deletes it twice. Valgrind reports `Invalid read of size 8` and `Invalid free()`. The edge test rejected this change.

- **Check for a full team before the `NULL` check.** For a full team and a `NULL` argument, the output would say `Team is full`. The main test expects `Employee is NULL`. The main test rejected this change.
- **Store the pointer in a slot other than the first empty slot.** The question requires the first empty slot, the lowest index that is `NULL`. The program prints the slots in index order, so a different slot gives a different order.
- **Compare the names instead of the pointers.** The question defines a duplicate by address. A second employee with the same name would be refused wrongly. Also, if the loop calls `members[i]->getFirstName()` on an empty slot, it follows a `NULL` pointer. Valgrind reports `Invalid read of size 8`, and the program stops with a segmentation fault (SIGSEGV).
- **Catch the exception inside `addMember` or print a message.** The caller then never sees the error, and the output lines `Caught: ...` are missing. The `catch` block in `main` that runs `delete lebo;` also never runs, so valgrind reports a leak (`definitely lost`).
- **Delete the employee inside `addMember` before the throw.** The question says that the caller still owns the employee after a throw. A second `delete` in `main` is then a double delete. The main output still looks correct, but valgrind reports `Invalid read of size 8` and `Invalid free()`.

A1-08. Exceptions: `Team::assignIDs` catches each error and continues (100)

File: A_1.cpp

```
#include "Team.h"

int Team::assignIDs(string* ids, int n) {
    if (ids == NULL || n <= 0) {
        return 0;
    }
    int count = 0;
    for (int i = 0; i < n; i++) {
        cout << "Slot " << i << ": ";
        if (i >= size) {
            cout << "no such slot" << endl;
        } else if (members[i] == NULL) {
            cout << "empty" << endl;
        } else {
            cout << members[i]->getFirstName() << " " << members[i]->getSurName() << " -> ";
            // The try is inside the loop, so one bad ID does not stop the others.
            try {
                members[i]->setIDNum(ids[i]);
                cout << ids[i] << endl;
                count++;
            } catch (Exception& e) {
                cout << "Error: " << e.what() << endl;
            }
        }
    }
    return count;
}
```

Expected output of the given main.cpp

```
Slot 0: Thandi Mokoena -> 9702140123081
Slot 1: Sipho Dlamini -> Error: ID number does not match gender
Slot 2: empty
Slot 3: no such slot
IDs set: 1
Team:
  Thandi Mokoena, 29, F, ID: 9702140123081
  Sipho Dlamini, 35, M, ID: none
```

Hints

A1-08 hints

Try the question first. Read one hint at a time. The hints go from gentle to specific. They do not give the full answer.

Hint 1

Each ID number needs its own chance to fail. Think about where a `try` block must start and end so that, after a `catch`, the function still goes on with the next `i`.

Hint 2

Put the `try` and the `catch` **inside** the loop body. A `try` around the whole loop stops the loop at the first bad ID number.

Hint 3

The two cases that are not errors (`i >= size` and an empty slot) do not need a `try`. Check them first. Add 1 to the count only after `setIDNum` returns without throwing.

What the function must do

The function `assignIDs` gives the ID number `ids[i]` to the employee in slot `i` of the team. For each `i` from 0 to `n - 1`, the function prints one line. The function `setIDNum` can throw an `Exception`. The function `assignIDs` must catch each exception, print the message and continue with the next slot. The function returns the number of ID numbers that were set.

How the solution works

The parameter `string* ids` is a pointer to the first element of an array of `string` values. The expression `ids[i]` is the element at position `i`. The member `members` is an array of `Employee*` pointers. An empty slot holds `NULL`.

Guard for bad input.

```
    if (ids == NULL || n <= 0) {
        return 0;
    }
```

If `ids` is `NULL`, the array does not exist, and `ids[i]` would read from address 0. If `n` is 0 or negative, the loop has nothing to do. In both cases the function prints nothing and returns 0, as the question requires. The check comes before the loop, so the loop never reads `ids` when `ids` is `NULL`.

The counter and the loop.

```
    int count = 0;
    for (int i = 0; i < n; i++) {
        cout << "Slot " << i << ": ";
```

The variable `count` holds the number of ID numbers that were set. The loop runs once for each position in `ids`. Each pass starts the line with `Slot <i>:`. Every case prints this start, so the code prints it once, before the cases.

Case 1: no such slot.

```
        if (i >= size) {
            cout << "no such slot" << endl;
```

The array `ids` can be longer than the team. The valid slots are 0 to `size - 1`. This test comes first. The next test reads `members[i]`, and for `i >= size` that read would be outside the array. Valgrind reports such a read.

Case 2: an empty slot.

```
        } else if (members[i] == NULL) {
            cout << "empty" << endl;
```

The slot exists but holds no employee. The function must not call `getFirstName` on a `NULL` pointer, because that stops the program abnormally. This case is not an error and needs no `try`.

Case 3: an employee is present.

```
        } else {
            cout << members[i]->getFirstName() << " " << members[i]->getSurName() << " -> ";
```

The arrow `->` follows a pointer to a member. The function prints the name and the arrow first, before the call that can fail. If `setIDNum` throws, the name is already on the line, and the error text continues the same line.

```
            try {
                members[i]->setIDNum(ids[i]);
                cout << ids[i] << endl;
                count++;
            } catch (Exception& e) {
                cout << "Error: " << e.what() << endl;
            }
```

- A `try` block holds code that can throw. If the code throws, C++ jumps to the matching `catch` and skips the rest of the `try` block.
- The `catch (Exception& e)` clause catches an `Exception` by reference. The reference `e` is the thrown object, not a copy. The call `e.what()` gives the message.
- If `setIDNum` returns normally, the two lines after it run. The function prints the new ID number and adds 1 to `count`. If `setIDNum` throws, these two lines are skipped, so `count` counts only the successful calls.
- The `try` and `catch` are inside the loop body. After a `catch`, the loop goes on with the next `i`. A `try` around the whole loop would end the loop at the first error.

```
    return count;
```

The function returns the number of successful calls. No exception leaves `assignIDs`. The function allocates and deletes nothing.

Trace

The team in `main.cpp` has `size` 3. Slot 0 holds Thandi (`'F'`), slot 1 holds Sipho (`'M'`), and slot 2 is `NULL`. The array `ids` has four values, so `n` is 4.

i	i >= size	members[i]	setIDNum	Printed after Slot i:	count
0	0 >= 3 false	Thandi	`9702140123081` has 13 characters, gender digits `0123` give `'F'`, match	`Thandi Mokoena -> 9702140123081`	1
1	1 >= 3 false	Sipho	`9105220800087` has gender digits `0800`, give `'F'`, Sipho is `'M'`, throw	`Sipho Dlamini -> Error: ID number does not match gender`	1
2	2 >= 3 false	`NULL`	not called	`empty`	1
3	3 >= 3 true	not read	not called	`no such slot`	1

The function returns 1. Then `main` prints `IDs set: 1` and the team. Sipho still has no ID number, because `setIDNum` keeps the old value when it throws.

The output of `main.cpp` is:

```
Slot 0: Thandi Mokoena -> 9702140123081
Slot 1: Sipho Dlamini -> Error: ID number does not match gender
Slot 2: empty
Slot 3: no such slot
IDs set: 1
Team:
  Thandi Mokoena, 29, F, ID: 9702140123081
  Sipho Dlamini, 35, M, ID: none
```

Common mistakes

- **Put one `try` around the whole loop.** The first exception ends the loop. Slots 2 and 3 never print, and the output differs. The main test rejected this change.
- **Skip the `NULL` check.** For slot 2, the call `members[2]->getFirstName()` uses a `NULL` pointer. Valgrind reports `Invalid read of size 8`, and the program stops with a segmentation fault (SIGSEGV) after it prints `Slot 2:`. The main test rejected this change.
- **Test `members[i]` before `i >= size`.** For `i` equal to 3, the code reads `members[3]`, outside the array. Valgrind reports `Invalid read of size 8`. The value read there is undefined. In the test run it was 0, so the line printed `Slot 3: empty` instead of `Slot 3: no such slot`.
- **Add 1 to `count` before `setIDNum`, or in the `catch`.** The count then includes failed calls. The line `IDs set: 1` would show `IDs set: 2`.
- **Skip the guard for a `NULL` array.** The call `assignIDs(NULL, 3)` then reads `ids[0]` from address 0, which is not a valid array. This is undefined behaviour. In the edge test, valgrind reports `Invalid read of size 8`, and the program stops with a segmentation fault (SIGSEGV).
- **Let an exception leave the function.** If a `catch` clause names another type, such as `catch (string& e)`, the `Exception` is not caught. The program stops with `terminate called after throwing an instance of 'Exception'`, and the mark is 0.

A1-09. Exceptions: `Team::setMemberID` catches two exception types (100)

File: A_1.cpp

```
#include "InvalidIDException.h"
#include "Team.h"

bool Team::setMemberID(int index, string idNum) {
    try {
        Employee* employee = getMember(index);
        employee->setIDNum(idNum);
        cout << "Member " << index << ": ID set to " << idNum << endl;
        return true;
    } catch (InvalidIDException& e) {
        // The most derived type must come first.
        cout << "Member " << index << ": bad ID [" << e.getBadID() << "]: " << e.what() << endl;
    } catch (Exception& e) {
        cout << "Member " << index << ": " << e.what() << endl;
    }
    return false;
}
```

Expected output of the given main.cpp

```
Member 0: ID set to 9702140123081
Member 1: bad ID [91052258000]: ID number must have 13 digits
Member 1: bad ID [9105225800O87]: ID number must contain only digits
Member 1: ID number does not match gender
Member 2: No member at this index
Member 1: ID set to 9105225800087
IDs set: 2
Team:
  Thandi Mokoena, 29, F, ID: 9702140123081
  Sipho Dlamini, 35, M, ID: 9105225800087
```

Hints

A1-09 hints

Try the question first. Read one hint at a time. The hints go from gentle to specific. They do not give the full answer.

Hint 1

An `InvalidIDException` object is also an `Exception` object. So a `catch (Exception& e)` block accepts both types.

Hint 2

When an exception is thrown, C++ tries the `catch` blocks from top to bottom and uses the first one that fits. Ask yourself which of your two blocks would take an `InvalidIDException`.

Hint 3

The `catch` block for the derived class must come before the `catch` block for its base class. In the opposite order, `-Wall -Werror` stops the build with "exception of type 'InvalidIDException' will be caught by earlier handler".

What the function must do

The function `setMemberID` sets the ID number of the member in slot `index`. Two kinds of exception can occur. The class `InvalidIDException` is derived from `Exception`, so an `InvalidIDException` is also an `Exception`. The function must catch both kinds. It prints one line for each case and returns `true` for success or `false` for any exception. No exception may leave the function.

How the solution works

```
bool Team::setMemberID(int index, string idNum) {
    try {
        Employee* employee = getMember(index);
```

```
        employee->setIDNum(idNum);
        cout << "Member " << index << ": ID set to " << idNum << endl;
        return true;
```

A `try` block holds code that can throw. If a statement throws, C++ skips the rest of the block and looks for a matching `catch`.

- The given function `getMember(index)` returns an `Employee*`, which is the address of the employee. It throws a plain `Exception("No member at this index")` for a bad index or an empty slot. A bad index or an empty slot is therefore handled by this function, and `setMemberID` does not test the slot itself.
- The arrow `employee->setIDNum(idNum)` calls the function on the employee that the pointer holds. This call can throw an `InvalidIDException` (wrong length or a non-digit) or a plain `Exception` (the gender does not match).
- The `cout` line and `return true;` come after the call. They run only if no exception occurred. If the line came before the call, a failed call would also print `ID set`.

```
    } catch (InvalidIDException& e) {
        // The most derived type must come first.
        cout << "Member " << index << ": bad ID [" << e.getBadID() << "]: " << e.what() << endl;
```

A `catch (InvalidIDException& e)` clause catches an `InvalidIDException`. The `&` makes `e` a reference to the thrown object, so C++ does not copy it. The call `e.getBadID()` gives the rejected ID number. The call `e.what()` gives the message, because `InvalidIDException` passes its message to the constructor of `Exception`. The function `getBadID` exists only in `InvalidIDException`, so this handler can use it.

```
    } catch (Exception& e) {
        cout << "Member " << index << ": " << e.what() << endl;
    }
```

This handler catches every other `Exception`: the gender mismatch from `setIDNum` and the error from `getMember`. It prints only the message.

```
    return false;
}
```

After a handler runs, the function continues after the last `catch` block. The line `return false;` runs for both handlers. The function also needs this line to return a value on every path.

The order of the handlers. C++ tests the handlers from top to bottom and uses the first one that fits. An `InvalidIDException` fits both `catch (InvalidIDException&)` and `catch (Exception&)`, because the derived class is also its base class. If the base handler is first, it takes every `InvalidIDException`, and the derived handler never runs. The compiler reports that the derived exception "will be caught by earlier handler". With `-Wall -Werror`, this warning is an error. The derived handler must therefore come first.

The function allocates and frees nothing. The employee belongs to the team.

Trace

The team has `size` 3. Slot 0 holds Thandi, slot 1 holds Sipho (`'M'`), and slot 2 is empty. The table shows four calls from `main.cpp`.

Call	`getMember`	`setIDNum`	Exception type	Handler used	Printed	Return
`setMemberID(1, "91052258000")`	returns Sipho	length is 11, not 13, throws `InvalidIDException`	`InvalidIDException`	first	`Member 1: bad ID [91052258000]: ID number must have 13 digits`	`false`
`setMemberID(1, "9105220800087")`	returns Sipho	form is valid, digits `0800` give `'F'`, Sipho is `'M'`, throws `Exception`	`Exception`	second	`Member 1: ID number does not match gender`	`false`
`setMemberID(2, "8503300800089")`	throws `Exception("No`	not called	`Exception`	second	`Member 2: No member at this index`	`false`

Call	getMember	setIDNum	Exception type	Handler used	Printed	Return
	member at this index")					
setMemberID(1, "9105225800087")	returns Sipho	digits 5800 give 'M', stored	none	none	Member 1: ID set to 9105225800087	true

For the second call, an `Exception` is not an `InvalidIDException`, so the first handler does not fit. C++ moves on to the second handler. For the third call, `setIDNum` is never reached, because `getMember` throws first.

In `main.cpp`, the calls for index 0 and the last call for index 1 return `true`. The value of `count` is 2.

The output of `main.cpp` is:

```
Member 0: ID set to 9702140123081
Member 1: bad ID [91052258000]: ID number must have 13 digits
Member 1: bad ID [9105225800O87]: ID number must contain only digits
Member 1: ID number does not match gender
Member 2: No member at this index
Member 1: ID set to 9105225800087
IDs set: 2
Team:
  Thandi Mokoena, 29, F, ID: 9702140123081
  Sipho Dlamini, 35, M, ID: 9105225800087
```

Common mistakes

- **Put `catch (Exception& e)` before `catch (InvalidIDException& e)`.** The compiler reports "exception of type 'InvalidIDException' will be caught by earlier handler". Under `-Wall -Werror` the build fails, and the mark is 0. The main test rejected this change at build time.
- **Catch only `Exception&`.** The code compiles, but an `InvalidIDException` prints as `Member 1: ID number must have 13 digits`. The text `bad ID [...]` is missing, and the output differs. The main test rejected this change.
- **Print `ID set` before the call to `setIDNum`.** A failed call then prints two lines for one call.
- **Leave out the final `return false;`.** The function then has a path that ends without a return value. Under `-Wall -Werror`, the warning `control reaches end of non-void function` is an error.
- **Let a handler return `true`.** The variable `count` in `main.cpp` then counts failed calls, and `IDs set: 2` would show a larger number.
- **Catch nothing for the `getMember` call.** The exception for an empty slot leaves the function. The program stops abnormally, and the mark is 0.

A1-10. Exceptions: `Team::hire` frees memory before it throws again (100)

File: A_1.cpp

```
#include "Team.h"

void Team::hire(string firstName, string surName, int age, char gender, string idNum) {
    Employee* newEmployee = new Employee(firstName, surName, age, gender);
    try {
        newEmployee->setIDNum(idNum);
        addMember(newEmployee);
    } catch (Exception& e) {
        // The team did not take the employee, so it is freed here.
        delete newEmployee;
        throw e;  // "throw;" also works: it throws the same exception again
    }
}
```

Expected output of the given main.cpp

```
Hired Thandi Mokoena
Caught: ID number does not match gender
Hired Sipho Dlamini
Caught: Team is full
Team:
  Thandi Mokoena, 29, F, ID: 9702140123081
  Sipho Dlamini, 35, M, ID: 9105225800087
```

Hints

A1-10 hints

Try the question first. Read one hint at a time. The hints go from gentle to specific. They do not give the full answer.

Hint 1

Only steps 2 and 3 can throw. When one of them throws, which variable still points to the new employee, and who owns it at that moment?

Hint 2

Put steps 2 and 3 in one `try` block. When the `catch (Exception& e)` block runs, the team does not hold the new employee, so `hire` is its only owner.

Hint 3

A `catch` block can pass the exception on: `throw;` throws the caught exception again, and `throw e;` throws a copy of it. Free the memory before that line, because nothing after a `throw` runs.

What the function must do

The function `hire` makes a new `Employee` with `new`, sets its ID number and adds it to the team. Two calls can throw an `Exception`: `setIDNum` and `addMember`. If one of them throws, the team does not keep the new employee, so `hire` must free it with `delete`. Then `hire` must throw the same error again, so the caller gets the same message.

How the solution works

```
void Team::hire(string firstName, string surName, int age, char gender, string idNum) {
    Employee* newEmployee = new Employee(firstName, surName, age, gender);
```

The operator `new` allocates an `Employee` on the heap and returns its address. The pointer `newEmployee` holds the address. Memory from `new` stays allocated until the program runs `delete` on it. At this line, `hire` is the only owner of the new employee.

The pointer is declared before the `try` block. A name declared inside a `try` block is not visible in the `catch` block. The `catch` block needs `newEmployee` to free the memory.

```
    try {
        newEmployee->setIDNum(idNum);
        addMember(newEmployee);
```

The order of the two calls is fixed by the question. The ID number is checked first. If the ID number is bad and the team is full, the caller must receive the ID number message. If the first call throws, C++ skips the second call, so `addMember` does not run. If the first call works, `addMember` runs. It stores the pointer in the first empty slot, or it throws `Exception("Team is full")` and the team keeps nothing.

If both calls return normally, the team owns the new employee, and `~Team()` deletes it later. The function `hire` must not delete the employee on this path. A second `delete` would be a double delete.

```
    } catch (Exception& e) {
```

This clause catches an `Exception` thrown by either call. The `&` makes `e` a reference to the thrown object. At this point the team does not hold the new employee, so `hire` is still its only owner.

```
        delete newEmployee;
```

This line frees the employee. Without it, the memory is never freed, and valgrind reports a leak. The `delete` comes before the `throw`. A `throw` leaves the function at once, and no statement after it runs.

```
        throw e;  // "throw;" also works: it throws the same exception again
    }
}
```

This line sends the error on to the caller. The caller of `hire` has its own `try` and `catch` and prints the message.

- The statement `throw e;` throws a copy of `e`. The copy has the type `Exception` and the same message.
- The statement `throw;` without an object throws the exception that is currently being handled, with no copy. It also keeps the real type of the object, which matters if a class derived from `Exception` is thrown. In this question, the given functions throw plain `Exception` objects, so both statements give the same result.
- The function must not print the message or return normally here. If it returned normally, the caller would think the hire worked.

Trace

The call is the fourth `tryHire` in `main.cpp`: `hire("Lebo", "Nkosi", 25, 'F', "0101014999081")`. The team has `size` 2 and already holds Thandi and Sipho.

Step	Code	Values	Result
1	`new Employee(...)`	Lebo, `'F'`, no ID number	`newEmployee` points to a new object
2	`setIDNum("0101014999081")`	length 13, digits `4999` give `'F'`, match	the ID number is stored in the new employee
3	`addMember(newEmployee)`	slot 0 and slot 1 are not `NULL`	throws `Exception("Team is full")`
4	`catch (Exception& e)`	`e.what()` is `Team is full`	the handler runs
5	`delete newEmployee;`		the new employee is freed
6	`throw e;`		the exception leaves `hire`
7	`catch` in `tryHire`		prints `Caught: Team is full`

The second `tryHire` call fails earlier. For Sipho with `"9105220800087"`, step 2 throws `ID number does not match gender`, and `addMember` never runs. The handler still frees the new employee. The third call, for Sipho with `"9105225800087"`, passes both steps. The team stores Sipho, and `hire` frees nothing.

The output of `main.cpp` is:

```
Hired Thandi Mokoena
Caught: ID number does not match gender
Hired Sipho Dlamini
Caught: Team is full
Team:
```

```
Thandi Mokoena, 29, F, ID: 9702140123081
Sipho Dlamini, 35, M, ID: 9105225800087
```

Common mistakes

- **Leave out the `delete`.** Each failed hire loses one `Employee`. Valgrind reports a leak. The main test rejected this change through valgrind.
- **Free the employee but do not throw again.** The function then returns normally after a failure. The caller prints `Hired Sipho Dlamini` and `Hired Lebo Nkosi` for hires that failed, and the output differs. The main test rejected this change.
- **Put `delete` after the `throw`.** The `throw` ends the function, so the `delete` never runs. This has the same result as no `delete`: a leak.
- **Call `addMember` before `setIDNum`.** For a full team and a bad ID number, the caller then gets `Team is full`. The question requires the ID number message. The edge test uses a team of size 0 and the ID number `"123"`. It rejected this change. There is a second problem. When `addMember` works and `setIDNum` then throws, the `catch` block deletes an employee that the team still holds. Valgrind reports `Invalid read` and `Invalid free()`.
- **Delete the employee on the success path.** The team still holds the pointer to the freed memory. Printing the team reads the freed employee, and `~Team()` deletes it again. Valgrind reports `Invalid read` and `Invalid free()`, even though the printed output can still look correct.
- **Declare `newEmployee` inside the `try` block.** The `catch` block cannot see the name, and the compiler reports an error.

A_2 Operator overloading: solutions

A2-01. Operator overloading: `operator<<` for `Employee` (100)

File: A_2.cpp

```
#include "Employee.h"

ostream& operator<<(ostream& out, const Employee& e) {
    // e is const and the getters are not const, so a friend reads the members directly.
    out << e.firstName << " " << e.surName << ", " << e.age << ", " << e.gender << ", ID: ";
    if (e.idNum == "") {
        out << "none";
    } else {
        out << e.idNum;
    }
    return out;
}
```

Expected output of the given main.cpp

```
Thandi Mokoena, 29, F, ID: 9702140123081
Sipho Dlamini, 35, M, ID: none
Thandi Mokoena, 29, F, ID: 9702140123081 | Sipho Dlamini, 35, M, ID: none
```

What the function must do

The function prints one `Employee` as a single line of text. It does not print a newline. The text has the first name, surname, age, gender and ID number.

If the employee has no ID number yet, the function prints `none` in place of the ID. The function writes to the stream it receives and returns that stream, so calls can be chained.

How the solution works

The full solution is short.

```
ostream& operator<<(ostream& out, const Employee& e) {
    // e is const and the getters are not const, so a friend reads the members directly.
    out << e.firstName << " " << e.surName << ", " << e.age << ", " << e.gender << ", ID: ";
    if (e.idNum == "") {
        out << "none";
    } else {
        out << e.idNum;
    }
    return out;
}
```

Signature. `ostream& operator<<(ostream& out, const Employee& e)` is an overloaded operator. It tells the compiler what `cout << someEmployee` means.

- `ostream& out` is a reference. A reference is another name for an existing object. The function writes into the real stream, not a copy. A copy of a stream is not allowed.
- `const Employee& e` is a reference to an `Employee` that the function must not change. No copy of the employee is made.
- The word `friend` appears only in `Employee.h`. A friend function may read private members such as `e.firstName`. The function in `A_2.cpp` must not repeat `friend`.

Why the members are read directly. The parameter `e` is `const`. The getters such as `getFirstName()` are not `const` member functions. The compiler refuses to call a non-`const` function on a `const` object. The friend function can read `e.firstName` directly, so it does not need the getters.

The first `out <<` line. The line `out << e.firstName << " " << e.surName << ", " << e.age << ", " << e.gender << ", ID: ";` prints everything before the ID.

- The function uses `out`, not `cout`. The marking program may pass a different stream, for example a string stream. Text sent to `cout` would never reach that stream.
- Each `<<` returns the stream, so the next `<<` can follow in the same statement.
- The text `", ID: "` ends with a space. The format needs one space after `ID:`.

The `if` condition. `if (e.idNum == "")` checks whether the ID number is still the empty string. The constructor sets `idNum` to `""`. Only `setIDNum` changes it.

- If the condition is true, the function prints `none`. This protects the case of an employee with no ID.
- If the condition is false, the `else` branch prints the stored ID number.
- Without this check, an employee without an ID would print `ID:` followed by nothing.

The return. `return out;` gives the same stream back to the caller. This is the reason that `cout << *thandi << " | " << *sipho << endl;` works. The expression `cout << *thandi` returns `cout`, and the next `<<` continues on it. The function must not print `endl`, because `main` adds it.

The function allocates no memory. There is nothing to free, so valgrind reports no leak from this function.

Trace

The call comes from the third print statement in `main.cpp`.

```
cout << *thandi << " | " << *sipho << endl;
```

The program reads this from left to right. The `Employee` Thandi has the ID `9702140123081`. The `Employee` Sipho has no ID.

Step	Call	Check `e.idNum == ""`	Text added to the stream	Value returned
1	`operator<<(cout, *thandi)`	false (the ID is set)	`Thandi Mokoena, 29, F, ID:` then `9702140123081`	`cout`
2	`cout << " | "`	not applicable	`|`	`cout`
3	`operator<<(cout, *sipho)`	true (the ID is empty)	`Sipho Dlamini, 35, M, ID:` then `none`	`cout`
4	`cout << endl`	not applicable	a newline	`cout`

The program output for the three print statements in `main.cpp` is:

```
Thandi Mokoena, 29, F, ID: 9702140123081
Sipho Dlamini, 35, M, ID: none
Thandi Mokoena, 29, F, ID: 9702140123081 | Sipho Dlamini, 35, M, ID: none
```

Common mistakes

1. **Writing to `cout` instead of `out`.** The output looks correct in `main.cpp`, because `main` uses `cout`. The edge test sends the employee to an `ostringstream`. The text appears on the screen instead, and the string stream is empty. The output differs from the expected output.
2. **Printing `endl` inside the function.** Every employee line then ends with an extra newline. The expected output of `main.cpp` differs, because `main` already adds `endl`.
3. **Calling the getters on `e`.** An expression such as `e.getFirstName()` does not compile. The reference `e` is `const`, and the getter is not a `const` member function. A compile error gives the mark 0.
4. **Forgetting the empty ID case.** The function then prints `ID:` with nothing after it. The expected line for Sipho is `ID: none`, so the output differs.

A2-02. Operator overloading: `operator<<` for `Team` (skips empty slots) (100)

File: A_2.cpp

```
#include "Team.h"

ostream& operator<<(ostream& out, const Team& t) {
    // t is const, so getMemberCount() and operator[] (not const) cannot be called. Use the members.
    int count = 0;
    for (int i = 0; i < t.size; i++) {
        if (t.members[i] != NULL) {
            count++;
        }
    }
    out << "Team (" << count << "/" << t.size << "):" << endl;
    if (count == 0) {
        out << "  (no members)" << endl;
    }
    for (int i = 0; i < t.size; i++) {
        if (t.members[i] != NULL) {
            out << "  " << *t.members[i] << endl;
        }
    }
    return out;
}
```

Expected output of the given main.cpp

```
Team (2/3):
  Thandi Mokoena, 29, F, ID: 9702140123081
  Sipho Dlamini, 35, M, ID: none
Team (1/3):
  Sipho Dlamini, 35, M, ID: none
Team (0/2):
  (no members)
```

What the function must do

The function prints a heading line for a `Team`, then one line for each employee. The heading shows how many slots hold an employee and how many slots the team has.

Some slots can be empty. An empty slot holds `NULL`. The function must not print or dereference these slots. If the team has no employees, the function prints one line with `(no members)`.

How the solution works

```
ostream& operator<<(ostream& out, const Team& t) {
    // t is const, so getMemberCount() and operator[] (not const) cannot be called. Use the members.
    int count = 0;
    for (int i = 0; i < t.size; i++) {
        if (t.members[i] != NULL) {
            count++;
        }
    }
    out << "Team (" << count << "/" << t.size << "):" << endl;
    if (count == 0) {
        out << "  (no members)" << endl;
    }
    for (int i = 0; i < t.size; i++) {
        if (t.members[i] != NULL) {
            out << "  " << *t.members[i] << endl;
        }
    }
    return out;
}
```

Signature. `ostream& operator<<(ostream& out, const Team& t)` is an overloaded operator. It defines what `cout << aTeam` does.

- `out` is a reference to the output stream. The function writes into the caller's stream and returns it.

- `t` is a `const` reference. The function may read the team but may not change it.
- The function is a `friend` in `Team.h`. Because of this, it may read the private members `t.members` and `t.size`. The word `friend` is not repeated in `A_2.cpp`.

Why the function counts by itself. The functions `getMemberCount()` and `operator[]` are not `const` member functions. A `const` object cannot call them. The function reads `t.members` and `t.size` directly instead.

The data. `t.members` is an array of pointers to `Employee`. A pointer holds the address of an object. `NULL` means the pointer holds no address. The `Team` constructor sets every slot to `NULL`. A slot becomes non-`NULL` only when `addMember` stores an employee there. The function `removeMember` deletes the employee and sets the slot back to `NULL`.

First loop: count the employees.

- `for (int i = 0; i < t.size; i++)` visits every slot from index 0 to `t.size - 1`. The loop stops before `t.size`, because index `t.size` is outside the array.
- `if (t.members[i] != NULL)` checks that the slot holds an employee. Only then does the function add 1 to `count`. The loop counts slots that are not empty, not the array length.
- If the team has size 0, the loop body never runs and `count` stays 0.

Heading. The statement `out << "Team (" << count << "/" << t.size << "):" << endl;` prints the heading. The counted number goes first, then the array length. The function prints the heading before the `if`, so every team gets a heading.

The empty case. `if (count == 0)` checks that no slot holds an employee. A team of size 0 and a team with only `NULL` slots both give `count` equal to 0. The function then prints the line `(no members)`, with two spaces before the text. The second loop prints nothing for such a team.

Second loop: print the employees.

- The loop runs through the slots in the same order, from index 0 upward. This gives slot order in the output.
- `if (t.members[i] != NULL)` protects the dereference. The expression `*t.members[i]` follows the pointer to the `Employee`. If the slot is `NULL`, the dereference is undefined behaviour and the program can crash.
- `out << " " << *t.members[i] << endl;` prints two spaces, then the employee, then a newline. The employee is printed by the given `operator<<` for `Employee`, which prints no newline. The function adds `endl` itself, so every line ends with a newline, including the last one.

Return. `return out;` returns the stream so the caller can chain more output after the team.

The function allocates no memory and frees no memory. The `Team` destructor frees the employees. The function only reads the data.

Trace

The call is the second `cout << *team;` in `main.cpp`. Before this call, `main` put Thandi in slot 0 and Sipho in slot 1. Then `main` called `team->removeMember(0)`. The array has size 3 and now holds:

Slot	0	1	2
Content	`NULL`	Sipho	`NULL`

First loop (counting):

`i`	`t.members[i] != NULL`	`count` after this step
0	false	0
1	true	1
2	false	1

The heading prints `Team (1/3):`. The check `count == 0` is false, so the function skips the `(no members)` line.

Second loop (printing):

`i`	`t.members[i] != NULL`	Output
0	false	nothing
1	true	`Sipho Dlamini, 35, M, ID: none` and a newline
2	false	nothing

The output of this call is the second block in the ExpectedOutput of `main.cpp`:

```
Team (1/3):
  Sipho Dlamini, 35, M, ID: none
```

Common mistakes

1. **Not skipping `NULL` slots.** If the second loop dereferences every slot, the function dereferences `NULL` after `removeMember`. This is undefined behaviour. In `main`, the first team prints correctly. The second print reaches the `NULL` in slot 0. Valgrind reports `Invalid read of size 8`, and the program stops with a segmentation fault (`SIGSEGV`). The rest of the expected output is missing.
2. **Printing the array length as the count.** The heading then shows `Team (3/3):` where `Team (1/3):` is expected. The output of `main` then differs from the expected output.
3. **Calling `t.getMemberCount()` or `t[i]`.** The reference `t` is `const`. These functions are not `const` member functions, so the code does not compile. A compile error gives the mark 0.
4. **Leaving out the `(no members)` line.** A team with only `NULL` slots then prints a heading with nothing below it. The expected output for `Team (0/2):` has a second line, so the output differs.

A2-03. Operator overloading: `operator<<` for `Department` (virtual calls) (100)

File: A_2.cpp

```
#include "Department.h"

ostream& operator<<(ostream& out, Department& d) {
    // getName() and costToCompany() are virtual, so a Sales object prints as Sales.
    out << "Department: " << d.getName() << " (cost R" << d.costToCompany() << ")" << endl;
    bool anyTeam = false;
    for (int i = 0; i < d.size; i++) {
        if (d.teams[i] != NULL) {
            out << *d.teams[i];
            anyTeam = true;
        }
    }
    if (!anyTeam) {
        out << "  (no teams)" << endl;
    }
    return out;
}
```

Expected output of the given main.cpp

```
Department: Development (cost R50000)
Team (2/2):
  Thandi Mokoena, 29, F, ID: none
  Sipho Dlamini, 35, M, ID: none
Department: HR (cost R12000)
Team (1/1):
  Anna Botha, 41, F, ID: none
```

What the function must do

The function prints a heading line for a `Department`, then prints every team in the department. The heading holds the department name and the cost to the company.

The function must print the name and cost of the real department type, such as `Development` or `HR`. If the department holds no team, the function prints one line with `(no teams)`.

How the solution works

```
ostream& operator<<(ostream& out, Department& d) {
    // getName() and costToCompany() are virtual, so a Sales object prints as Sales.
    out << "Department: " << d.getName() << " (cost R" << d.costToCompany() << ")" << endl;
    bool anyTeam = false;
    for (int i = 0; i < d.size; i++) {
        if (d.teams[i] != NULL) {
            out << *d.teams[i];
            anyTeam = true;
        }
    }
    if (!anyTeam) {
        out << "  (no teams)" << endl;
    }
    return out;
}
```

Signature. `ostream& operator<<(ostream& out, Department& d)` is an overloaded operator for `cout << aDepartment`.

- `out` is a reference to the output stream. The function writes to it and returns it.
- `d` is a reference to a `Department`. It is not `const`, because `getName()` and `costToCompany()` are not `const` member functions.
- The function is a `friend` in `Department.h`, so it may read the private members `d.teams` and `d.size`.

Virtual calls. `Sales`, `Marketing`, `HR` and `Development` are subclasses of `Department`. A friend function is not inherited, so only this one operator exists. A `Development` object reaches the operator as a `Department&`.

- `getName()` and `costToCompany()` are `virtual`. A `virtual` function runs the version of the real object, even when the reference has the type `Department&`.
- The call `d.getName()` returns `"Development"` for a `Development` object. The call `d.costToCompany()` returns the cost that the `Development` class calculates.
- Without `virtual`, the base versions would run. The base `getName()` returns `"Department"` and the base `costToCompany()` returns 0.

Heading. The first `out <<` statement prints `Department:`, the name, `(cost R`, the cost and `)`. The cost is a `float`. With the normal stream format, `50000.0f` prints as `50000` with no decimals. The question states that all costs stay below one million.

The flag. `bool anyTeam = false;` records whether the function has found a team. The function cannot decide this from `d.size` alone. A department of size 2 can have two `NULL` slots.

The loop.

- `for (int i = 0; i < d.size; i++)` visits slots 0 up to `d.size - 1`, in order. A department of size 0 never enters the loop body.
- `if (d.teams[i] != NULL)` checks that the slot holds a team. `d.teams` is an array of pointers to `Team`. An empty slot holds `NULL`. Dereferencing `NULL` is undefined behaviour, so the check comes before `*d.teams[i]`.
- `out << *d.teams[i];` follows the pointer and prints the team with the given `operator<<` for `Team`. That operator prints its own `endl`, so the function adds none here.
- `anyTeam = true;` runs only for a slot that holds a team. The flag changes after the team prints.

The `(no teams)` check. `if (!anyTeam)` is true when the loop found no team. This covers a department of size 0 and a department whose slots are all `NULL`. The function prints `(no teams)` with two spaces and an `endl`.

A team with zero employees still counts as a team. The flag becomes true, and the team prints its own `(no members)` line. The edge test checks this case. A version that tests `d.size == 0` instead of the flag fails the edge test.

Return. `return out;` returns the stream for chaining.

The function allocates and frees nothing. The `Department` destructor deletes the teams.

Trace

The call is `cout << *hr;` in `main.cpp`. The object `hr` is a pointer to `HR` stored as `Department*`. Its size is 3. Slot 0 holds the team `people` with one employee, Anna. Slots 1 and 2 are `NULL`.

Step	Action	Result
1	`d.getName()`	virtual call runs `HR::getName`, gives `HR`
2	`d.costToCompany()`	virtual call runs `HR::costToCompany`, gives 1 employee times 12000, which is `12000`
3	Print heading	`Department: HR (cost R12000)` and a newline
4	`anyTeam = false`	`anyTeam` is false

`i`	`d.teams[i] != NULL`	Action	`anyTeam` after
0	true	print the team `people`: `Team (1/1):` then `Anna Botha, 41, F, ID: none`	true
1	false	none	true
2	false	none	true

After the loop, `!anyTeam` is false, so the function does not print `(no teams)`. The output of this call is the second block of the ExpectedOutput in `main.cpp`:

```
Department: HR (cost R12000)
Team (1/1):
  Anna Botha, 41, F, ID: none
```

Common mistakes

1. **Calling the base versions.** Code that avoids the `virtual` calls prints `Department` and `R0` for every department. The output of `main` then differs from the expected output. The correct code calls `d.getName()` and `d.costToCompany()` through the reference.

2. **Testing `size` for the `(no teams)` line.** A department of size 2 with only `NULL` slots then prints nothing under the heading. The output of the edge test then differs from the expected output.
3. **Not checking for `NULL` before `*d.teams[i].`** Dereferencing a `NULL` slot is undefined behaviour. In `main`, the `Development` department has 2 slots and one team, so slot 1 is `NULL`. The first team prints, then valgrind reports `Invalid read of size 4` and the program stops with a segmentation fault (`SIGSEGV`). The edge test also has `NULL` slots, for example the gap in slot 1 of the `Marketing` department.
4. **Declaring the parameter as `const Department& d`.** The calls to `getName()` and `costToCompany()` do not compile, because these functions are not `const` member functions. The function is also no longer the friend that `Department.h` declares, so `d.size` and `d.teams` give "is private within this context" errors. A compile error gives the mark 0.

A2-04. Operator overloading: template `operator<<` for `List<T>` (100)

File: A_2.cpp

```
#ifndef A_2_CPP
#define A_2_CPP

#include "List.h"

template <class T>
ostream& operator<<(ostream& out, const List<T>& list) {
    if (list.headNode == NULL) {
        out << "Empty List";
        return out;
    }
    Node<T>* curr = list.headNode;
    // Stop at the last node, so that no "->" is printed after it.
    while (curr->next() != NULL) {
        out << curr->getData() << "->";
        curr = curr->next();
    }
    out << curr->getData();
    return out;
}

#endif
```

Expected output of the given main.cpp

```
Empty List
1->2->3
Thandi->Sipho (size 2)
```

What the function must do

The function prints a linked list of any value type `T`. The values appear in order from the head node to the last node, with `->` between two values.

An empty list prints the text `Empty List`. The function prints no `->` after the last value and no newline.

How the solution works

```
template <class T>
ostream& operator<<(ostream& out, const List<T>& list) {
    if (list.headNode == NULL) {
        out << "Empty List";
        return out;
    }
    Node<T>* curr = list.headNode;
    // Stop at the last node, so that no "->" is printed after it.
    while (curr->next() != NULL) {
        out << curr->getData() << "->";
        curr = curr->next();
    }
    out << curr->getData();
    return out;
}
```

The template line. `template <class T>` makes the function a template. The compiler writes one real function for each type `T` that the program uses. In `main.cpp`, `T` is `int` and `string`. The same text works for both, because `out << curr->getData()` uses the `operator<<` of the value type.

The `List.h` file already declares the function and contains the `friend` line with `<>`. The definition in `A_2.cpp` has no `<>` and no `friend`. The file keeps its include guard, and the code stays between `#define A_2_CPP` and `#endif`.

Parameters. `out` is a reference to the output stream. `list` is a `const` reference to the list, so the function reads the list but does not change it. The function is a friend, so it may read the private member `list.headNode`.

The empty check. `if (list.headNode == NULL)` checks whether the list has any node. A pointer holds the address of an object. `NULL` means the pointer holds no address. An empty list has `headNode` equal to `NULL`.

- If the check is true, the function prints `Empty List` and returns `out` at once.
- The check must come first. The later line `curr->next()` follows `curr`. If `curr` were `NULL`, this would follow a `NULL` pointer, which is undefined behaviour.

The walking pointer. `Node<T>* curr = list.headNode;` copies the head pointer into a new pointer `curr`. The function moves `curr` along the list. The pointer `list.headNode` stays unchanged. The parameter `list` is `const`, so the function must not change `list.headNode`.

The loop condition. `while (curr->next() != NULL)` is true while the current node has a next node. The function `next()` returns the pointer to the next node. After the last node, `next()` returns `NULL`.

- The loop stops on the last node, not after it. The last node is therefore not printed inside the loop.
- This is the reason that the function prints no `->` after the last value. Each loop pass prints a value and `->`, because another value always follows.
- If the list has one node, the condition is false at once. The loop body never runs.

Loop body. `out << curr->getData() << "->";` prints the value in the current node, then the arrow. `getData()` returns a reference to the stored value. The next line `curr = curr->next();` moves `curr` to the next node. The move happens after the print, so the function prints the current node before it leaves it.

After the loop. `out << curr->getData();` prints the last node. At this point `curr` points to the last node, because the loop stopped there. The function adds no arrow and no newline. `return out;` returns the stream for chaining.

The function allocates nothing. The `List` destructor frees the nodes, and each `Node` destructor frees its data.

Trace

The call is `cout << numbers << endl;` in `main.cpp`. The calls `prepend(3)`, `prepend(2)` and `prepend(1)` put each new node at the front. The list is then:

```
headNode -> [1] -> [2] -> [3] -> NULL
```

Step	`curr` points to	`curr->next() != NULL`	Output added	`curr` after
Start	node 1	not applicable	none (`headNode` is not `NULL`)	node 1
Loop pass 1	node 1	true (next is node 2)	`1->`	node 2
Loop pass 2	node 2	true (next is node 3)	`2->`	node 3
Loop test	node 3	false (next is `NULL`)	none	node 3
After the loop	node 3	not applicable	`3`	node 3

The function returns `out`. Then `main` prints `endl`. For the empty list `numbers` that `main` prints first, the check `list.headNode == NULL` is true and the output is `Empty List`.

The ExpectedOutput of `main.cpp` is:

```
Empty List
1->2->3
Thandi->Sipho (size 2)
```

Common mistakes

1. **Printing `->` after the last value.** A loop of the form `while (curr != NULL)` that prints a value and an arrow on each pass gives `1->2->3->`. The output of `main` then differs from the expected output.
2. **Skipping the empty check.** For an empty list, `curr` is `NULL`. The call `curr->next()` then dereferences `NULL`. This is undefined behaviour. In `main`, the first print is the empty list `numbers`. Valgrind reports `Invalid read of size 8`, and the program stops with a segmentation fault (`SIGSEGV`) before it prints anything.
3. **Writing to `cout` instead of `out`.** The edge test sends two lists to an `ostringstream`. Text sent to `cout` never reaches it. The output of the edge test then differs from the expected output.
4. **Changing the file layout.** The question states that `A_2.cpp` already has `#include "List.h"`. An added `#include "List.cpp"`, or code after `#endif`, gives a redefinition error. A compile error gives the mark 0.

A2-05. Operator overloading: `Team::operator[]` (const) returning `const Employee&` (100)

File: A_2.cpp

```
#include "Team.h"

const Employee& Team::operator[](int index) const {
    if (index < 0 || index >= size || members[index] == NULL) {
        throw Exception("Index out of bounds");
    }
    return *members[index];
}
```

Expected output of the given main.cpp

```
First: Thandi Mokoena, 29, F, ID: none
Last: Anna Botha, 41, F, ID: none
Sipho Dlamini, 35, M, ID: none
Caught: Index out of bounds
```

What the function must do

The function lets code write `team[i]` to get the employee in slot `i`, like an array index. The function returns the employee by `const` reference, so the caller can read the employee but not change it.

If the index is outside the array, or the slot is empty, the function throws an `Exception`. A reference cannot be `NULL`, so a throw is the only way to report a bad index.

How the solution works

```
const Employee& Team::operator[](int index) const {
    if (index < 0 || index >= size || members[index] == NULL) {
        throw Exception("Index out of bounds");
    }
    return *members[index];
}
```

The header line. `const Employee& Team::operator[](int index) const` has three parts that need an explanation.

- `operator[]` is an overloaded operator. It gives meaning to `team[i]` for a `Team` object.
- The return type `const Employee&` is a reference to an `Employee` that the caller must not change. The function returns the real object, not a copy.
- The second `const`, after the parameter list, makes this a `const` member function. A `const` member function promises not to change the team. A `const Team&` may call only `const` member functions. In `main.cpp`, the function `printFirstAndLast` receives a `const Team&` and uses `team[0]`, so the word `const` is required. The definition in `A_2.cpp` must match the declaration in `Team.h`, including this `const`.

The data. `members` is an array of pointers to `Employee`. `size` is the length of the array. A pointer holds the address of an object. An empty slot holds `NULL`, which means no address.

The `if` condition. The condition has three tests joined with `||` (or). The tests run from left to right. C++ stops at the first test that is true. This is called short-circuit evaluation. The order protects the array.

1. `index < 0` catches a negative index. An array has no slot -1.
2. `index >= size` catches an index past the end. The last valid index is `size - 1`. A team of size 0 has no valid index, and this test catches index 0 for that team.
3. `members[index] == NULL` catches an empty slot. A slot can be empty because the team was never full, or because `removeMember` set it to `NULL`.

The range tests come before `members[index]`. If `index` is -1 or equal to `size`, the expression `members[index]` would read memory outside the array. Because of short-circuit evaluation, the function never reads `members[index]` for those values.

The throw. `throw Exception("Index out of bounds");` creates an `Exception` object with that message and stops the function. The program jumps to the nearest matching `catch`. The function does not print anything and does not catch the exception.

The caller decides what to do. In `main.cpp`, the `catch (Exception& e)` block receives the object by reference, and `e.what()` gives the message.

If nothing catches the exception, the program ends abnormally and the mark is 0.

The return. `return *members[index];` runs only when the index is in range and the slot is not `NULL`. The `*` follows the pointer and gives the `Employee` object itself. The function returns a reference to that object. The returned reference stays valid, because the team still owns the employee.

The function allocates and frees nothing. The `Team` destructor frees the employees.

Trace

The calls come from `main.cpp`. The team has size 3 and holds Thandi (slot 0), Sipho (slot 1) and Anna (slot 2).

Call	`index < 0`	`index >= size`	`members[index] == NULL`	Result
`team[0]`	false	false (0 >= 3 is false)	false (Thandi)	returns Thandi
`team[2]`	false	false (2 >= 3 is false)	false (Anna)	returns Anna
`(*team)[1]`	false	false	false (Sipho)	returns Sipho
`(*team)[3]`	false	true (3 >= 3 is true)	not evaluated	throws `Exception("Index out of bounds")`

For the last call, the `catch (Exception& e)` block in `main` prints the message. The ExpectedOutput of `main.cpp` is:

```
First: Thandi Mokoena, 29, F, ID: none
Last: Anna Botha, 41, F, ID: none
Sipho Dlamini, 35, M, ID: none
Caught: Index out of bounds
```

Common mistakes

1. **Leaving out the `NULL` test.** The edge test removes the employee in slot 1 and then asks for `team[1]`. Without the `NULL` test, the function returns a reference made from a `NULL` pointer, which is undefined behaviour. When the edge test prints that employee, valgrind reports `Invalid read of size 8` and the program stops with a segmentation fault (`SIGSEGV`).
2. **Reading `members[index]` before the range tests.** With an index such as 3 or -1, the function reads memory outside the array. Valgrind reports `Invalid read of size 8` (once in `main`, three times in the edge test). The printed output can still look correct, because the value that is read is undefined. Only valgrind shows the mistake. The range tests must come first.
3. **Using `index > size` instead of `index >= size`.** The index equal to `size` then passes the test. For a team of size 3, index 3 is outside the array, so `members[3]` reads memory past the end. Valgrind reports `Invalid read of size 8`. The printed output can still look correct, so only valgrind shows the mistake. The correct test is `>=`.
4. **Leaving out the last `const`.** A definition `const Employee& Team::operator[](int index)` does not match the declaration in `Team.h`. The compiler reports `no declaration matches 'const Employee& Team::operator[](int)'`. A compile error gives the mark 0.

A2-06. Operator overloading: `Department::operator[]` returning `Team*&` (100)

File: A_2.cpp

```
#include "Department.h"

Team*& Department::operator[](int index) {
    if (index < 0 || index >= size) {
        throw Exception("Index out of bounds");
    }
    // Return the slot itself (not a copy), so the caller can change what it points to.
    return teams[index];
}
```

Expected output of the given main.cpp

```
Development (cost R75000)
Team:
  Thandi Mokoena, 29, F, ID: none
  Sipho Dlamini, 35, M, ID: none
Team:
  Anna Botha, 41, F, ID: none
Development (cost R75000)
Team:
  Anna Botha, 41, F, ID: none
Team:
  Thandi Mokoena, 29, F, ID: none
  Sipho Dlamini, 35, M, ID: none
Caught: Index out of bounds
```

What the function must do

The function lets code write `(*dev)[i]` to reach slot `i` of the `teams` array in a `Department`. The function returns the slot itself, not a copy. The caller can then read the pointer in the slot, call a function through it, or store a new pointer in the slot.

If the index is outside the array, the function throws an `Exception`. An empty slot is not an error. The function returns the `NULL` pointer in that slot.

How the solution works

```
Team*& Department::operator[](int index) {
    if (index < 0 || index >= size) {
        throw Exception("Index out of bounds");
    }
    // Return the slot itself (not a copy), so the caller can change what it points to.
    return teams[index];
}
```

The return type. `Team*&` reads from right to left as "a reference to a pointer to a `Team`".

- A pointer holds the address of an object. `Team*` is a pointer to a `Team`.
- A reference is another name for an existing variable. A reference to a pointer is another name for the pointer variable itself.
- `teams` is an array of `Team*` values. The element `teams[index]` is one pointer variable. The function returns a reference to that variable.

Because the caller receives the variable itself, an assignment such as `(*dev)[0] = new Team(2);` changes the real slot in the array. If the function returned a copy of the pointer, the assignment would change only the copy. The array would stay unchanged.

The operator. `operator[]` is an overloaded operator. It gives meaning to `dev[i]` for a `Department`. In `main.cpp`, `dev` is a pointer, so the code writes `(*dev)[0]`. The `*` turns the pointer into the object, and then `[0]` calls the operator.

The `if` condition. `if (index < 0 || index >= size)` checks that the index is a valid slot number.

- `index < 0` catches a negative index. An array has no slot -1.
- `index >= size` catches an index past the end. The last valid index is `size - 1`. A department of size 0 has no valid index, and this test catches index 0 for that department.
- The check comes before `teams[index]`. The expression `teams[index]` with a bad index would refer to memory outside the array.

The condition has no test for `NULL`. An empty slot is a valid slot. The caller must be able to read it, test it or fill it. The code in `main.cpp` first fills each empty slot through this operator. A `NULL` test would throw before the slot ever gets a team.

The throw. `throw Exception("Index out of bounds");` creates an `Exception` object with that message. The function stops at this point. The program jumps to the nearest matching `catch`. The function does not print and does not catch. In `main.cpp`, the block `catch (Exception& e)` receives the object by reference and prints `e.what()`. If no `catch` handles the throw, the program ends abnormally and the mark is 0.

The return. `return teams[index];` returns the array element itself. The function must not copy the pointer into a local variable first. A local variable ends when the function ends, so a reference to it would not be valid. The compiler reports this under `-Wall -Werror`.

Ownership. The department owns its teams and deletes every slot in its destructor. If the caller stores a new pointer into a slot that already holds a team, the old team is lost unless the caller deletes it first. The operator does not delete anything. In `main.cpp`, the swap moves pointers between slots and creates no new team, so each team is still deleted once.

Trace

The department `dev` has size 3. After the first lines of `main`, slot 0 holds the team of Thandi and Sipho (call it team A). Slot 1 is `NULL`. Slot 2 holds the team of Anna (call it team B).

The swap in `main.cpp` has three statements.

Statement	Index checks	What the operator returns	Slots after the statement (0, 1, 2)
`Team* temp = (*dev)[0];`	0 is in range	the slot 0 variable, read as a value	A, `NULL`, B (`temp` holds A)
`(*dev)[0] = (*dev)[2];`	0 and 2 are in range	the slot 2 variable is read (B), the slot 0 variable is assigned	B, `NULL`, B
`(*dev)[2] = temp;`	2 is in range	the slot 2 variable is assigned	B, `NULL`, A

Then `(*dev)[3] = NULL;` runs inside a `try` block. The test `index >= size` is `3 >= 3`, which is true. The function throws before any assignment. The `catch` block prints the message.

The second `cout << *dev;` prints team B first and team A second. The ExpectedOutput of `main.cpp` ends with:

```
Development (cost R75000)
Team:
  Anna Botha, 41, F, ID: none
Team:
  Thandi Mokoena, 29, F, ID: none
  Sipho Dlamini, 35, M, ID: none
Caught: Index out of bounds
```

Common mistakes

1. **Allowing `index == size`.** Code with `index > size` accepts index 3 for a department of size 3. The assignment `(*dev)[3] = NULL;` in `main` then writes outside the array. Valgrind reports `Invalid write of size 8`. No exception is thrown, so the line `Caught: Index out of bounds` is missing from the output.
2. **Throwing for an empty slot.** The first statement `(*dev)[0] = new Team(2);` in `main.cpp` uses a `NULL` slot. If the function throws for `NULL`, this statement throws outside any `try` block. The exception is not caught. The program prints `terminate called after throwing an instance of 'Exception'` and stops with `SIGABRT` before it prints anything.
3. **Returning a copy or a local variable.** A version such as `Team* t = teams[index]; return t;` returns a reference to a local variable. The compiler gives `reference to local variable 't' returned`, and `-Werror` turns this warning into an error. A return type of `Team*` also does not match the declaration in `Department.h`. The compiler reports `no declaration matches 'Team* Department::operator[](int)'`.
4. **Reading `teams[index]` before the range check.** A bad index then reads memory outside the array. Valgrind reports `Invalid read of size 8` for each bad index. The printed output can still look correct, so only valgrind shows the mistake. The range check must come first.

A2-07. Operator overloading: `Employee::operator==` and `operator!=` (100)

File: A_2.cpp

```
#include "Employee.h"

bool Employee::operator==(const Employee& other) const {
    // Only the ID numbers are compared. Two IDs that are not set ("") are equal too.
    return idNum == other.idNum;
}

bool Employee::operator!=(const Employee& other) const {
    return !(*this == other);
}
```

Expected output of the given main.cpp

```
a, b (no IDs): == gives true, != gives false
a, b (same ID): == gives true, != gives false
a, c (different IDs): == gives false, != gives true
```

Hints

A2-07 hints

Try the question first. Read one hint at a time. The hints go from gentle to specific. They do not give the full answer.

Hint 1

`a != b` must always be the opposite of `a == b`. You do not need to compare the ID numbers a second time.

Hint 2

Inside a member function, `*this` is the current object. So `*this == other` calls your own `operator==`.

Hint 3

`operator!=` can return the opposite (`!`) of that call. Then the two operators can never disagree.

What the function must do

The two operators compare two `Employee` objects. The operator `==` returns `true` when the two ID number strings are equal. It ignores names, age and gender.

An employee with no ID number has the empty string `""` as the ID. Two such employees are therefore equal. The operator `!=` must always return the opposite of `==`.

How the solution works

```
bool Employee::operator==(const Employee& other) const {
    // Only the ID numbers are compared. Two IDs that are not set ("") are equal too.
    return idNum == other.idNum;
}

bool Employee::operator!=(const Employee& other) const {
    return !(*this == other);
}
```

Overloaded operators as members. `operator==` and `operator!=` are member functions. They give meaning to `x == y` and `x != y` for two `Employee` objects. In the call `x == y`, the object on the left is the current object, and `y` is the parameter `other`.

The parameter. `const Employee& other` is a reference to the other employee. A reference is another name for an existing object, so the function makes no copy. The word `const` in front means the function must not change `other`.

The `const` after the parameter list. The second `const` makes the function a `const` member function. It promises not to change the current object. Both operators need it for two reasons.

- The header `Employee.h` declares them with `const`. The definition in `A_2.cpp` must match the declaration.
- In `main.cpp`, the function `compare` receives `const Employee&` parameters. A `const` object can call only `const` member functions.

The `==` operator. The statement `return idNum == other.idNum;` compares two `string` values. The `string` class defines `==` as "same characters in the same order". The expression gives a `bool`, and the function returns it.

- The function reads `other.idNum` directly. This is allowed because `other` has the same class as the current object. Private members are open to code in the same class.
- The function has no `if` and no special case for the empty ID. The constructor sets `idNum` to `""`. Two employees with no ID both hold `""`, and `"" == ""` is `true`. This gives the required result for row 4 of the question, where both IDs are not set.
- The function does not compare the names, age or gender. The question says only the ID matters.
- When an employee compares with itself, the two strings are the same, so the result is `true`.

The `!=` operator. The statement `return !(*this == other);` calls the operator `==` and reverses the answer.

- Inside a member function, `this` is a pointer to the current object. `*this` follows the pointer and gives the object itself. So `*this == other` calls the `operator==` above.
- The `!` operator turns `true` into `false` and `false` into `true`.
- The function reuses `==` and does not compare the ID numbers a second time. If the rule for `==` changes, the function `!=` follows it. The two operators cannot disagree.
- The function `!=` calls `==`, and `==` does not call `!=`. The calls do not repeat forever.

The functions allocate and free nothing, so valgrind reports no memory problem from them.

Trace

The calls come from `compare` in `main.cpp`. The line `bool same = (x == y);` calls `==`. The line `bool different = (x != y);` calls `!=`.

Call 1: `a` and `b`, no IDs set. Both `idNum` values are `""`.

Step	Expression	Value
1	`idNum == other.idNum`	`"" == ""`, which is `true`
2	`*this == other` inside `!=`	`true`
3	`!(*this == other)`	`false`

Call 3: `a` and `c`, different IDs. The ID of `a` is `9702140123081`. The ID of `c` is `9702140123082`. The names also differ, but the function ignores them.

Step	Expression	Value
1	`idNum == other.idNum`	the strings differ in the last character, so `false`
2	`*this == other` inside `!=`	`false`
3	`!(*this == other)`	`true`

For call 2, `a` and `b` both hold `9702140123081`, so `==` gives `true` and `!=` gives `false`. The ExpectedOutput of `main.cpp` is:

```
a, b (no IDs): == gives true, != gives false
a, b (same ID): == gives true, != gives false
a, c (different IDs): == gives false, != gives true
```

Common mistakes

1. **Treating an ID that is not set as never equal.** A version such as `idNum != "" && idNum == other.idNum` returns `false` for two employees without an ID. The question requires `true`. The output of `main` then differs from the expected output.
2. **Comparing the names as well.** The edge test compares two employees with the same ID and different names. The expected result is `true`. The output of the edge test then differs from the expected output.
3. **Making `!=` give the same result as `==`.** If the body of `!=` is `return idNum == other.idNum;`, the two operators always agree. The output of `main` then differs from the expected output.
4. **Calling `*this != other` inside `operator!=`.** The function then calls itself, and the calls never end. The compiler (g++ 13 with `-Wall`) detects this and gives `infinite recursion detected`. Under `-Werror` this warning is an error, so the program

does not compile. A compile error gives the mark 0.

A2-08. Operator overloading: `Department operator< and operator> by cost (100)`

File: A_2.cpp

```
#include "Department.h"

// costToCompany() is virtual, so each department uses its own cost rule.
bool Department::operator<(Department& other) {
    return costToCompany() < other.costToCompany();
}

bool Department::operator>(Department& other) {
    return costToCompany() > other.costToCompany();
}
```

Expected output of the given main.cpp

```
Sales costs R30000
Development costs R25000
HR costs R0
Sales < Development: false
Sales > Development: true
Development < Sales: true
Development > Sales: false
HR < Development: true
HR > Development: false
```

What the function must do

The two operators compare two `Department` objects by their monthly cost to the company. The operator `<` returns `true` when the left department costs less than the right department. The operator `>` returns `true` when the left department costs more.

If the two costs are equal, both operators return `false`. The functions compare costs, not the number of employees, because each department type has its own cost per employee.

How the solution works

```
// costToCompany() is virtual, so each department uses its own cost rule.
bool Department::operator<(Department& other) {
    return costToCompany() < other.costToCompany();
}

bool Department::operator>(Department& other) {
    return costToCompany() > other.costToCompany();
}
```

Overloaded operators as members. `operator<` and `operator>` are member functions of `Department`. They give meaning to `x < y` and `x > y`. In the expression `x < y`, the object `x` on the left is the current object. The object `y` on the right is the parameter `other`.

The parameter. `Department& other` is a reference. A reference is another name for an existing object, so the function makes no copy. The reference is not `const`, because `costToCompany()` is not a `const` member function. A `const` object cannot call it. The function declares `Department&` to match `Department.h`.

The virtual call. `Sales`, `Marketing`, `HR` and `Development` are subclasses of `Department`. The function `costToCompany()` is `virtual`.

- A `virtual` function runs the version that belongs to the real object, even when the code reaches the object through a `Department&`.
- In `main.cpp`, `x` and `y` have the type `Department&`, but they refer to a `Sales` object and a `Development` object. The call runs `Sales::costToCompany` and `Development::costToCompany`.
- The subclass versions multiply the number of employees by a rate. The rates are R15000 for Sales, R18000 for Marketing, R12000 for HR and R25000 for Development. The base `Department` returns 0.

The two cost calls. In `costToCompany() < other.costToCompany()`, the call without an object name runs on the current object, as if written `this->costToCompany()`. The pointer `this` points to the current object. The call `other.costToCompany()` runs on the right-hand object. Both calls use the virtual rule of each real object.

The comparison. The built-in `<` on two `float` values gives `true` or `false`. The built-in `>` works the same way. The function returns this `bool` at once. The function has no `if`.

- For equal costs, `<` and `>` are both `false`. The built-in operators use strict comparison, so equal values give `false` for both.
- The function `>` does its own comparison. It does not use `!(*this < other)`. That expression would give `true` for equal costs.
- A department compared with itself gives `false` for both operators, because the two costs are equal.

The functions allocate and free nothing, so valgrind reports no memory problem from them.

Trace

The calls come from `compare` in `main.cpp`. The department `sales` has one team of 2 employees. The department `dev` has one team of 1 employee.

`compare(*sales, *dev)`

Step	Expression	Value
1	`costToCompany()` on `sales` (current object)	2 employees times 15000, which is `30000`
2	`other.costToCompany()` on `dev`	1 employee times 25000, which is `25000`
3	`30000 < 25000`	`false`
4	`30000 > 25000`	`true`

`compare(*hr, *dev)`. The department `hr` has one team with 0 employees.

Step	Expression	Value
1	`costToCompany()` on `hr`	0 employees times 12000, which is `0`
2	`other.costToCompany()` on `dev`	`25000`
3	`0 < 25000`	`true`
4	`0 > 25000`	`false`

The ExpectedOutput of `main.cpp` is:

```
Sales costs R30000
Development costs R25000
HR costs R0
Sales < Development: false
Sales > Development: true
Development < Sales: true
Development > Sales: false
HR < Development: true
HR > Development: false
```

Common mistakes

1. **Using `<=` and `>=`.** Two departments with equal costs then return `true`. The edge test compares Marketing (5 employees, R90000) with Sales (6 employees, R90000), and both operators must return `false`. The output of the edge test then differs from the expected output.
2. **Comparing the number of employees.** A version that compares `countEmployees()` ignores the rate. In the edge test, Marketing has 5 employees and Sales has 6, but their costs are equal. The output of the edge test then differs from the expected output.
3. **Calling the base version with a scope name.** A call such as `Department::costToCompany()` turns off the virtual rule. The base version returns 0 for every department, so every comparison gives `false`.
4. **Writing `>` as `!(*this < other)`.** For equal costs, `<` gives `false`, and the `!` makes `>` give `true`. The question requires `false`.

A2-09. Operator overloading: `Team::operator+=` grows the array by one (100)

File: A_2.cpp

```
#include "Team.h"

Team& Team::operator+=(Employee* employee) {
    if (employee == NULL) {
        return *this;
    }
    Employee** temp = new Employee*[size + 1];
    for (int i = 0; i < size; i++) {
        temp[i] = members[i];  // move the pointers; the employees stay where they are
    }
    temp[size] = new Employee(*employee);  // the team keeps its own copy
    delete[] members;                      // only the old array, not the employees
    members = temp;
    size++;
    return *this;
}
```

Expected output of the given main.cpp

```
Team (2/2):
  Thandi Mokoena, 29, F, ID: none
  Sipho Dlamini, 35, M, ID: none
Main's employee: Sipho Dlamini, 35, M, ID: 9105225800087
Team (4/4):
  Thandi Mokoena, 29, F, ID: none
  Sipho Dlamini, 35, M, ID: none
  Anna Botha, 41, F, ID: none
  Lebo Nkosi, 25, F, ID: none
```

What the function must do

The operator `team += employee;` adds a copy of an employee to the end of a `Team`. The array `members` grows by one slot each time, so the team never becomes full.

The team stores its own copy. The caller still owns the employee it passed in. If the pointer is `NULL`, the function changes nothing. In all cases, the function returns the team itself, so calls can be chained.

How the solution works

```
Team& Team::operator+=(Employee* employee) {
    if (employee == NULL) {
        return *this;
    }
    Employee** temp = new Employee*[size + 1];
    for (int i = 0; i < size; i++) {
        temp[i] = members[i];  // move the pointers; the employees stay where they are
    }
    temp[size] = new Employee(*employee);  // the team keeps its own copy
    delete[] members;                      // only the old array, not the employees
    members = temp;
    size++;
    return *this;
}
```

The signature. `Team& Team::operator+=(Employee* employee)` is an overloaded operator. It gives meaning to `team += pointer`. The return type `Team&` is a reference to a `Team`. The parameter `employee` is a pointer. A pointer holds the address of an object, and `NULL` means it holds no address.

Data. `members` is an array of pointers to `Employee`. `size` is the length of that array. An array made with `new` has a fixed length. The array cannot grow in place. The function therefore builds a larger array and replaces the old one.

The `NULL` check. `if (employee == NULL)` checks that the caller passed an employee.

- If the check is true, the function returns `*this` at once. The team stays unchanged.

- The check protects the later statement `new Employee(*employee)`. The `*` follows the pointer. Following a `NULL` pointer is undefined behaviour.
- The check comes first, before any allocation. The function therefore allocates nothing for a `NULL` input.
- `*this` is the current object. Inside a member function, `this` is a pointer to the current object, and `*this` is the object itself. Returning it by reference allows a chain such as `(team += a) += b;`.

The new array. `Employee** temp = new Employee*[size + 1];` allocates an array with one more slot than the old array. `Employee**` is a pointer to a pointer to an `Employee`, which is the type of an array of `Employee*` values. The name `temp` holds the new array until the function is ready to swap.

The copy loop. `for (int i = 0; i < size; i++)` visits every index of the old array. The statement `temp[i] = members[i];` copies the pointer into the same index of the new array.

- The loop copies pointers, not employees. The employees stay where they are in memory. The new array points to the same employees as the old array.
- A `NULL` slot is copied as `NULL`, so empty slots keep their places.
- The loop uses the old `size`. The function must not change `size` before the loop finishes. The old array has `size` slots only.

The new employee. `temp[size] = new Employee(*employee);` puts a copy in the last slot, index `size`. The index `size` is valid in the new array, because the new array has `size + 1` slots. The expression `new Employee(*employee)` calls the copy constructor `Employee(const Employee&)`. The copy is a separate object. A later change to the caller's employee does not change the team's copy. The team's destructor deletes this copy, and the caller deletes the original.

Free the old array. `delete[] members;` frees the old array only. Memory made with `new[]` must be freed with `delete[]`. The function must not delete the employees. The new array still points to them, and the team's destructor will delete them later.

- This statement comes before `members = temp;`. After the reassignment, the address of the old array would be lost, and the memory could never be freed. Valgrind would report a leak.

Update the members. `members = temp;` makes the team use the new array. Then `size++;` adds 1 to the length. The new `size` matches the new array. The statement comes after all uses of the old `size`.

The return. `return *this;` returns the team by reference.

Trace

The first call in `main.cpp` is `*team += sipho;`. The team starts with `size` 1 and holds Thandi in slot 0. The employee `sipho` has no ID number yet.

Step	Statement	State
1	`employee == NULL`	false, so the function continues
2	`temp = new Employee*[size + 1]`	`temp` has 2 slots (uninitialised)
3	loop `i = 0`: `temp[0] = members[0]`	`temp[0]` points to Thandi
4	loop test `i = 1`, `1 < 1`	false, the loop ends
5	`temp[1] = new Employee(*employee)`	`temp[1]` points to a new copy of Sipho
6	`delete[] members`	the old 1-slot array is freed. Thandi is not freed
7	`members = temp`	the team uses the 2-slot array
8	`size++`	`size` is 2
9	`return *this`	the team is returned

After this call, `main` sets the ID number on its own `sipho`. The team holds a copy made earlier, so the team's Sipho still shows `ID: none`. The line `(*team += anna) += lebo;` runs the same steps twice. The first call grows `size` from 2 to 3 and returns the team. The second call grows `size` from 3 to 4.

The ExpectedOutput of `main.cpp` is:

```
Team (2/2):
  Thandi Mokoena, 29, F, ID: none
  Sipho Dlamini, 35, M, ID: none
Main's employee: Sipho Dlamini, 35, M, ID: 9105225800087
Team (4/4):
```

```
Thandi Mokoena, 29, F, ID: none
Sipho Dlamini, 35, M, ID: none
Anna Botha, 41, F, ID: none
Lebo Nkosi, 25, F, ID: none
```

Common mistakes

1. **Storing the caller's pointer.** The statement `temp[size] = employee;` stores the caller's object in the team. The team's output then shows the ID number that `main` set later (`ID: 9105225800087` and not `ID: none`), so the output differs. `main` deletes `sipho`, and then the `Team` destructor deletes the same object again. Valgrind reports `Invalid read of size 8` and `Invalid free() / delete / delete[] / realloc()`, which is a double delete.
2. **Not deleting the old array.** The output is still correct, but the old array is never freed. Valgrind reports a memory leak: in `main`, `definitely lost: 48 bytes in 3 blocks`, one block for each call.
3. **Deleting the old employees.** A loop that deletes `members[i]` before the swap frees employees that the new array still points to. The team then has dangling pointers. The printed output can still look correct, because reading freed memory is undefined behaviour. Valgrind reports many `Invalid read` errors and `Invalid free() / delete / delete[] / realloc()` when the destructor deletes the same employees again.
4. **Missing the `NULL` check.** The edge test runs `*team += NULL;`. Without the check, the function evaluates `*employee` on a `NULL` pointer, which is undefined behaviour. Valgrind reports `Invalid read of size 4`, and the program stops with a segmentation fault (`SIGSEGV`).

A2-10. Operator overloading: `Team::operator+` returns a merged team (100)

File: A_2.cpp

```
#include "Team.h"

Team Team::operator+(const Team& other) const {
    int count = 0;
    for (int i = 0; i < size; i++) {
        if (members[i] != NULL) {
            count++;
        }
    }
    for (int i = 0; i < other.size; i++) {
        if (other.members[i] != NULL) {
            count++;
        }
    }

    Team result(count);
    int next = 0;
    for (int i = 0; i < size; i++) {
        if (members[i] != NULL) {
            result.members[next] = new Employee(*members[i]);
            next++;
        }
    }
    for (int i = 0; i < other.size; i++) {
        if (other.members[i] != NULL) {
            result.members[next] = new Employee(*other.members[i]);
            next++;
        }
    }
    return result;
}
```

Expected output of the given main.cpp

```
Team (3/3):
  Thandi Mokoena, 29, F, ID: none
  Sipho Dlamini, 35, M, ID: none
  Anna Botha, 41, F, ID: none
Team (2/3):
  Thandi Mokoena, 29, F, ID: none
  Sipho Dlamini, 35, M, ID: none
Team (1/2):
  Anna Botha, 41, F, ID: none
```

What the function must do

The operator `a + b` makes a new `Team` that holds copies of the members of `a`, then copies of the members of `b`. Neither `a` nor `b` changes.

The new team has exactly as many slots as there are members in both teams. Empty slots in `a` and `b` are left out, so the new team has no gaps. The function returns the new team by value.

How the solution works

```
Team Team::operator+(const Team& other) const {
    int count = 0;
    for (int i = 0; i < size; i++) {
        if (members[i] != NULL) {
            count++;
        }
    }
    for (int i = 0; i < other.size; i++) {
        if (other.members[i] != NULL) {
            count++;
        }
    }
```

```
    Team result(count);
    int next = 0;
    for (int i = 0; i < size; i++) {
        if (members[i] != NULL) {
            result.members[next] = new Employee(*members[i]);
            next++;
        }
    }
    for (int i = 0; i < other.size; i++) {
        if (other.members[i] != NULL) {
            result.members[next] = new Employee(*other.members[i]);
            next++;
        }
    }
    return result;
}
```

The signature. `Team Team::operator+(const Team& other) const` is an overloaded operator for `a + b`.

- The return type is `Team`, not `Team&`. The function returns a new object by value.
- `const Team& other` is a reference to the right-hand team. The function reads it and does not change it. A reference is another name for an existing object, so no copy is made.
- The `const` after the parameter list means the function does not change the left-hand team.
- The functions `getMemberCount()` and `operator[]` are not `const` member functions. A `const` function cannot call them. The function therefore counts and reads `members` and `size` directly.

Data. `members` is an array of pointers to `Employee`. A pointer holds the address of an object. An empty slot holds `NULL`. Reading `other.members` is allowed because `other` has the same class as the current object.

Step 1: count the members. Two loops count the slots that are not `NULL`.

- `for (int i = 0; i < size; i++)` visits each slot of the current team. `for (int i = 0; i < other.size; i++)` does the same for `other`. Each loop stops before its own array length.
- `if (members[i] != NULL)` and `if (other.members[i] != NULL)` count only slots that hold an employee. The test also protects the later dereference. An empty slot is not counted and not copied.
- The count is needed first because a `Team` array has a fixed length. The function must know the final length before it creates the new team. Adding `size + other.size` would keep the empty slots and leave gaps.
- If both teams have no members, the loops leave `count` at 0, and the result has size 0.

Step 2: create the result. `Team result(count);` makes a local `Team` with `count` slots, all `NULL`. The variable `next` holds the index of the next free slot in `result`. It starts at 0.

Step 3: copy the current team. The loop visits the slots in order from index 0.

- `if (members[i] != NULL)` skips empty slots again. Following a `NULL` pointer with `*members[i]` is undefined behaviour.
- `result.members[next] = new Employee(*members[i]);` creates a copy of the employee with the copy constructor of `Employee`. The expression `*members[i]` follows the pointer to the employee. The result team now owns a separate object.
- `next++;` moves to the next free slot. The increment is inside the `if`, so empty slots do not use up a position in `result`.
- If the function stored the pointer `members[i]` instead, two teams would hold the same employee. Both destructors would delete it, which is a double delete.

Step 4: copy `other`. The last loop does the same for `other`. It starts with `next` at the position where the first loop ended, so the members of `other` come after the members of the current team. The order is current team first, then `other`. For `a + a`, the second loop makes a second set of copies from the same employees.

The return. `return result;` returns the local team by value. The `Team` class has a deep copy constructor, so the returned copy gets its own employees. The compiler may also skip the copy and build the object directly in the caller's variable. The result is correct in both cases. When `result` goes out of scope, its destructor deletes its employees, and the caller's team keeps its own.

Trace

The call is `Team merged = *north + *south;` in `main.cpp`. The team `north` has size 3 and holds `[Thandi, Sipho, NULL]`. The team `south` has size 2 and holds `[Anna, NULL]`.

Counting:

Loop	i	Slot	Not NULL	count after
north	0	Thandi	true	1
north	1	Sipho	true	2
north	2	NULL	false	2
south	0	Anna	true	3
south	1	NULL	false	3

The function creates `result(3)`. Copying:

Loop	i	Slot	Action	next after
north	0	Thandi	`result.members[0]` is a copy of Thandi	1
north	1	Sipho	`result.members[1]` is a copy of Sipho	2
north	2	NULL	skipped	2
south	0	Anna	`result.members[2]` is a copy of Anna	3
south	1	NULL	skipped	3

The three `cout` lines in `main` print `merged`, then `north`, then `south`. The teams `north` and `south` are unchanged. The ExpectedOutput of `main.cpp` is:

```
Team (3/3):
  Thandi Mokoena, 29, F, ID: none
  Sipho Dlamini, 35, M, ID: none
  Anna Botha, 41, F, ID: none
Team (2/3):
  Thandi Mokoena, 29, F, ID: none
  Sipho Dlamini, 35, M, ID: none
Team (1/2):
  Anna Botha, 41, F, ID: none
```

Common mistakes

1. **Copying the pointers.** The statement `result.members[next] = members[i];` shares the employees between teams. Each destructor deletes them, which gives a double delete. In `main`, the printed output still looks correct. Valgrind reports `Invalid read of size 8` and `Invalid free() / delete / delete[] / realloc()` when `merged` is destroyed after `north` and `south` were deleted.
2. **Sizing the result with `size + other.size`.** The result then has empty slots. In `main.cpp` it prints `Team (3/5):` where `Team (3/3):` is expected. The output of `main` then differs from the expected output.
3. **Building the result on the heap.** Code such as `Team* result = new Team(count); return *result;` returns a copy and never deletes the original. The output is still correct, but valgrind reports a leak. In `main` it shows `definitely lost: 16 bytes in 1 blocks` (the team), with its array and employees lost through it.
4. **Calling `getMemberCount()`.** The current team and `other` are both `const` inside the function. The function `getMemberCount()` is not a `const` member function, so the code does not compile. A compile error gives the mark 0.

A2-11. Operator overloading: `Employee` prefix and postfix `operator++` (a birthday) (100)

File: A_2.cpp

```
#include "Employee.h"

// Prefix: ++e
Employee& Employee::operator++() {
    age++;
    return *this;
}

// Postfix: e++ (the int parameter only tells the two versions apart)
Employee Employee::operator++(int) {
    Employee old(*this);
    age++;
    return old;
}
```

Expected output of the given main.cpp

```
Thandi Mokoena, 30, F, ID: none
Before: Thandi Mokoena, 30, F, ID: none
After: Thandi Mokoena, 31, F, ID: none
```

What the function must do

Both increment operators give an `Employee` a birthday. Each one adds 1 to `age` and changes nothing else.

The two versions return different things. The prefix version `++e` returns the employee itself. The postfix version `e++` returns a copy of the employee as it was before the birthday.

How the solution works

```
// Prefix: ++e
Employee& Employee::operator++() {
    age++;
    return *this;
}

// Postfix: e++ (the int parameter only tells the two versions apart)
Employee Employee::operator++(int) {
    Employee old(*this);
    age++;
    return old;
}
```

Two functions with one name. C++ has two forms of `++`. Both are overloaded operators of `Employee`. The compiler tells them apart by the parameter list.

- `operator++()` has no parameter. The compiler uses it for `++e`.
- `operator++(int)` has one `int` parameter. The compiler uses it for `e++`. The parameter has no name, because the function never uses its value. It exists only to mark the postfix version.

Prefix: `Employee& Employee::operator++()`.

- The return type `Employee&` is a reference. A reference is another name for an existing object. The function returns the real employee, not a copy.
- `age++;` adds 1 to the member `age` of the current object. Nothing else changes: the names, gender and ID number stay as they are.
- `return *this;` returns the current object. Inside a member function, `this` is a pointer to the current object. A pointer holds the address of an object. The `*` follows the pointer and gives the object itself.
- The reference points to the same object that the caller holds. Because of this, `++(++e)` adds 2 to `e`. The inner call returns `e`, and the outer call increments `e` again. The edge test also checks this: it stores the result of `++lebo` in `Employee& ref`, and a change through `ref` changes `lebo`.
- The function adds 1 first and returns afterwards. The prefix operator returns the new value, so the add must come before the return.

Postfix: `Employee Employee::operator++(int)`.

- The return type `Employee` has no `&`. The function returns a new object by value.
- `Employee old(*this);` creates a local object `old` with the copy constructor `Employee(const Employee&)`. The expression `*this` is the current object. The copy holds the age before the birthday.
- `age++;` adds 1 to the age of the current object. The copy `old` is a separate object and keeps the old age.
- `return old;` returns the copy.
- The copy must come before `age++`. If `age++` came first, the copy would hold the new age, and the postfix version would behave like the prefix version.
- The function cannot return a reference. The local variable `old` ends when the function ends. A reference to it would not be valid. With `-Wall -Werror`, the compiler rejects code that returns a reference to a local variable.

Why the postfix version returns by value. The caller of `e++` receives the old state of the employee. The employee itself already holds the new state. The only way to give back the old state is a separate object.

The functions allocate nothing with `new`, so valgrind reports no memory problem from them.

Trace

The calls come from `main.cpp`. The employee `thandi` starts with age 29.

Step	Statement	Operator called	age of `thandi` after	Value returned
1	`++(*thandi);`	prefix	30	reference to `thandi` (unused)
2	`cout << *thandi << endl;`	none	30	not applicable
3	`(*thandi)++` inside `Employee before = ...`	postfix	31	copy `old` with age 30
4	`before` is built from the returned copy	none	31	`before` has age 30

Inside the postfix call in step 3, the order is:

Order	Statement	`old.age`	age of `thandi`
1	`Employee old(*this);`	30	30
2	`age++;`	30	31
3	`return old;`	30	31

The output is:

```
Thandi Mokoena, 30, F, ID: none
Before: Thandi Mokoena, 30, F, ID: none
After: Thandi Mokoena, 31, F, ID: none
```

Common mistakes

1. **Returning the new value from the postfix version.** A postfix version that increments first and then copies returns the new age. The line `Before:` would show 31 where 30 is expected. The output of `main` then differs from the expected output.
2. **Returning a copy from the prefix version.** A prefix version such as `Employee* copy = new Employee(*this); age++; copy->age++; return *copy;` returns a reference to a separate object, not to the employee itself. In `main` the output still looks correct, but the copy is never deleted. Valgrind reports `definitely lost: 112 bytes in 1 blocks`. In the edge test, `++(++lebo)` then adds only 1 to `lebo`, and `ref.setIDNum(...)` changes the copy and not `lebo`, so the edge output differs.
3. **Returning a reference to the local variable `old`.** A return type of `Employee&` in the postfix version, with `return old;`, returns a reference to a variable that ends with the function. This definition does not match `Employee operator++(int);` in `Employee.h`, so the compiler reports `no declaration matches 'Employee& Employee::operator++(int)'`. A compile error gives the mark 0.
4. **Using the same signature for both versions.** Two functions named `operator++()` with no parameter cannot exist together. The postfix version needs the extra `int` parameter, as in `Employee.h`. A postfix definition written as `Employee Employee::operator++()` gives `no declaration matches 'Employee Employee::operator++()'`.

A2-12. Operator overloading: `Team::operator()` finds a surname (100)

File: A_2.cpp

```
#include "Team.h"

int Team::operator()(string surName) const {
    for (int i = 0; i < size; i++) {
        if (members[i] != NULL && members[i]->getSurName() == surName) {
            return i;
        }
    }
    return -1;
}
```

Expected output of the given main.cpp

```
Dlamini: 1
Botha: 2
Nkosi: -1
```

What the function must do

The function call operator lets code write `team("Dlamini")` as if the team were a function. The function searches the team for a member with the given surname. It returns the index of the first member that matches.

If no member has that surname, the function returns `-1`. The match is exact. Capital letters and spaces matter. Empty slots are skipped.

How the solution works

```
int Team::operator()(string surName) const {
    for (int i = 0; i < size; i++) {
        if (members[i] != NULL && members[i]->getSurName() == surName) {
            return i;
        }
    }
    return -1;
}
```

The signature. `int Team::operator()(string surName) const` has these parts.

- `operator()` is an overloaded operator. It gives meaning to a call such as `(*team)("Dlamini")`. In `main.cpp`, `team` is a pointer, so the code writes `(*team)`. The `*` turns the pointer into the object, and then `("Dlamini")` calls the operator.
- The parameter `string surName` is a copy of the surname to find.
- The return type `int` is the index of the match, or `-1`. The value `-1` is never a valid index, so it signals "not found".
- The `const` after the parameter list makes this a `const` member function. It promises not to change the team. The edge test calls the operator on a `const Team&`, which is possible only for a `const` member function. The definition must match the declaration in `Team.h`.

Data. `members` is an array of pointers to `Employee`. `size` is the length of the array. A pointer holds the address of an object. An empty slot holds `NULL`, which means no address.

The loop. `for (int i = 0; i < size; i++)` visits the slots in order, from index 0 to `size - 1`. The loop stops before `size`, because index `size` is outside the array.

- The order of the loop gives the "first match" rule. The function returns at the first match, so a later match with the same surname is never reached. In the edge test, the surname `Mokoena` appears in slot 0 and slot 1, and the function returns 0.
- If `size` is 0, the loop body never runs, and the function returns `-1`.

The `if` condition. The condition has two tests joined with `&&` (and). C++ checks the left test first. If the left test is false, C++ does not check the right test. This is called short-circuit evaluation. The order of the two tests matters.

1. `members[i] != NULL` checks that the slot holds an employee. An employee may have been removed with `removeMember`, and a team may not be full.

2. `members[i]->getSurName() == surName` compares the surname of that employee with the parameter. The arrow `->` follows the pointer and calls the function on the employee. The `string` class defines `==` as an exact comparison, character by character. The text `nkosi` does not equal `Nkosi`, and `"Nkosi "` with a space does not equal `Nkosi`.

The `NULL` test comes first. If the slot is `NULL`, the arrow `->` would follow a pointer that points to nothing. This is undefined behaviour. Because of short-circuit evaluation, the right side never runs for an empty slot.

The `const` and `getSurName()`. The function `getSurName()` is not a `const` member function. A `const` function usually cannot call non-`const` functions on its own object. Here the call is on the employee that `members[i]` points to, not on the team. In a `const` member function, the pointer `members` cannot change, but the `Employee` objects it points to can be used normally. The call compiles.

The return. `return i;` leaves the function at once with the index of the first match. `return -1;` comes after the loop. It runs only when the loop finishes without a match.

The function allocates and frees nothing.

Trace

The team in `main.cpp` has size 4. The array holds `[Mokoena, Dlamini, Botha, NULL]`, where each name is the surname of the employee in that slot.

Call `(*team)("Nkosi")`: no member has this surname, so the loop runs to the end.

`i`	`members[i] != NULL`	`getSurName()`	equals `"Nkosi"`	Action
0	true	`Mokoena`	false	next `i`
1	true	`Dlamini`	false	next `i`
2	true	`Botha`	false	next `i`
3	false	not called	not checked	next `i`

After `i` is 3, the test `4 < 4` is false and the loop ends. The function returns `-1`.

Call `(*team)("Botha")`: the loop stops at the match.

`i`	`members[i] != NULL`	`getSurName()`	equals `"Botha"`	Action
0	true	`Mokoena`	false	next `i`
1	true	`Dlamini`	false	next `i`
2	true	`Botha`	true	`return 2`

The call `(*team)("Dlamini")` stops at `i` equal to 1 in the same way. The ExpectedOutput of `main.cpp` is:

```
Dlamini: 1
Botha: 2
Nkosi: -1
```

Common mistakes

1. **Leaving out the `NULL` test.** The team in `main.cpp` has an empty slot 3. Without the test, the search for `Nkosi` calls `getSurName()` through a `NULL` pointer, which is undefined behaviour. Valgrind reports `Invalid read of size 8`, and the program stops with a segmentation fault (`SIGSEGV`). The line `Nkosi: -1` is never printed.
2. **Returning the last match.** A loop that does not stop at the first match, and remembers the last index, returns 1 instead of 0 for `Mokoena` in the edge test. The output of the edge test then differs from the expected output.
3. **Returning `-1` inside the loop.** An `else { return -1; }` after the `if` ends the function on the first slot that does not match. The searches for `Dlamini` and `Botha` in `main.cpp` would return `-1` instead of 1 and 2. The `return -1;` belongs after the loop.
4. **Looping to `i <= size`.** The last pass reads `members[size]`, which is outside the array. Valgrind reports `Invalid read of size 8`. The printed output can still look correct, because the value that is read is undefined. Only valgrind shows the mistake.

A_3 Assignment operator: solutions

A3-01. Assignment operator: `Employee::operator=` (100)

File: A_3.cpp

```
#include "Employee.h"

Employee& Employee::operator=(const Employee& other) {
    // 1. Self-assignment check: compare the addresses.
    if (this == &other) {
        return *this;
    }
    // 2. Free the old memory: Employee has no pointers, so there is nothing to delete.
    // 3. Copy every attribute from the other employee.
    firstName = other.firstName;
    surName = other.surName;
    age = other.age;
    gender = other.gender;
    idNum = other.idNum;
    // 4. Return the current object so that a = b = c works.
    return *this;
}
```

Expected output of the given main.cpp

```
Before:
Thandi Mokoena, 29, F, ID: 9702140123081
Sipho Dlamini, 34, M, ID: 9105225800087
After *thandi = *sipho:
Sipho Dlamini, 34, M, ID: 9105225800087
Sipho Dlamini, 34, M, ID: 9105225800087
```

What the function must do

The statement `a = b;` must make the employee `a` hold the same details as the employee `b`. The function copies the five attributes: `firstName`, `surName`, `age`, `gender` and `idNum`. The function does nothing when an object is assigned to itself. The function returns the left-hand object so that `a = b = c;` works.

How the solution works

An **assignment operator** is a function with the name `operator=`. C++ calls this function when the program writes `a = b;`. The object `a` is the current object. The object `b` is the parameter named `other`.

The parameter type is `const Employee&`. The `&` means `other` is a reference: it names the real object `b` and does not make a copy. The word `const` means the function promises not to change `other`.

```
Employee& Employee::operator=(const Employee& other) {
```

The return type is `Employee&`. The function returns a reference to the left-hand object. This reference is what makes chaining possible.

```
    if (this == &other) {
        return *this;
    }
```

`this` is a pointer to the current object. `&other` is the address of the object that `other` names. The condition `this == &other` compares the two addresses. The condition is true only when both sides are the same object, as in `a = a;`.

The check handles self-assignment. In this class, a copy of an object onto itself would do no harm, because `Employee` has no pointers. The check is step 1 of the four-step pattern that the solution follows. In classes that own memory, the same check is necessary. If the condition is true, the function returns `*this` and does no copying. `*this` is the current object itself.

```
    firstName = other.firstName;
    surName = other.surName;
    age = other.age;
    gender = other.gender;
    idNum = other.idNum;
```

The function copies each attribute with `=`. The `string` class and the built-in types already know how to copy themselves. Memory is not a problem here, because `Employee` holds `string`, `int` and `char` only. Nothing is allocated with `new`, so nothing needs `delete`. Step 2 of the pattern (free old memory) is empty for this class.

The function reads `other.firstName` directly. The getters such as `getFirstName()` are not `const` functions. The compiler would refuse to call them on a `const Employee&`. A member function of `Employee` is allowed to read the private attributes of another `Employee` object.

The function copies `idNum` even when it is empty (`""`). If the function skipped an empty `idNum`, the old ID of the left-hand object would stay. The output would then show the wrong ID.

The function does not call `setIDNum`. That function validates the ID against the gender and can throw. A plain copy of `idNum` copies a value that was already valid in `other`.

```
    return *this;
}
```

The last line returns the current object. For `a = b = c;`, C++ runs `b = c` first. That call returns `b`. Then C++ runs `a = b` with that returned object.

Trace

The call is `*thandi = *sipho;` from `main.cpp`. The left side `*thandi` is the current object. The right side `*sipho` is `other`.

Step	Code	firstName	surName	age	gender	idNum
Before	none	Thandi	Mokoena	29	F	9702140123081
1	`this == &other`: the two addresses differ, so the condition is false	Thandi	Mokoena	29	F	9702140123081
2	`firstName = other.firstName;`	Sipho	Mokoena	29	F	9702140123081
3	`surName = other.surName;`	Sipho	Dlamini	29	F	9702140123081
4	`age = other.age;`	Sipho	Dlamini	34	F	9702140123081
5	`gender = other.gender;`	Sipho	Dlamini	34	M	9702140123081
6	`idNum = other.idNum;`	Sipho	Dlamini	34	M	9105225800087
7	`return *this;`	Sipho	Dlamini	34	M	9105225800087

The object `*sipho` does not change in any step. The two objects are separate after the call. A later change to one object does not change the other.

The output of `main.cpp` is:

```
Before:
Thandi Mokoena, 29, F, ID: 9702140123081
Sipho Dlamini, 34, M, ID: 9105225800087
After *thandi = *sipho:
Sipho Dlamini, 34, M, ID: 9105225800087
Sipho Dlamini, 34, M, ID: 9105225800087
```

Common mistakes

- **Skip one attribute.** The test program checks each attribute separately. If the function omits `age`, `firstName`, `surName`, `gender` or `idNum`, the printed line differs from the expected output.
- **Skip an empty `idNum`.** If the function copies `idNum` only when it is not empty, test case 2 prints the old ID `[8807125800083]` (the ID that `a` received in case 1). The expected line shows `[]`.

- **Return `Employee` instead of `Employee&`.** This returns a copy. The statement `Employee& result = (c = b);` then does not compile, because a reference cannot bind to a temporary copy. This also breaks the rule that `operator=` returns the left-hand object.
- **Forget `return *this;`.** The compiler under `-Wall -Werror` reports a warning for a missing return in a function that must return a value. The warning becomes an error, and the program does not compile. The mark is 0.
- **Call the getters on `other`.** `other.getFirstName()` does not compile, because `other` is `const` and the getters are not `const` functions.

A3-02. Assignment operator: `Team::operator=` (100)

File: A_3.cpp

```
#include "Team.h"

Team& Team::operator=(const Team& other) {
    // 1. Self-assignment check: compare the addresses.
    if (this == &other) {
        return *this;
    }

    // 2. Free the old memory: every employee, then the array.
    for (int i = 0; i < size; i++) {
        delete members[i];
    }
    delete[] members;

    // 3. Deep copy: a new array, and a new copy of every employee.
    size = other.size;
    members = new Employee*[size];
    for (int i = 0; i < size; i++) {
        if (other.members[i] == NULL) {
            members[i] = NULL;
        } else {
            members[i] = new Employee(*other.members[i]);
        }
    }

    // 4. Return the current object so that a = b = c works.
    return *this;
}
```

Expected output of the given main.cpp

```
Team a before:
Team of size 2:
  0: Thandi Mokoena, 29, F, ID: none
  1: empty
Team b:
Team of size 3:
  0: Sipho Dlamini, 34, M, ID: none
  1: empty
  2: Ravi Naidoo, 38, M, ID: none
Team a after *a = *b:
Team of size 3:
  0: Sipho Dlamini, 34, M, ID: none
  1: empty
  2: Ravi Naidoo, 38, M, ID: none
Team b after removing member 0:
Team of size 3:
  0: empty
  1: empty
  2: Ravi Naidoo, 38, M, ID: none
Team a has not changed:
Team of size 3:
  0: Sipho Dlamini, 34, M, ID: none
  1: empty
  2: Ravi Naidoo, 38, M, ID: none
```

What the function must do

The statement `*a = *b;` must make team `a` hold its own copy of every employee in team `b`. The two teams must have the same size, and each employee must stay in the same slot number. After the call, the teams share no `Employee` object. The function must also free the memory that team `a` owned before the call.

How the solution works

A `Team` has two members. `members` has type `Employee**`: a pointer to an array, and each array slot holds an `Employee*` (a pointer to one employee) or `NULL`. `size` is the number of slots. The team owns its employees, so the team must delete them.

A **shallow copy** copies the pointers only. After a shallow copy, both teams point at the same `Employee` objects. Both destructors would then delete the same objects, and valgrind reports a double delete. A **deep copy** makes new `Employee` objects. The function must make a deep copy.

```
Team& Team::operator=(const Team& other) {
```

`other` is a reference to the right-hand team. The word `const` means the function does not change `other`. The function returns `Team&`, a reference to the left-hand team.

```
    if (this == &other) {
        return *this;
    }
```

`this` is a pointer to the current object (the left-hand team). `&other` is the address of the right-hand team. The condition is true when both are the same team, as in `a = a;`. The check must come first, because the next step deletes the memory that `other` would use. In `a = a`, `other` is the current team itself. Without the check, the function deletes the employees and the array. The line `members = new Employee*[size];` then changes `other.members` too, because `other` is the same object. The copy loop reads the slots of this new array, and these slots hold no set values. The employees are lost. If the condition is true, the function returns `*this` (the current object) and changes nothing.

```
    for (int i = 0; i < size; i++) {
        delete members[i];
    }
    delete[] members;
```

This step frees the old memory of the current team. The loop runs from slot `0` to slot `size - 1` and deletes each employee. A slot that holds `NULL` is safe: `delete` on a `NULL` pointer does nothing. Without the loop, the old employees stay in memory and valgrind reports a leak. `delete[] members` frees the array of pointers. The function must delete the employees first. After `delete[] members`, the function cannot read `members[i]` any more.

The deletion loop needs the old `size`. For this reason, the line `size = other.size;` comes after the deletions.

```
    size = other.size;
    members = new Employee*[size];
```

The function reads `other.size` and `other.members` directly. The getters `getSize()` and `getMemberCount()` are not `const` functions, so the function cannot call them on a `const Team&`. The first line copies the size. The second line allocates a new array of that many `Employee*` slots. The slots hold no valid values yet. The loop must set every slot.

```
    for (int i = 0; i < size; i++) {
        if (other.members[i] == NULL) {
            members[i] = NULL;
        } else {
            members[i] = new Employee(*other.members[i]);
        }
    }
```

The loop visits every slot number `i`. The `if` checks whether the slot in `other` is empty. If the slot is `NULL`, the new slot becomes `NULL`. The function must not dereference `NULL`. The expression `*other.members[i]` on a `NULL` pointer would crash. If the slot holds an employee, `new Employee(*other.members[i])` creates a new employee. `*other.members[i]` is the employee object that the pointer points to. The `Employee` copy constructor copies all its attributes. The new slot then points at the new object, in the same slot number `i`.

When `size` is `0`, the loop does not run and `new Employee*[0]` gives an empty array. The destructor can delete it.

```
    return *this;
}
```

The function returns the current team. For `a = b = c;`, C++ first runs `b = c`, and then `a = b`.

Trace

The call is `*a = *b;` from `main.cpp`. Before the call, team `a` has size 2 and team `b` has size 3. Employee `Sipho` is the object `S`, `Ravi` is `R`, and `Thandi` is `T`. In `b`, slot 1 is `NULL` because of `removeMember(1)`.

Step	Code	size	members of a
Before	none	2	`[ T, NULL ]` (one array of 2 slots)
1	`this == &other`: the two addresses differ, so the condition is false	2	`[ T, NULL ]`
2	loop `i = 0`: `delete members[0]` frees `T`	2	`[ freed, NULL ]`
3	loop `i = 1`: `delete members[1]` on `NULL` does nothing	2	`[ freed, NULL ]`
4	`delete[] members;` frees the old array	2	no array
5	`size = other.size;`	3	no array
6	`members = new Employee*[size];`	3	new array of 3 slots, values not set
7	`i = 0`: `other.members[0]` is not `NULL`, so the slot gets a new copy `S'` of `S`	3	`[ S', ?, ? ]`
8	`i = 1`: `other.members[1]` is `NULL`, so `members[1] = NULL`	3	`[ S', NULL, ? ]`
9	`i = 2`: `other.members[2]` is not `NULL`, so the slot gets a new copy `R'` of `R`	3	`[ S', NULL, R' ]`
10	`return *this;`	3	`[ S', NULL, R' ]`

The objects `S'` and `R'` are new. Team `b` still points at `S` and `R`. After `b->removeMember(0)`, team `b` deletes `S`, but `a` still holds `S'`. The output shows that team `a` has not changed.

The output of `main.cpp` is:

```
Team a before:
Team of size 2:
  0: Thandi Mokoena, 29, F, ID: none
  1: empty
Team b:
Team of size 3:
  0: Sipho Dlamini, 34, M, ID: none
  1: empty
  2: Ravi Naidoo, 38, M, ID: none
Team a after *a = *b:
Team of size 3:
  0: Sipho Dlamini, 34, M, ID: none
  1: empty
  2: Ravi Naidoo, 38, M, ID: none
Team b after removing member 0:
Team of size 3:
  0: empty
  1: empty
  2: Ravi Naidoo, 38, M, ID: none
Team a has not changed:
Team of size 3:
  0: Sipho Dlamini, 34, M, ID: none
  1: empty
  2: Ravi Naidoo, 38, M, ID: none
```

Common mistakes

- **Shallow copy.** The code `members[i] = other.members[i];` copies the pointers. Both teams then own the same employees. The destructors delete each employee twice, and valgrind reports invalid frees. This happens in `main.cpp`.
- **No self-assignment check.** For `a = a;`, the function deletes the employees and the array of the team. Because `other` is the same team, the copy loop then reads the new array, and its slots hold no set values. In the edge test (test 1, `a = a`), valgrind reports "Conditional jump or move depends on uninitialised value(s)". The employee Lebo is lost, and the team prints `0: empty`.
- **Delete only the array.** If the function runs `delete[] members;` without the loop, the old employees have no pointer any more. Valgrind reports a leak.
- **Copy a `NULL` slot with `new Employee(*other.members[i])`.** The function dereferences a `NULL` pointer, and the program stops with a segmentation fault. The `if` for `NULL` slots prevents this.

- **Keep the old `size`.** If the function does not set `size = other.size;`, the new array has the wrong length. The copy has the wrong slot count. If the source team is smaller than the old size, the copy loop reads beyond the end of the source array.

A3-03. Assignment operator: `Team::operator=` with a compact copy (100)

File: A_3.cpp

```
#include "Team.h"

Team& Team::operator=(const Team& other) {
    // 1. Self-assignment check: compare the addresses.
    if (this == &other) {
        return *this;
    }

    // 2. Free the old memory: every employee, then the array.
    for (int i = 0; i < size; i++) {
        delete members[i];
    }
    delete[] members;

    // 3. The new size is the number of employees in the other team.
    int count = 0;
    for (int i = 0; i < other.size; i++) {
        if (other.members[i] != NULL) {
            count++;
        }
    }
    size = count;
    members = new Employee*[size];

    // 4. Copy every employee into the next free slot, keeping the order.
    int next = 0;
    for (int i = 0; i < other.size; i++) {
        if (other.members[i] != NULL) {
            members[next] = new Employee(*other.members[i]);
            next++;
        }
    }

    // 5. Return the current object so that a = b = c works.
    return *this;
}
```

Expected output of the given main.cpp

```
Team a before:
Team of size 2:
  0: Thandi Mokoena, 29, F, ID: none
  1: empty
Team b:
Team of size 4:
  0: Sipho Dlamini, 34, M, ID: none
  1: empty
  2: Ravi Naidoo, 38, M, ID: none
  3: empty
Team a after *a = *b:
Team of size 2:
  0: Sipho Dlamini, 34, M, ID: none
  1: Ravi Naidoo, 38, M, ID: none
a: size 2, members 2
Team c after *d = *c = *b:
Team of size 2:
  0: Sipho Dlamini, 34, M, ID: none
  1: Ravi Naidoo, 38, M, ID: none
Team d after *d = *c = *b:
Team of size 2:
  0: Sipho Dlamini, 34, M, ID: none
  1: Ravi Naidoo, 38, M, ID: none
Team b after removing member 0:
Team of size 4:
  0: empty
  1: empty
  2: Ravi Naidoo, 38, M, ID: none
  3: empty
Team a has not changed:
```

```
Team of size 2:
  0: Sipho Dlamini, 34, M, ID: none
  1: Ravi Naidoo, 38, M, ID: none
```

What the function must do

The statement `*a = *b;` must give team `a` its own copy of every employee in team `b`. The copy is compact. The new `size` equals the number of employees in `b`, and the copies fill slots `0, 1, 2, ...` in the same order as in `b`. The new team has no `NULL` slots. A normal deep copy keeps the slot numbers and the empty slots. This question does not.

How the solution works

A `Team` has `Employee** members` (an array of `Employee*` pointers) and `int size` (the number of slots). A slot holds a pointer to an employee or `NULL`. The team owns its employees and must delete them. A **deep copy** creates new `Employee` objects. A **shallow copy** copies only the pointers, so two teams would point at the same employees and delete them twice.

```
Team& Team::operator=(const Team& other) {
```

`other` is a reference to the right-hand team. The word `const` means the function does not change `other`. The function returns `Team&`, a reference to the left-hand team.

```
    if (this == &other) {
        return *this;
    }
```

`this` is a pointer to the current object (the left-hand team). `&other` is the address of the right-hand team. The condition is true for `a = a;`. The check must come first, because the next step deletes the old memory. In `a = a`, the function would delete the employees and then try to count and copy them. The function would read freed memory. If the condition is true, the function returns `*this` (the current object). The team keeps its empty slots, as test 1 requires.

```
    for (int i = 0; i < size; i++) {
        delete members[i];
    }
    delete[] members;
```

The loop deletes every employee of the current team. A `NULL` slot is safe, because `delete` on a `NULL` pointer does nothing. Without the loop, valgrind reports a leak. `delete[] members` frees the array of pointers. The employees must be deleted first, because the function cannot read the array after `delete[]`.

```
    int count = 0;
    for (int i = 0; i < other.size; i++) {
        if (other.members[i] != NULL) {
            count++;
        }
    }
    size = count;
    members = new Employee*[size];
```

The function reads `other.size` and `other.members` directly. The getters `getSize()` and `getMemberCount()` are not `const` functions, so the function cannot call them on a `const Team&`.

The loop runs over all slots of `other`. The condition `other.members[i] != NULL` checks that the slot holds an employee. Only those slots are counted. The loop uses `other.size`, not `size`, because the new size is not known yet and the old `size` belongs to the old array.

The function then sets `size = count;` and allocates an array of that length. If `other` has no employees, `count` is `0`, and the array has zero slots. The team then has size `0`. The destructor can still delete this array.

The count must come before `new`, because the array length must be known when the array is created.

```
    int next = 0;
    for (int i = 0; i < other.size; i++) {
        if (other.members[i] != NULL) {
            members[next] = new Employee(*other.members[i]);
            next++;
```

```
        }
    }
```

Two counters are used. `i` is the slot number in `other`. `next` is the slot number in the new array. The condition `other.members[i] != NULL` skips empty slots, so the function does not dereference a `NULL` pointer. `*other.members[i]` is the employee that the pointer points to. `new Employee(*other.members[i])` calls the `Employee` copy constructor to create a new employee. The new pointer goes to `members[next]`. Then `next` increases by one.

`next` increases only when the function copies an employee. For this reason, the new array has no gaps. The number of copies equals `count`, so every slot of the new array gets a value.

```
    return *this;
}
```

The function returns the current team. For `*d = *c = *b;`, C++ runs `*c = *b` first, and then `*d = *c`.

Trace

The call is `*a = *b;` from `main.cpp`. Before the call, `a` has size 2 with `[ Thandi, NULL ]`. Team `b` has size 4 with `[ Sipho, NULL, Ravi, NULL ]`. The slot 1 of `b` is `NULL` because of `b->removeMember(1)`. `Sipho'` and `Ravi'` are new copies.

Step	Code	`count`	`next`	`size`	`members` of `a`
Before	none	none	none	2	`[ Thandi, NULL ]`
1	`this == &other`: the addresses differ, so the condition is false	none	none	2	`[ Thandi, NULL ]`
2	the first loop deletes `Thandi`; `delete` on `NULL` does nothing; `delete[] members`	none	none	2	no array
3	count loop, `i = 0`: not `NULL`	1	none	2	no array
4	`i = 1`: `NULL`	1	none	2	no array
5	`i = 2`: not `NULL`	2	none	2	no array
6	`i = 3`: `NULL`	2	none	2	no array
7	`size = count;` and `new Employee*[size]`	2	none	2	new array of 2 slots, values not set
8	copy loop, `i = 0`: not `NULL`, so `members[0]` is a copy `Sipho'`; `next` becomes 1	2	1	2	`[ Sipho', ? ]`
9	`i = 1`: `NULL`, nothing happens	2	1	2	`[ Sipho', ? ]`
10	`i = 2`: not `NULL`, so `members[1]` is a copy `Ravi'`; `next` becomes 2	2	2	2	`[ Sipho', Ravi' ]`
11	`i = 3`: `NULL`, nothing happens; `return *this;`	2	2	2	`[ Sipho', Ravi' ]`

Team `b` is unchanged: it still has size 4 and points at `Sipho` and `Ravi`. After `b->removeMember(0)`, team `b` deletes `Sipho`, but team `a` still holds `Sipho'`.

The output of `main.cpp` is:

```
Team a before:
Team of size 2:
  0: Thandi Mokoena, 29, F, ID: none
  1: empty
Team b:
Team of size 4:
  0: Sipho Dlamini, 34, M, ID: none
  1: empty
  2: Ravi Naidoo, 38, M, ID: none
  3: empty
Team a after *a = *b:
Team of size 2:
  0: Sipho Dlamini, 34, M, ID: none
  1: Ravi Naidoo, 38, M, ID: none
a: size 2, members 2
Team c after *d = *c = *b:
```

```
Team of size 2:
  0: Sipho Dlamini, 34, M, ID: none
  1: Ravi Naidoo, 38, M, ID: none
Team d after *d = *c = *b:
Team of size 2:
  0: Sipho Dlamini, 34, M, ID: none
  1: Ravi Naidoo, 38, M, ID: none
Team b after removing member 0:
Team of size 4:
  0: empty
  1: empty
  2: Ravi Naidoo, 38, M, ID: none
  3: empty
Team a has not changed:
Team of size 2:
  0: Sipho Dlamini, 34, M, ID: none
  1: Ravi Naidoo, 38, M, ID: none
```

Common mistakes

- **Normal deep copy.** If the function sets `size = other.size;` and keeps the empty slots, team `a` prints `Team of size 4` with `empty` lines. The expected output shows size 2. The output then differs from the expected output.
- **Use `i` for the new array.** If the function writes `members[i] = ...` in the copy loop, each copy keeps its old slot number. In `main.cpp`, the copy of Ravi goes to `members[2]`, but the new array has only 2 slots. This writes past the end of the array, and slot 1 is never set. This is undefined behaviour. In the test runs, valgrind reported "Invalid write of size 8", "Conditional jump or move depends on uninitialised value(s)" and a leak ("definitely lost"). The output showed `1: empty` where Ravi should be.
- **Allocate before counting.** If the function calls `new Employee*[size]` before it sets `size = count;`, the array has the length of the old team, not `count`. When the old team has fewer slots than `count`, the copy loop writes past the end of the array. This is undefined behaviour. In the test runs, the output still looked correct, but valgrind reported "Invalid write of size 8" and "Invalid read of size 8" on both tests.
- **Shallow copy.** The code `members[next] = other.members[i];` copies the pointer. Both teams then own the same employee, and the destructors delete the employee twice. Valgrind reports invalid frees.
- **No self-assignment check.** For `a = a;`, the count loop reads the array that the function deleted in the step before, and valgrind reports "Invalid read of size 8". The copy loop then reads the new array, whose slots hold no set values, and valgrind reports "Conditional jump or move depends on uninitialised value(s)". In test 1, the team prints `Team of size 1:` and `0: empty`. Lebo and the empty slots are lost.

A3-04. Assignment operator: `Department::operator=` (100)

File: A_3.cpp

```
#include "Department.h"

Department& Department::operator=(const Department& other) {
    // 1. Self-assignment check: compare the addresses.
    if (this == &other) {
        return *this;
    }

    // 2. Free the old memory: every team (each team deletes its employees), then the array.
    for (int i = 0; i < size; i++) {
        delete teams[i];
    }
    delete[] teams;

    // 3. Deep copy: a new array, and a new copy of every team.
    size = other.size;
    teams = new Team*[size];
    for (int i = 0; i < size; i++) {
        if (other.teams[i] == NULL) {
            teams[i] = NULL;
        } else {
            teams[i] = new Team(*other.teams[i]);
        }
    }

    // 4. Return the current object so that a = b = c works.
    return *this;
}
```

Expected output of the given main.cpp

```
Department a before:
Department of size 1:
Slot 0: Team of size 2:
  0: Thandi Mokoena, 29, F, ID: none
  1: empty
Department b:
Department of size 3:
Slot 0: Team of size 2:
  0: Sipho Dlamini, 34, M, ID: none
  1: Anna Botha, 41, F, ID: none
Slot 1: empty
Slot 2: Team of size 1:
  0: Lebo Nkosi, 25, F, ID: none
Department a after *a = *b:
Department of size 3:
Slot 0: Team of size 2:
  0: Sipho Dlamini, 34, M, ID: none
  1: Anna Botha, 41, F, ID: none
Slot 1: empty
Slot 2: Team of size 1:
  0: Lebo Nkosi, 25, F, ID: none
Department b after the changes:
Department of size 3:
Slot 0: Team of size 2:
  0: Sipho Dlamini, 34, M, ID: none
  1: empty
Slot 1: empty
Slot 2: empty
Department a has not changed:
Department of size 3:
Slot 0: Team of size 2:
  0: Sipho Dlamini, 34, M, ID: none
  1: Anna Botha, 41, F, ID: none
Slot 1: empty
Slot 2: Team of size 1:
  0: Lebo Nkosi, 25, F, ID: none
```

What the function must do

The statement `*a = *b;` must make department `a` hold its own copy of every team in department `b`. The sizes become equal, and each team stays in the same slot number. The copy goes two levels deep: every `Team` is new, and every `Employee` inside each team is new. The function must free the teams that department `a` owned before the call.

How the solution works

A `Department` has `Team** teams` (an array of `Team*` pointers) and `int size` (the number of slots). A slot holds a pointer to a team or `NULL`. The department owns its teams. `~Department()` deletes each team, and each `~Team()` deletes its own employees.

A **shallow copy** copies only the pointers, so two departments would point at the same `Team` objects and delete them twice. A **deep copy** creates new objects. The function does not copy employees itself. The given `Team(const Team&)` constructor makes a deep copy of one team, including a new copy of every employee. The function calls this constructor.

```
Department& Department::operator=(const Department& other) {
```

`other` is a reference to the right-hand department. The word `const` means the function does not change `other`. The function returns `Department&`, a reference to the left-hand department.

```
    if (this == &other) {
        return *this;
    }
```

`this` is a pointer to the current object (the left-hand department). `&other` is the address of the right-hand department. The condition is true for `a = a;`. The check must come first, because the next step deletes the teams that `other` would supply. In `a = a`, `other` is the current department itself. Without the check, the function deletes every team and the array. The line `teams = new Team*[size];` then changes `other.teams` too, because `other` is the same object. The copy loop reads the slots of this new array, and these slots hold no set values. The teams are lost. If the condition is true, the function returns `*this` (the current object) and changes nothing.

```
    for (int i = 0; i < size; i++) {
        delete teams[i];
    }
    delete[] teams;
```

The loop deletes every team of the current department. Each `delete` runs `~Team()`, and that destructor deletes the employees of the team. A slot that holds `NULL` is safe, because `delete` on a `NULL` pointer does nothing. Without the loop, valgrind reports a leak for every old team and employee. `delete[] teams` frees the array of pointers. The teams must be deleted first, because the function cannot read the array after `delete[]`.

```
    size = other.size;
    teams = new Team*[size];
```

The loop above needs the old `size`, so the new size is set after the loop. The function reads `other.size` and `other.teams` directly. A member function of `Department` is allowed to read the private members of another `Department`. The first line copies the size. The second line allocates a new array of that many `Team*` slots. The slots hold no valid values yet, so the next loop must set every slot.

```
    for (int i = 0; i < size; i++) {
        if (other.teams[i] == NULL) {
            teams[i] = NULL;
        } else {
            teams[i] = new Team(*other.teams[i]);
        }
    }
```

The loop visits every slot number `i`. The `if` checks whether the slot in `other` is empty. If the slot is `NULL`, the new slot becomes `NULL`. The function must not dereference `NULL`, because `*other.teams[i]` on a `NULL` pointer crashes. If the slot holds a team, `*other.teams[i]` is the team object that the pointer points to. `new Team(*other.teams[i])` calls the `Team` copy constructor, which creates a new team with new employees. The new slot points at the new team, in the same slot number `i`.

The condition tests for `NULL` only. A team with no employees is not `NULL`, so the function copies it too (test 6 checks this). When `size` is `0`, the loop does not run, and the department has an array with zero slots.

```
    return *this;
}
```

The function returns the current department. For `x = y = z;`, C++ runs `y = z` first, and then `x = y`.

Trace

The call is `*a = *b;` from `main.cpp`. Before the call, `a` has size 1 and holds one team `t1`, which holds `Thandi`. Department `b` has size 3. The slots are `t2` (with `Sipho` and `Anna`), `NULL`, and `t4` (with `Lebo`). The slot 1 of `b` is `NULL` because `main.cpp` deleted that team and set the slot to `NULL`. `t2'` and `t4'` are new copies.

Step	Code	size	teams of a
Before	none	1	`[ t1 ]`
1	`this == &other`: the addresses differ, so the condition is false	1	`[ t1 ]`
2	loop `i = 0`: `delete teams[0]` frees `t1` and `Thandi`	1	`[ freed ]`
3	`delete[] teams;` frees the old array	1	no array
4	`size = other.size;`	3	no array
5	`teams = new Team*[size];`	3	new array of 3 slots, values not set
6	`i = 0`: `other.teams[0]` is not `NULL`, so the slot gets `t2'`, a copy of `t2` with new copies of `Sipho` and `Anna`	3	`[ t2', ?, ? ]`
7	`i = 1`: `other.teams[1]` is `NULL`, so `teams[1] = NULL`	3	`[ t2', NULL, ? ]`
8	`i = 2`: `other.teams[2]` is not `NULL`, so the slot gets `t4'`, a copy of `t4` with a new copy of `Lebo`	3	`[ t2', NULL, t4' ]`
9	`return *this;`	3	`[ t2', NULL, t4' ]`

Department `b` still points at `t2` and `t4`. Later, `main.cpp` runs `(*b)[0]->removeMember(1)`, which deletes `Anna` from `t2`. Then it deletes `t4` and sets slot 2 of `b` to `NULL`. These changes affect `b`, `t2` and `t4` only. Department `a` holds `t2'` and `t4'`, so the output shows that `a` has not changed.

The output of `main.cpp` is:

```
Department a before:
Department of size 1:
Slot 0: Team of size 2:
  0: Thandi Mokoena, 29, F, ID: none
  1: empty
Department b:
Department of size 3:
Slot 0: Team of size 2:
  0: Sipho Dlamini, 34, M, ID: none
  1: Anna Botha, 41, F, ID: none
Slot 1: empty
Slot 2: Team of size 1:
  0: Lebo Nkosi, 25, F, ID: none
Department a after *a = *b:
Department of size 3:
Slot 0: Team of size 2:
  0: Sipho Dlamini, 34, M, ID: none
  1: Anna Botha, 41, F, ID: none
Slot 1: empty
Slot 2: Team of size 1:
  0: Lebo Nkosi, 25, F, ID: none
Department b after the changes:
Department of size 3:
Slot 0: Team of size 2:
  0: Sipho Dlamini, 34, M, ID: none
  1: empty
Slot 1: empty
Slot 2: empty
Department a has not changed:
Department of size 3:
Slot 0: Team of size 2:
```

```
  0: Sipho Dlamini, 34, M, ID: none
  1: Anna Botha, 41, F, ID: none
Slot 1: empty
Slot 2: Team of size 1:
  0: Lebo Nkosi, 25, F, ID: none
```

Common mistakes

- **Shallow copy.** The code `teams[i] = other.teams[i];` copies the pointers. After `main.cpp` deletes `t4` from `b`, department `a` holds a pointer to freed memory. Printing `a` reads freed memory, and the destructors delete the same teams twice. Valgrind reports invalid reads and invalid frees.
- **No self-assignment check.** For `a = a;`, the function deletes the teams and the array. Because `other` is the same department, the copy loop then reads the new array, and its slots hold no set values. In the edge test (test 1, `a = a`), valgrind reports "Conditional jump or move depends on uninitialised value(s)". The team of Lebo is lost, and the department prints `Slot 0: empty`.
- **Delete only the array.** If the function runs `delete[] teams;` without the loop, the old teams and employees have no pointer any more. Valgrind reports a leak.
- **Copy a `NULL` slot with `new Team(*other.teams[i])`.** The function dereferences a `NULL` pointer, and the program stops with a segmentation fault. The `if` for `NULL` slots prevents this.
- **Skip a team that has no employees.** The condition must test the team pointer, not the number of employees. A function that skips empty teams leaves a `NULL` slot where the original has a team. Test 6 (`d = e`) then prints `Slot 0: empty` instead of a team of size 2.

A3-05. Assignment operator: `Sales::operator=` in a derived class (100)

File: A_3.cpp

```
#include "Sales.h"

Sales& Sales::operator=(const Sales& other) {
    // 1. Self-assignment check: compare the addresses.
    if (this == &other) {
        return *this;
    }
    // 2. The base class copies its own part (the teams and the size).
    Department::operator=(other);
    // 3. Copy the member that Sales adds.
    target = other.target;
    // 4. Return the current object so that a = b = c works.
    return *this;
}
```

Expected output of the given main.cpp

```
Sales a before:
Sales of size 1:
Slot 0: Team of size 1:
  0: Thandi Mokoena, 29, F, ID: none
Target: 100, cost: R15000
Sales b:
Sales of size 2:
Slot 0: Team of size 2:
  0: Sipho Dlamini, 34, M, ID: none
  1: Anna Botha, 41, F, ID: none
Slot 1: empty
Target: 250, cost: R30000
Sales a after *a = *b:
Sales of size 2:
Slot 0: Team of size 2:
  0: Sipho Dlamini, 34, M, ID: none
  1: Anna Botha, 41, F, ID: none
Slot 1: empty
Target: 250, cost: R30000
Sales b after the changes:
Sales of size 2:
Slot 0: Team of size 2:
  0: empty
  1: Anna Botha, 41, F, ID: none
Slot 1: empty
Target: 300, cost: R15000
Sales a has not changed:
Sales of size 2:
Slot 0: Team of size 2:
  0: Sipho Dlamini, 34, M, ID: none
  1: Anna Botha, 41, F, ID: none
Slot 1: empty
Target: 250, cost: R30000
```

Hints

A3-05 hints

Try the question first. Read one hint at a time. The hints go from gentle to specific. They do not give the full answer.

Hint 1

Step 2 is the only new idea. A `Sales` object has a `Department` part, and that part has private attributes. Which function that is already written can copy them?

Hint 2

Inside a member function of `Sales`, you can call a function of the base class by writing the class name and `::` in front of the function name. For example, `Department::getSize()` calls the `Department` version.

Hint 3

A `Sales` object is a `Department`, so `other` (a `const Sales&`) can be passed where a `const Department&` is expected. The step-2 line is: `Department::operator=(other);`. Steps 1, 3 and 4 are the same as in any assignment operator.

What the function must do

`Sales` inherits from `Department` and adds one attribute, `target`. The statement `*a = *b;` must copy the `Department` part (the teams and the size) and the `Sales` part (`target`). The `Department` part uses private members, so `Sales` cannot copy it directly. The function must ask the base class to copy its own part.

How the solution works

A **derived class** such as `Sales` contains a base part (the `Department` members `teams` and `size`) and its own members (`target`). The members `teams` and `size` are `private` in `Department`. Code in `Sales` cannot read or write them.

The given `Department::operator=` already makes a deep copy of `teams` and sets `size`. It deletes the old teams, creates a new array, and copies each team with the `Team` copy constructor. `Sales::operator=` must call that function. The function must not repeat the work.

```
Sales& Sales::operator=(const Sales& other) {
```

`other` is a reference to the right-hand `Sales` object. The word `const` means the function does not change `other`. The function returns `Sales&`, a reference to the left-hand object.

```
    if (this == &other) {
        return *this;
    }
```

`this` is a pointer to the current object. `&other` is the address of the right-hand object. The condition is true for `a = a;`. In that case the function returns `*this` (the current object) and does nothing else. The given `Department::operator=` has its own check, so a self-assignment would not damage the data here. The question lists this check as step 1, and the same step protects classes that own memory directly.

```
    Department::operator=(other);
```

This line calls the base class version of the assignment operator. The text `Department::` in front of the function name tells the compiler to call the `Department` version. Without it, the call `operator=(other)` would be the `Sales` version, which would call itself without end.

The argument `other` has type `const Sales&`. A `Sales` object is a `Department`, so C++ allows a `const Sales&` where a `const Department&` is expected. The base function receives only the `Department` part of `other`. The base function frees the old teams of the current object. It then copies `size` and makes new copies of the teams and the employees.

C++ does not call the base class assignment operator by itself when a derived class defines its own `operator=`. If the line is missing, the `Department` part of the current object keeps its old teams and old size.

The order of the base call and `target = other.target;` does not change the result, because the two parts are independent. The solution follows the order of the question.

```
    target = other.target;
```

`target` is an `int`. A plain `=` copies it. `target` is a member of `Sales`, so the base function does not copy it. The function reads `other.target` directly, because a member function of `Sales` is allowed to read the private members of another `Sales` object.

```
    return *this;
}
```

The function returns the current object. For `a = b = c;`, C++ runs `b = c` first, and then `a = b`.

Trace

The call is `*a = *b;` from `main.cpp`. Before the call, `a` has size 1 and target 100, and holds the team `t1` (with `Thandi`). Object `b` has size 2 and target 250. Its slots are `t2` (with `Sipho` and `Anna`) and `NULL`. `t2'` is a new copy of `t2`.

Step	Code	size	teams of a	target of a
Before	none	1	`[ t1 ]`	100
1	`this == &other`: the addresses differ, so the condition is false	1	`[ t1 ]`	100
2	`Department::operator=(other);` runs: it deletes `t1` and the old array	1	no array	100
3	in the base function: `size = other.size;` and a new array of 2 slots	2	new array of 2 slots	100
4	in the base function: slot 0 gets `t2'` (a copy of `t2` with new employees); slot 1 is `NULL`	2	`[ t2', NULL ]`	100
5	`target = other.target;`	2	`[ t2', NULL ]`	250
6	`return *this;`	2	`[ t2', NULL ]`	250

Object `b` still holds `t2`. Later, `main.cpp` runs `(*b)[0]->removeMember(0)` and `b->setTarget(300)`. These calls change `t2` and the target of `b`. Object `a` holds `t2'` and its own target 250, so `a` does not change. The cost line uses the number of employees times 15000, so `a` shows `R30000` and `b` shows `R15000` after the change.

The output of `main.cpp` is:

```
Sales a before:
Sales of size 1:
Slot 0: Team of size 1:
  0: Thandi Mokoena, 29, F, ID: none
Target: 100, cost: R15000
Sales b:
Sales of size 2:
Slot 0: Team of size 2:
  0: Sipho Dlamini, 34, M, ID: none
  1: Anna Botha, 41, F, ID: none
Slot 1: empty
Target: 250, cost: R30000
Sales a after *a = *b:
Sales of size 2:
Slot 0: Team of size 2:
  0: Sipho Dlamini, 34, M, ID: none
  1: Anna Botha, 41, F, ID: none
Slot 1: empty
Target: 250, cost: R30000
Sales b after the changes:
Sales of size 2:
Slot 0: Team of size 2:
  0: empty
  1: Anna Botha, 41, F, ID: none
Slot 1: empty
Target: 300, cost: R15000
Sales a has not changed:
Sales of size 2:
Slot 0: Team of size 2:
  0: Sipho Dlamini, 34, M, ID: none
  1: Anna Botha, 41, F, ID: none
Slot 1: empty
Target: 250, cost: R30000
```

Common mistakes

- **Leave out the base call.** Without `Department::operator=(other);`, the teams and the size of the current object do not change. The output of `a` after the assignment shows the old size and the old team. The output then differs from the expected output.
- **Leave out `target = other.target;`.** The base part is copied, but `target` keeps the old value. The line `Target: 250` prints `Target: 100`. The output then differs from the expected output.
- **Try to copy `teams` and `size` in `Sales`.** The members are `private` in `Department`. The code does not compile, and the mark is 0.

- **Call `operator=(other)` without `Department::`.** The call finds `Sales::operator=` and calls it again with the same arguments. The function then calls itself without end, and the program stops with a stack overflow.
- **Return a value instead of a reference.** If the return type is `Sales` and not `Sales&`, the function returns a copy. The `Department` copy constructor is private, so the code does not compile.

A3-06. Assignment operator: `List<T>::operator= (100)`

File: A_3.cpp

```
#ifndef A_3_CPP
#define A_3_CPP

#include "List.h"

template <class T>
List<T>& List<T>::operator=(const List<T>& other) {
    // 1. Self-assignment check: compare the addresses.
    if (this == &other) {
        return *this;
    }

    // 2. Free the old nodes. Each node deletes its own data.
    Node<T>* curr = headNode;
    while (curr != NULL) {
        Node<T>* temp = curr;
        curr = curr->next();
        delete temp;
    }
    headNode = NULL;
    size = 0;

    // 3. Copy the other list node by node, keeping the order.
    //    tail is the last node of this list, so each new node goes at the end.
    Node<T>* tail = NULL;
    Node<T>* otherCurr = other.headNode;
    while (otherCurr != NULL) {
        Node<T>* newNode = new Node<T>(new T(otherCurr->getData()));
        if (tail == NULL) {
            headNode = newNode;
        } else {
            tail->next() = newNode;
        }
        tail = newNode;
        size++;
        otherCurr = otherCurr->next();
    }

    // 4. Return the current object so that a = b = c works.
    return *this;
}

#endif
```

Expected output of the given main.cpp

```
a: 9 (size 1)
b: 1->2->3 (size 3)
a after a = b: 1->2->3 (size 3)
b after b.prepend(0): 0->1->2->3 (size 4)
a has not changed: 1->2->3 (size 3)
```

What the function must do

The statement `a = b;` must make list `a` hold new nodes with copies of the values of list `b`, in the same order. The function first deletes the old nodes of `a`. The function then builds a new chain of nodes, one node for each node in `b`. After the call, `a` and `b` share no node and no data, and `size` equals the size of `b`.

How the solution works

`List<T>` is a **class template**. The name `T` is a template parameter: a placeholder for the type of the stored values, such as `int` or `string`. The compiler creates a separate version of the code for each type that the program uses.

A list has `Node<T>* headNode` (a pointer to the first node, or `NULL` for an empty list) and `int size`. Each `Node<T>` has `T* data` (a pointer to the value) and `Node<T>* nextNode` (a pointer to the next node). A node owns its `data`: `~Node()` deletes `data`, but does not delete the next node.

```
template <class T>
List<T>& List<T>::operator=(const List<T>& other) {
```

The first line says that the function is a template. `T` is the type of the values. The parameter `other` is a reference to the right-hand list. The word `const` means the function does not change `other`. The function returns `List<T>&`, a reference to the left-hand list.

```
    if (this == &other) {
        return *this;
    }
```

`this` is a pointer to the current list. `&other` is the address of the right-hand list. The condition is true for `a = a;`. The check must come first, because the next step deletes the nodes that `other` would supply. In `a = a`, `other` is the current list itself. Without the check, the function deletes every node and sets `headNode` to `NULL`. Then `other.headNode` is `NULL` too, because `other` is the same object. The copy loop does not run, and the list becomes empty. If the condition is true, the function returns `*this` (the current list) and changes nothing.

```
    Node<T>* curr = headNode;
    while (curr != NULL) {
        Node<T>* temp = curr;
        curr = curr->next();
        delete temp;
    }
    headNode = NULL;
    size = 0;
```

This step frees the old nodes. `curr` points at the node that the loop visits. The condition `curr != NULL` stops the loop after the last node. An empty list has `headNode` equal to `NULL`, so the loop does not run, and the function does not dereference `NULL`.

Inside the loop, the order of the three lines matters. `temp = curr` saves the current node. `curr = curr->next()` moves `curr` to the next node. `delete temp` then deletes the saved node and its data. The function must move `curr` before it deletes. If the function deleted the node first, `curr->next()` would read a deleted node.

After the loop, every old node is gone, but `headNode` still holds the address of the first deleted node. The line `headNode = NULL;` removes this pointer to freed memory. The line `size = 0;` makes the list empty. The copy step starts with an empty list.

```
    Node<T>* tail = NULL;
    Node<T>* otherCurr = other.headNode;
```

`tail` points at the last node of the new chain. It starts as `NULL` because the new chain has no node. `otherCurr` points at the node of `other` that the loop visits. The function reads `other.headNode` directly. A member function of `List<T>` is allowed to read the private members of another `List<T>`.

```
    while (otherCurr != NULL) {
        Node<T>* newNode = new Node<T>(new T(otherCurr->getData()));
```

The condition `otherCurr != NULL` stops the loop after the last node of `other`. If `other` is empty, the loop does not run, and the current list stays empty. `otherCurr->getData()` gives the value of the node. `new T(...)` creates a new value from a copy of it. `new Node<T>(...)` creates a new node that owns this new value. The new node does not share the value with `other`.

```
        if (tail == NULL) {
            headNode = newNode;
        } else {
            tail->next() = newNode;
        }
        tail = newNode;
        size++;
        otherCurr = otherCurr->next();
    }
```

The `if` handles the first node. If `tail == NULL`, the chain has no node yet, so the new node becomes `headNode`. Otherwise, `tail->next() = newNode;` links the new node after the last node. The function `next()` returns a reference to `nextNode`, so the assignment changes the real pointer. The line `tail = newNode;` makes the new node the last node. `size++` counts the node. `otherCurr = otherCurr->next();` moves to the next node of `other`.

The function adds each new node at the end. This keeps the order of `other`. The function `prepend` adds at the front. A loop that calls `prepend` while it walks from the head would reverse the order.

```
    return *this;
}
```

The function returns the current list. For `a = b = c;`, C++ runs `b = c` first, and then `a = b`.

Trace

The call is `a = b;` from `main.cpp`. Before the call, `a` is `9` with size 1. List `b` is `1->2->3` with size 3. The new nodes are `1'`, `2'` and `3'`.

Step	Code	curr or otherCurr	tail	headNode of a	size
Before	none	none	none	`9`	1
1	`this == &other`: the addresses differ, so the condition is false	none	none	`9`	1
2	delete loop: `temp` saves node `9`, `curr` becomes `NULL`, `delete temp` frees node `9` and its data; the loop ends	`curr` is `NULL`	none	freed	1
3	`headNode = NULL; size = 0;`	none	none	`NULL`	0
4	`tail = NULL;` and `otherCurr` points at node `1` of `b`	`1`	`NULL`	`NULL`	0
5	copy loop, round 1: `newNode` is `1'`; `tail == NULL`, so `headNode = newNode`; `tail = 1'`; `size++`	`2` (after the move)	`1'`	`1'`	1
6	round 2: `newNode` is `2'`; `tail` is not `NULL`, so `tail->next() = newNode`; `tail = 2'`; `size++`	`3`	`2'`	`1'->2'`	2
7	round 3: `newNode` is `3'`; `tail->next() = newNode`; `tail = 3'`; `size++`	`NULL`	`3'`	`1'->2'->3'`	3
8	`otherCurr != NULL` is false; the loop ends; `return *this;`	`NULL`	`3'`	`1'->2'->3'`	3

The nodes of `b` do not change. When `main.cpp` runs `b.prepend(0)`, only `b` gets a new node. List `a` stays `1->2->3`.

The output of `main.cpp` is:

```
a: 9 (size 1)
b: 1->2->3 (size 3)
a after a = b: 1->2->3 (size 3)
b after b.prepend(0): 0->1->2->3 (size 4)
a has not changed: 1->2->3 (size 3)
```

Common mistakes

- **Copy with `prepend`.** A loop that calls `prepend(otherCurr->getData())` while it walks from the head adds every value at the front. The list prints `3->2->1`, and the expected output shows `1->2->3`. The output then differs from the expected output.
- **Delete a node, then read `next()`.** The code `delete curr; curr = curr->next();` reads a deleted node. Valgrind reports "Invalid read of size 8". The solution saves the node in `temp` first.
- **Forget `headNode = NULL;` and `size = 0;`.** The old size stays, and `size++` adds to it. In `main.cpp`, list `a` prints `(size 4)` instead of `(size 3)`. When `other` is empty, `headNode` also keeps the address of a deleted node. In the edge test (test 4, `y = empty`), the list prints a value from a deleted node, and valgrind reports "Invalid read of size 8" and "Invalid free() / delete / delete[]".
- **Link the nodes of `other` instead of new nodes.** The code `tail->next() = otherCurr;` makes the two lists share nodes. Both destructors then delete the same nodes, and valgrind reports invalid frees. The code `new Node<T>(new T(otherCurr->getData()))` makes a new node with its own value.
- **No self-assignment check.** For `x = x;`, the function deletes all the nodes and sets `headNode` to `NULL`. Because `other` is the same list, the copy loop finds no node to copy. Valgrind reports no error, but the list is now empty. Test 1 prints `1. x = x: Empty List (size 0)` instead of `5->6 (size 2)`.

A3-07. Copy constructor: `Team::Team(const Team&)` (100)

File: A_3.cpp

```
#include "Team.h"

Team::Team(const Team& other) : members(NULL), size(0) {
    // A new object has no array yet. members is NULL and size is 0,
    // so operator= has nothing old to delete before it makes the deep copy.
    *this = other;
}
```

Expected output of the given main.cpp

```
Team a:
Team of size 3:
  0: Thandi Mokoena, 29, F, ID: none
  1: empty
  2: Anna Botha, 41, F, ID: none
Team b, a copy of a:
Team of size 3:
  0: Thandi Mokoena, 29, F, ID: none
  1: empty
  2: Anna Botha, 41, F, ID: none
Team a after removing member 0:
Team of size 3:
  0: empty
  1: empty
  2: Anna Botha, 41, F, ID: none
Team b has not changed:
Team of size 3:
  0: Thandi Mokoena, 29, F, ID: none
  1: empty
  2: Anna Botha, 41, F, ID: none
```

Hints

A3-07 hints

Try the question first. Read one hint at a time. The hints go from gentle to specific. They do not give the full answer.

Hint 1

Think about the state of the new object when the copy constructor starts. What values do `members` and `size` hold before your code gives them values?

Hint 2

If you reuse `operator=`, look at what the given `operator=` does first: it deletes every employee in `members` and then the array. In a brand-new object, those pointers were never set.

Hint 3

Give `members` and `size` safe values before you reuse `operator=`: `NULL` and `0`. Deleting `NULL` does nothing, and a loop up to a size of `0` runs zero times. You can set them in the member initialiser list of the constructor.

What the function must do

A **copy constructor** builds a new `Team` from an existing `Team`. It runs for `Team b(a);`, for `Team b = a;`, for `new Team(*a)` and when a `Team` is passed to a function by value. The new team must have the same size as the old team and its own copy of every employee. The two teams must share no `Employee` object.

How the solution works

A constructor starts with an object whose members hold no meaningful values. The members `members` (an `Employee**`, a pointer to an array of `Employee*` pointers) and `size` (an `int`) hold no set values until the constructor sets them. They are uninitialised.

The given `Team::operator=` already makes a deep copy. The solution reuses it.

```
Team::Team(const Team& other) : members(NULL), size(0) {
```

The text after the colon is the **member initialiser list**. It sets `members` to `NULL` and `size` to `0` before the body runs. The parameter `other` is a reference to the existing team. The word `const` means the constructor does not change `other`.

The order in the list follows the order of declaration in `Team.h`: `members` first, then `size`. If the list names `size` first, g++ with `-Wall` reports a `-Wreorder` warning, and `-Werror` makes the warning an error.

The initial values matter because of what `operator=` does first. The given `operator=` deletes every employee in the old array and then deletes the array itself:

```
    for (int i = 0; i < size; i++) {
        delete members[i];
    }
    delete[] members;
```

This code is in `Team.cpp`. In a new object, no array exists yet. If `members` and `size` are not set, the loop uses an uninitialised `size`, and `delete` uses uninitialised addresses. This is undefined behaviour: C++ does not say what happens. With `size` equal to `0`, the loop runs zero times. With `members` equal to `NULL`, `delete[] members` does nothing, because deleting a `NULL` pointer is safe. The old state looks like an empty team, so `operator=` has nothing to free.

```
    *this = other;
}
```

`this` is a pointer to the new object. `*this` is the new object itself. The statement `*this = other;` calls `Team::operator=` for the new object, with `other` as the right-hand side. The `if (this == &other)` check inside `operator=` is false here, because the new object and `other` are different objects. The function then sets `size = other.size;`, creates a new array, and makes a new copy of each employee with the `Employee` copy constructor. A `NULL` slot in `other` stays `NULL` in the copy. Each employee stays in the same slot number.

The solution does not repeat this loop. It relies on the given `operator=`, and the question allows this.

The copy is deep. A **shallow copy** would copy the pointers only, and both teams would then delete the same employees. A deep copy makes new `Employee` objects, so each team deletes its own.

Trace

The call is `new Team(*a)` from `main.cpp`. Before the call, team `a` has size 3. Its slots are `[ Thandi, NULL, Anna ]`, because `a->removeMember(1)` deleted `Sipho`. The new team is `*b`. `Thandi'` and `Anna'` are new copies.

Step	Code	`size` of the new team	`members` of the new team
Start	the constructor is called; no array exists	not set	not set
1	initialiser list: `members(NULL), size(0)`	0	`NULL`
2	`*this = other;` calls `operator=`; `this == &other` is false	0	`NULL`
3	in `operator=`: the loop runs 0 times; `delete[] members` on `NULL` does nothing	0	`NULL`
4	in `operator=`: `size = other.size;`	3	`NULL`
5	in `operator=`: `members = new Employee*[size];`	3	new array of 3 slots, values not set
6	`i = 0`: `other.members[0]` is not `NULL`, so the slot gets `Thandi'`	3	`[ Thandi', ?, ? ]`
7	`i = 1`: `other.members[1]` is `NULL`, so the slot is `NULL`	3	`[ Thandi', NULL, ? ]`
8	`i = 2`: `other.members[2]` is not `NULL`, so the slot gets `Anna'`	3	`[ Thandi', NULL, Anna' ]`
9	`operator=` returns; the constructor ends	3	`[ Thandi', NULL, Anna' ]`

Team `a` still points at `Thandi` and `Anna`. After `a->removeMember(0)`, team `a` deletes `Thandi`, but team `b` still holds `Thandi'`. The output shows that team `b` has not changed.

The output of `main.cpp` is:

```
Team a:
Team of size 3:
  0: Thandi Mokoena, 29, F, ID: none
  1: empty
  2: Anna Botha, 41, F, ID: none
Team b, a copy of a:
Team of size 3:
  0: Thandi Mokoena, 29, F, ID: none
  1: empty
  2: Anna Botha, 41, F, ID: none
Team a after removing member 0:
Team of size 3:
  0: empty
  1: empty
  2: Anna Botha, 41, F, ID: none
Team b has not changed:
Team of size 3:
  0: Thandi Mokoena, 29, F, ID: none
  1: empty
  2: Anna Botha, 41, F, ID: none
```

Common mistakes

- **No `NULL` and `0` before `*this = other;`.** The constructor has an empty initialiser list and only the line `*this = other;`. The members are uninitialised. The loop in `operator=` then uses an uninitialised `size`, and `delete[] members` uses an uninitialised address. This is undefined behaviour. In the test runs, the output still looked correct, but valgrind reported errors. On `main.cpp` it reported "Conditional jump or move depends on uninitialised value(s)". On the edge test it also reported "Invalid free() / delete / delete[]" and "Invalid read of size 8".
- **Shallow copy.** The code `members = other.members; size = other.size;` copies the pointers. Both teams own the same array and employees. The destructors free them twice, and valgrind reports invalid frees.
- **Initialiser list in the wrong order.** The list `size(0), members(NULL)` does not match the order in `Team.h`. With `-Wall -Werror`, the warning `-Wreorder` becomes an error, and the program does not compile. The mark is 0.
- **Write a hand-made copy loop that skips `NULL` slots.** If the loop sets a slot only when `other.members[i]` is not `NULL`, the empty slots of the new array hold no set value. Later code, such as `operator<<` and the destructor, reads these slots and compares them with `NULL`. This is undefined behaviour. In the test runs, the output still looked correct, and valgrind reported "Conditional jump or move depends on uninitialised value(s)" on both tests. Each slot needs a value: `NULL` or a new copy.

B_1 Recursion: solutions

B1-01. Recursion: `Employee::extractIntFromString` (100)

File: B_1.cpp

```
#include "Employee.h"

int Employee::extractIntFromString(string str, int start, int end) {
    // Invalid range: return -1.
    if (start < 0 || start >= end || end > (int)str.length()) {
        return -1;
    }
    // The last character of the range must be a digit.
    if (str[end - 1] < '0' || str[end - 1] > '9') {
        return -1;
    }
    int lastDigit = str[end - 1] - '0';

    // Base case: the range has one character.
    if (end - start == 1) {
        return lastDigit;
    }

    // Recursive case: read the range without its last character.
    int front = extractIntFromString(str, start, end - 1);
    if (front == -1) {
        return -1;
    }
    return front * 10 + lastDigit;
}
```

Expected output of the given main.cpp

```
Year: 97
Month: 2
Day: 14
Gender digits: 123
Bad range: -1
Not a digit: -1
Sipho Dlamini, 34, M, ID: 9105225800087
Caught: ID number must only contain digits
Caught: ID number must have 13 digits
Sipho Dlamini, 34, M, ID: 9105225800087
```

Hints

B1-01 hints

Try the question first. Read one hint at a time. The hints go from gentle to specific. They do not give the full answer.

Hint 1

Find the smallest valid range. An empty range (`start >= end`) is invalid, so the smallest valid range has one character, for example `(2, 3)`. For that range, return that one digit (or `-1` if it is not a digit). This is your base case, so the recursion must stop there and never call `(start, start)`.

Hint 2

Now make the range one character shorter. The value of `"1234"` is the value of `"123"` times 10, plus 4. Which shorter range of `str` holds `"123"`?

Hint 3

A `-1` from the shorter range means that range was not valid. The whole answer must then be `-1` too, so check the result of the recursive call before you multiply it by 10.

What the function must do

The function `Employee::extractIntFromString(str, start, end)` turns a part of a string into a whole number. The function reads the characters from index `start` up to, but not including, index `end`. For `"9702140123081"` with range `6` to `10`, the characters are `"0123"` and the result is `123`.

The function returns `-1` if the range is invalid or if one character in the range is not a digit. The value `0` is a real answer (for example `"00"`), so `-1` marks an error. The function must use recursion, with no loops and no helper functions.

How the solution works

The function reads the range from the right. The value of `"1234"` is the value of `"123"` times 10, plus 4. So the function asks itself for the range that is one character shorter.

```
if (start < 0 || start >= end || end > (int)str.length()) {
    return -1;
}
```

This `if` checks the range. `start < 0` protects against a negative index. `start >= end` rejects an empty range, such as 2 to 2, and a reversed range, such as 3 to 2. `end > (int)str.length()` rejects a range that goes past the string. Without this check, `str[end - 1]` would read outside the string. The cast `(int)str.length()` is needed because `length()` is unsigned. Without the cast, g++ reports `comparison of integer expressions of different signedness`, and `-Wall -Werror` stops the build.

```
if (str[end - 1] < '0' || str[end - 1] > '9') {
    return -1;
}
int lastDigit = str[end - 1] - '0';
```

`str[end - 1]` is the last character of the range. A `char` holds a number code. The codes of `'0'` to `'9'` are next to each other, so this `if` is true when the character is not a digit. The function then returns `-1` at once. Without this check, a letter would become a wrong number. The line `str[end - 1] - '0'` turns a digit character into its value. For example, `'3' - '0'` is `3`.

```
if (end - start == 1) {
    return lastDigit;
}
```

This is the base case. The range has one character, and the function already checked that it is a digit. So the answer is that digit. Every recursive call lowers `end` by 1, so the range gets shorter until it has one character. Without this base case, the calls go on to the empty range `(start, start)`. The range check returns `-1` for that call, and the `-1` passes upward, so every valid range gives `-1`.

```
int front = extractIntFromString(str, start, end - 1);
if (front == -1) {
    return -1;
}
return front * 10 + lastDigit;
```

This is the recursive case. The variable `front` holds the number made from the shorter range. The `if` checks for the error value. If the shorter range holds a letter, `front` is `-1`. The function must pass that `-1` upward. Without this `if`, the line `front * 10 + lastDigit` would turn `-1` into another number. For `"x123"` the result would be wrong instead of `-1`. The order matters: the function adds `lastDigit` after the recursive call returns, because the last digit has the smallest place value.

The function allocates no memory, so nothing is freed.

Trace

The call is `extractIntFromString("9702140123081", 6, 10)`. The string has 13 characters. The characters at index 6 to 9 are `0`, `1`, `2`, `3`.

Call	start, end	Last character	Branch taken	Result
1	6, 10	`str[9]` is `'3'`	recursive case, calls `(6, 9)`	waits
2	6, 9	`str[8]` is `'2'`	recursive case, calls `(6, 8)`	waits

Call	start, end	Last character	Branch taken	Result
3	6, 8	`str[7]` is `'1'`	recursive case, calls `(6, 7)`	waits
4	6, 7	`str[6]` is `'0'`	base case, `end - start == 1`	returns `0`

The calls return in the order 4, 3, 2, 1:

- Call 3: `front` is `0`, so the call returns `0 * 10 + 1 = 1`.
- Call 2: `front` is `1`, so the call returns `1 * 10 + 2 = 12`.
- Call 1: `front` is `12`, so the call returns `12 * 10 + 3 = 123`.

A second call shows the error path: `extractIntFromString("12a45", 0, 5)`.

Call	start, end	Last character	Branch taken	Result
1	0, 5	`str[4]` is `'5'`	recursive case, calls `(0, 4)`	waits
2	0, 4	`str[3]` is `'4'`	recursive case, calls `(0, 3)`	waits
3	0, 3	`str[2]` is `'a'`	not a digit	returns `-1`

Call 2 receives `front == -1` and returns `-1`. Call 1 does the same, so the final answer is `-1`.

The output of `main.cpp` is:

```
Year: 97
Month: 2
Day: 14
Gender digits: 123
Bad range: -1
Not a digit: -1
Sipho Dlamini, 34, M, ID: 9105225800087
Caught: ID number must only contain digits
Caught: ID number must have 13 digits
Sipho Dlamini, 34, M, ID: 9105225800087
```

Common mistakes

- **No check of `front` for `-1`.** The code `return extractIntFromString(str, start, end - 1) * 10 + lastDigit;` gives wrong output when a non-digit is before the last character. The main test `"12a45"` has the letter in the middle. With this mistake, the line `Not a digit:` shows `-55` instead of `-1`.
- **A base case at an empty range.** Suppose the function recurses until `start` equals `end`. The range `if` at the top returns `-1` for that call. The `-1` then travels upward through the check of `front`, so every valid range gives `-1`. In `main.cpp`, the lines `Year:`, `Month:`, `Day:` and `Gender digits:` all show `-1`. The base case must be the range with one character.
- **A wrong digit test.** A test that rejects `'0'`, or that accepts `'/'` or `':'`, changes the output for `"000"`, `"12/4"` and `"12:4"` in the edge tests. The character `'/'` has the code before `'0'`, and `':'` has the code after `'9'`. Rejecting `'0'` makes the edge line `3. all zeros:` show `-1` instead of `0`. Accepting `'/'` makes line `11.` show `1194`, and accepting `':'` makes line `12.` show `1304`.
- **A loop or a helper function.** QUESTION.md states that either one scores 0.
- **Comparing `str.length()` with an `int` without a cast.** g++ reports `comparison of integer expressions of different signedness` (`-Werror=sign-compare`). The build fails, so the mark is 0.

B1-02. Recursion: `Employee::isAllDigits` (100)

File: B_1.cpp

```
#include "Employee.h"

bool Employee::isAllDigits(string str, int index) {
    // Invalid index: return false.
    if (index < 0 || index > (int)str.length()) {
        return false;
    }
    // Base case: no characters are left to check.
    if (index == (int)str.length()) {
        return true;
    }
    // This character is not a digit.
    if (str[index] < '0' || str[index] > '9') {
        return false;
    }
    // Recursive case: check the rest of the string.
    return isAllDigits(str, index + 1);
}
```

Expected output of the given main.cpp

```
9702140123081 from 0: true
97021401230X1 from 0: false
abc123 from 3: true
abc123 from 0: false
Caught: ID number must only contain digits
Thandi Mokoena, 29, F, ID: 9702140123081
```

What the function must do

The function `Employee::isAllDigits(str, index)` checks the end part of a string. The function starts at position `index` and goes to the last character. The function returns `true` if every character in that part is a digit from `'0'` to `'9'`. The function returns `false` as soon as it finds a character that is not a digit.

Characters before `index` are not checked. If `index` is equal to the length of `str`, no characters are left, so the answer is `true`. If `index` is negative or larger than the length, the answer is `false`. The given `setIDNum` calls this function to check an ID number.

How the solution works

The function checks one character, then asks itself about the rest of the string. The parameter `str` is a `string` passed by value, so the function works on its own copy. The function allocates no memory, so nothing is freed.

```
if (index < 0 || index > (int)str.length()) {
    return false;
}
```

This `if` rejects an invalid `index`. `index < 0` protects against a negative position. `index > (int)str.length()` protects against a position past the end. The cast `(int)` is needed because `length()` is unsigned, and `-Wall -Werror` stops a comparison between an unsigned value and an `int`. The test uses `>`, not `>=`. The value `index == str.length()` is valid and means "nothing left to check".

```
if (index == (int)str.length()) {
    return true;
}
```

This is the base case. When `index` equals the length, the function has checked every character and found no error, so the answer is `true`. The same line gives `true` for an empty string, because `isAllDigits("", 0)` has `index == 0 == length`. Without this base case, the function reads `str[index]` when `index` equals the length. In C++98 this is undefined behaviour. With g++ 13 the read gives the character `'\0'`, which is not a digit, so the function returns `false` for every string that reaches the end.

```
if (str[index] < '0' || str[index] > '9') {
    return false;
```

```
}
```

This `if` tests the current character. The character is not a digit when its code is less than the code of `'0'` or more than the code of `'9'`. The function returns `false` at once and makes no further call. This line runs only after the two earlier checks, so `index` is a valid position.

```
return isAllDigits(str, index + 1);
```

This is the recursive case. The current character is a digit, so the answer depends on the rest of the string. The call uses `index + 1`, which moves one position closer to the base case `index == str.length()`. The function returns the value of this call without a change. No statement runs after the call, so the order of statements does not change the result.

Trace

The call is `isAllDigits("abc123", 3)`. The string has 6 characters, and the characters from index 3 are `1`, `2`, `3`.

Call	index	Branch taken	Result
1	3	`str[3]` is `'1'`, a digit, so the recursive case runs	waits for call 2
2	4	`str[4]` is `'2'`, a digit, so the recursive case runs	waits for call 3
3	5	`str[5]` is `'3'`, a digit, so the recursive case runs	waits for call 4
4	6	`index == (int)str.length()`, base case	returns `true`

The calls return in the order 4, 3, 2, 1. Each call returns the `true` that it receives, so the final answer is `true`.

The call `isAllDigits("abc123", 0)` stops at once. At index 0 the character is `'a'`. The character is not a digit, so call 1 returns `false` and makes no recursive call.

The output of `main.cpp` is:

```
9702140123081 from 0: true
97021401230X1 from 0: false
abc123 from 3: true
abc123 from 0: false
Caught: ID number must only contain digits
Thandi Mokoena, 29, F, ID: 9702140123081
```

Common mistakes

- **Using `>=` in the first `if`.** Then `index == str.length()` returns `false`. Every recursion that reaches the end of the string returns `false`. So `isAllDigits("9702140123081", 0)` is `false`, and the edge tests for `""` and for `"123"` from index `3` fail.
- **No base case for `index == (int)str.length()`.** The function then reads `str[index]` when `index` equals the length. In C++98 this is undefined behaviour, so the result cannot be trusted. With g++ 13 the read gave `'\0'`, which is not a digit, so every string of digits gave `false`, the same output as the `>=` mistake. In `main.cpp`, `setIDNum("9702140123081")` then throws an exception that nothing catches, and the program stops with `terminate called after throwing an instance of 'Exception'`.
- **Calling the function without `return`.** The line `isAllDigits(str, index + 1);` computes the answer and discards it. The compiler warns that control reaches the end of a non-void function. Under `-Wall -Werror` this stops the build, and the mark is 0.
- **Checking characters before `index`.** The edge test `isAllDigits("12x45", 3)` expects `true`, because the `'x'` is at index 2. A function that starts at index 0 returns `false`.
- **A loop or a helper function.** QUESTION.md states that either one scores 0.

B1-03. Recursion: `Team::countMembers` (100)

File: B_1.cpp

```
#include "Team.h"

int Team::countMembers(int index) {
    // Base case: an invalid index, or past the last slot. Nothing to count.
    if (index < 0 || index >= size) {
        return 0;
    }
    // Count this slot if it holds an employee.
    int here = 0;
    if (members[index] != NULL) {
        here = 1;
    }
    // Recursive case: add the count of the remaining slots.
    return here + countMembers(index + 1);
}
```

Expected output of the given main.cpp

```
Team of size 5:
  0: Thandi Mokoena, 29, F, ID: none
  1: empty
  2: Anna Botha, 41, F, ID: none
  3: Ravi Naidoo, 38, M, ID: none
  4: empty
Members from index 0: 3
Members from index 2: 2
Members from index 4: 0
After removing member 0, from index 0: 2
```

What the function must do

The function `Team::countMembers(index)` counts the employees in a team, starting at slot `index` and ending at the last slot. A slot holds an employee when it is not `NULL`. A `NULL` slot is empty and adds nothing to the count.

Empty slots can be anywhere in the array, because `removeMember` sets a slot to `NULL`. If `index` is negative or larger than `size`, the answer is `0`. If `index` equals `size`, no slots are left, so the answer is also `0`.

How the solution works

The member `members` is an array of pointers (`Employee**`). A pointer holds the address of an object, and `NULL` means the pointer points to nothing. The function looks at one slot, then asks itself about the slots after it. The function does not allocate or free memory. The team still owns its employees and deletes them in its destructor.

```
if (index < 0 || index >= size) {
    return 0;
}
```

This is the base case. `index < 0` protects against a negative index, because `members[-1]` is outside the array. `index >= size` covers two situations. The value `index == size` means the function has passed the last slot, so no slots are left. A larger value is invalid. Both give `0`. This `if` also handles a team of size 0, because `0 >= 0` is true. Without the base case, every call would make another call, so the recursion could never end. g++ 13 sees this and reports `infinite recursion detected`. Under `-Wall -Werror` the build fails.

```
int here = 0;
if (members[index] != NULL) {
    here = 1;
}
```

The variable `here` holds the count for the current slot. The `if` checks that the slot holds an employee. A slot that is `NULL` keeps `here` at `0`. The check is needed because the function must count only employees, not slots.

```
return here + countMembers(index + 1);
```

This is the recursive case. The call `countMembers(index + 1)` counts the slots after the current slot. Each call raises `index` by 1, so the calls move toward `index == size`, which is the base case. The addition runs after the recursive call returns. Each call adds its own `here` to the count from the later slots.

Trace

In `main.cpp`, the team has size 5. The slots are `Thandi`, empty, `Anna`, `Ravi`, empty. The call is `countMembers(0)`.

Call	index	Slot	Branch taken	here	Result
1	0	Thandi	recursive case	1	waits for call 2
2	1	empty (`NULL`)	recursive case	0	waits for call 3
3	2	Anna	recursive case	1	waits for call 4
4	3	Ravi	recursive case	1	waits for call 5
5	4	empty (`NULL`)	recursive case	0	waits for call 6
6	5	none	base case, `5 >= 5`	not set	returns `0`

The calls return in the order 6, 5, 4, 3, 2, 1:

- Call 5 returns `0 + 0 = 0`.
- Call 4 returns `1 + 0 = 1`.
- Call 3 returns `1 + 1 = 2`.
- Call 2 returns `0 + 2 = 2`.
- Call 1 returns `1 + 2 = 3`.

The output of `main.cpp` is:

```
Team of size 5:
  0: Thandi Mokoena, 29, F, ID: none
  1: empty
  2: Anna Botha, 41, F, ID: none
  3: Ravi Naidoo, 38, M, ID: none
  4: empty
Members from index 0: 3
Members from index 2: 2
Members from index 4: 0
After removing member 0, from index 0: 2
```

Common mistakes

- **Counting a slot without the `NULL` check.** If `here` is always `1`, the function counts slots, not employees. For the team above, the answer is `5` instead of `3`.
- **Using `index > size` in the base case.** When `index` equals `size`, the function then reads `members[size]`, which is outside the array. This is undefined behaviour, so the result cannot be trusted. In the test run the output was still correct, but valgrind reported `Invalid read of size 8` for every call that reached the end, so the valgrind check fails.
- **No check for a negative index.** The function reads `members[-1]`, which is outside the array. This is undefined behaviour. The edge test `countMembers(-1)` expects `0`. In the test run, the line `6. negative index:` showed `3` instead of `0`, and valgrind reported `Invalid read of size 8`.
- **Calling `countMembers(index)` instead of `countMembers(index + 1)`.** The argument never changes, so the recursion never ends. The program compiles, but at run time valgrind reports `Stack overflow` and the program stops with a segmentation fault (`SIGSEGV`). No `Members from index` line is printed.
- **A loop or a helper function.** QUESTION.md states that either one scores 0.

B1-04. Recursion: `Department::totalEmployees` (100)

File: B_1.cpp

```
#include "Department.h"

int Department::totalEmployees(int index) {
    // Base case: an invalid index, or past the last slot. Nothing to add.
    if (index < 0 || index >= size) {
        return 0;
    }
    // Employees in this slot (an empty slot adds 0).
    int here = 0;
    if (teams[index] != NULL) {
        here = teams[index]->getMemberCount();
    }
    // Recursive case: add the employees of the remaining slots.
    return here + totalEmployees(index + 1);
}
```

Expected output of the given main.cpp

```
Department of size 4:
Slot 0: Team of size 3:
  0: Thandi Mokoena, 29, F, ID: none
  1: Sipho Dlamini, 34, M, ID: none
  2: empty
Slot 1: empty
Slot 2: Team of size 2:
  0: Lebo Nkosi, 25, F, ID: none
  1: empty
Slot 3: empty
Employees from team 0: 3
Employees from team 2: 1
Employees from team 3: 0
After removing a member of team 0: 2
```

What the function must do

The function `Department::totalEmployees(index)` adds up the employees in all teams from slot `index` to the last slot. Each slot of `teams` holds a `Team*` or `NULL`. A `NULL` slot has no team and adds `0`. A team with no employees also adds `0`.

If `index` equals `size`, no slots are left, so the answer is `0`. If `index` is negative or larger than `size`, the answer is also `0`. The class `Team` has a given function `getMemberCount()` that returns the number of employees in one team.

How the solution works

The array `teams` is an array of pointers (`Team**`). The function handles one slot, then asks itself about the slots after it. The function does not allocate or free memory. The department keeps ownership of its teams.

```
if (index < 0 || index >= size) {
    return 0;
}
```

This is the base case. `index < 0` protects against a negative index, because `teams[-1]` is outside the array. `index >= size` covers the value `size`, which means the function has passed the last slot, and any larger invalid value. Both give `0`. The same line handles a department of size 0. Without a base case, every call would make another call, so the recursion could never end. g++ 13 sees this and reports `infinite recursion detected`. Under `-Wall -Werror` the build fails.

```
int here = 0;
if (teams[index] != NULL) {
    here = teams[index]->getMemberCount();
}
```

The variable `here` holds the employee count of the current slot. The `if` checks that the slot holds a team. The arrow `->` calls a function through a pointer, so `teams[index]->getMemberCount()` calls the function on the team that the pointer points to. If the slot is `NULL`, the pointer points to nothing, and a call through it is undefined behaviour. The check keeps `here` at `0` for an empty slot.

```
return here + totalEmployees(index + 1);
```

This is the recursive case. The call `totalEmployees(index + 1)` returns the employee count for the slots after the current slot. Each call raises `index` by 1, so the calls move toward `index == size`, which is the base case. The addition runs after the recursive call returns. Each call adds its own `here` to the total from the later slots.

Trace

In `main.cpp`, the department has size 4. Slot 0 holds a team with 2 employees. Slot 1 is `NULL`, because the program deletes that team and sets the slot to `NULL`. Slot 2 holds a team with 1 employee. Slot 3 was never filled, so it is `NULL`. The call is `totalEmployees(0)`.

Call	index	Slot	Branch taken	here	Result
1	0	team with 2 employees	recursive case	2	waits for call 2
2	1	`NULL`	recursive case	0	waits for call 3
3	2	team with 1 employee	recursive case	1	waits for call 4
4	3	`NULL`	recursive case	0	waits for call 5
5	4	none	base case, `4 >= 4`	not set	returns `0`

The calls return in the order 5, 4, 3, 2, 1:

- Call 4 returns `0 + 0 = 0`.
- Call 3 returns `1 + 0 = 1`.
- Call 2 returns `0 + 1 = 1`.
- Call 1 returns `2 + 1 = 3`.

The output of `main.cpp` is:

```
Department of size 4:
Slot 0: Team of size 3:
  0: Thandi Mokoena, 29, F, ID: none
  1: Sipho Dlamini, 34, M, ID: none
  2: empty
Slot 1: empty
Slot 2: Team of size 2:
  0: Lebo Nkosi, 25, F, ID: none
  1: empty
Slot 3: empty
Employees from team 0: 3
Employees from team 2: 1
Employees from team 3: 0
After removing a member of team 0: 2
```

Common mistakes

- **No `NULL` check before `->`.** The first `NULL` slot makes the program call `getMemberCount()` through a null pointer. This is undefined behaviour. In the test run, valgrind reported `Invalid read of size 4`, and the program stopped with a segmentation fault (`SIGSEGV`) at the first call of `totalEmployees`. The main test and the edge test with an empty department both contain `NULL` slots.
- **Using `getSize()` of the team instead of `getMemberCount()`.** The function `getSize()` returns the number of slots, not the number of employees. The team in slot 0 has 3 slots but 2 employees, so the total is wrong. In `main.cpp`, the line `Employees from team 0:` shows `5` instead of `3`.
- **Using `index > size` in the base case.** When `index` equals `size`, the function then reads `teams[size]`, which is outside the array. This is undefined behaviour, so the result cannot be trusted. In the test run the output was still correct, but valgrind reported `Invalid read of size 8`, so the valgrind check fails.
- **Calling `totalEmployees(index)` instead of `totalEmployees(index + 1)`.** The argument never changes, so the recursion never ends. The program compiles, but at run time valgrind reports `Stack overflow` and the program stops with a segmentation fault (`SIGSEGV`).
- **A loop or a helper function.** QUESTION.md states that either one scores 0. The call to the given `Team::getMemberCount()` is allowed.

B1-05. Recursion: `List<T>::countNodes` (100)

File: B_1.cpp

```
#ifndef B_1_CPP
#define B_1_CPP

#include "List.h"

template <class T>
int List<T>::countNodes(Node<T>* curr) {
    // Base case: past the last node, so there is nothing to count.
    if (curr == NULL) {
        return 0;
    }
    // Recursive case: this node, plus the nodes after it.
    return 1 + countNodes(curr->next());
}

#endif
```

Expected output of the given main.cpp

```
10->20->30->40
Nodes from the head: 4
Nodes from the second node: 3
Nodes in an empty list: 0
```

What the function must do

The function `List<T>::countNodes(curr)` counts the nodes of a linked list from the node `curr` to the end of the list. The count includes `curr` itself. The call `list.countNodes(list.head())` counts the whole list. The call `list.countNodes(list.head()->next())` counts every node except the first.

If `curr` is `NULL`, there are no nodes, so the answer is `0`. This also covers an empty list, because `head()` returns `NULL` for an empty list. The function must not use `size` or `getSize()`, because those count the whole list.

How the solution works

```
template <class T>
int List<T>::countNodes(Node<T>* curr) {
```

The line `template <class T>` makes `T` a placeholder for the type of the stored values, such as `int` or `string`. The name `List<T>::countNodes` says that the function belongs to the class `List<T>`. The parameter `curr` is a pointer to a `Node<T>`. A pointer holds the address of a node, and `NULL` means it points to nothing. The function only reads the list. The function allocates no memory and frees none.

The whole definition sits between `#define B_1_CPP` and `#endif`. The header `List.h` includes `B_1.cpp` at the bottom, so the template code is visible where the template is used.

```
if (curr == NULL) {
    return 0;
}
```

This is the base case. A linked list ends with a node whose `next()` is `NULL`. When `curr` is `NULL`, the function has moved past the last node, so no nodes are left to count. This `if` also protects the empty list, where the first call receives `NULL`. Without the base case, every call would make another call, so the recursion could never end. g++ 13 sees this and reports `infinite recursion detected`. Under `-Wall -Werror` the build fails.

```
return 1 + countNodes(curr->next());
```

This is the recursive case. The expression `curr->next()` uses the arrow `->` to call `next()` on the node that `curr` points to. The call gives the pointer to the following node. The call `countNodes(curr->next())` counts all nodes after the current one, and the `1` counts the current node. Each call moves one node forward, so the calls move toward the end of the list, where `curr` is `NULL`. The addition runs after the recursive call returns.

Trace

In `main.cpp`, the list is `10->20->30->40`. The call is `countNodes(list.head())`.

Call	curr points to	Branch taken	Result
1	node 10	recursive case	waits for call 2
2	node 20	recursive case	waits for call 3
3	node 30	recursive case	waits for call 4
4	node 40	recursive case	waits for call 5
5	`NULL`	base case	returns `0`

The calls return in the order 5, 4, 3, 2, 1:

- Call 4 returns `1 + 0 = 1`.
- Call 3 returns `1 + 1 = 2`.
- Call 2 returns `1 + 2 = 3`.
- Call 1 returns `1 + 3 = 4`.

The call `countNodes(list.head()->next())` starts at node 20. This call makes 4 calls, with the last call on `NULL`, and returns `3`. The call `empty.countNodes(empty.head())` starts at `NULL`, so call 1 is the base case and returns `0`.

The output of `main.cpp` is:

```
10->20->30->40
Nodes from the head: 4
Nodes from the second node: 3
Nodes in an empty list: 0
```

Common mistakes

- **Using `size` or `getSize()`.** QUESTION.md forbids both, because they give the length of the whole list. A call that starts at the second node would still give `4` for this list, not `3`.
- **Stopping at the last node without a check of `curr`.** A base case such as `curr->next() == NULL` reads `next()` through `curr`. For an empty list, `curr` is `NULL`, so this reads through a null pointer. This is undefined behaviour. In the test run, valgrind reported `Invalid read of size 8`, and the program stopped with a segmentation fault (`SIGSEGV`) at the empty list in `main.cpp`. The edge test `countNodes(NULL)` covers this too.
- **Leaving out the `1`.** The line `return countNodes(curr->next());` adds nothing for each node, so the function returns `0` for every list.
- **Calling `countNodes(curr)` instead of `countNodes(curr->next())`.** The argument never changes, so the recursion never ends. The program compiles, but at run time valgrind reports `Stack overflow` and the program stops with a segmentation fault (`SIGSEGV`).
- **Placing code after `#endif`.** QUESTION.md states that this gives a redefinition error. g++ reports `redefinition of 'int List<T>::countNodes(Node<T>*)'`.
- **A loop or a helper function.** QUESTION.md states that either one scores 0.

B1-06. Recursion: `List<T>::printReverse` (100)

File: B_1.cpp

```
#ifndef B_1_CPP
#define B_1_CPP

#include "List.h"

template <class T>
void List<T>::printReverse(Node<T>* curr) {
    // Base case: past the last node, so there is nothing to print.
    if (curr == NULL) {
        return;
    }
    // Recursive case: print the nodes after this one first, then this node.
    printReverse(curr->next());
    cout << curr->getData() << endl;
}

#endif
```

Expected output of the given main.cpp

```
1->2->3
Reverse:
3
2
1
Reverse from the second node:
3
2
Reverse of an empty list:
(end)
```

What the function must do

The function `List<T>::printReverse(curr)` prints the values of a linked list in reverse order. The function starts with the last node and ends with the node `curr`. Each value goes on its own line, with `cout` and `endl`.

For the list `1->2->3`, the call `printReverse(head())` prints `3`, `2` and `1`. A linked list has only forward links, so the function cannot walk backward. If `curr` is `NULL`, the function prints nothing. The function must not change the list.

How the solution works

```
template <class T>
void List<T>::printReverse(Node<T>* curr) {
```

The line `template <class T>` makes `T` a placeholder for the type of the stored values. The parameter `curr` is a pointer to a `Node<T>`. A pointer holds the address of a node, and `NULL` means it points to nothing. The function allocates no memory and frees none. The code sits between `#define B_1_CPP` and `#endif`, because `List.h` includes `B_1.cpp`.

```
if (curr == NULL) {
    return;
}
```

This is the base case. The value `NULL` appears after the last node, and it also appears as the head of an empty list. In both situations there is nothing to print, so the function returns. The type `void` means the function returns no value, so `return;` has no value. Without this base case, every call would make another call, so the recursion could never end. g++ 13 sees this and reports `infinite recursion detected`. Under `-Wall -Werror` the build fails.

```
printReverse(curr->next());
cout << curr->getData() << endl;
```

This is the recursive case. The expression `curr->next()` uses the arrow `->` to call `next()` on the node that `curr` points to. The call gives the pointer to the following node. Each call moves one node forward, so the calls move toward `NULL`, which is the base

case.

The order of the two lines is what reverses the output. The function first calls itself for the rest of the list. That call prints all later nodes, from the last one backward. Only after that call returns does the function print its own value with `cout << curr->getData() << endl;`. The call `getData()` returns the value stored in the node. The function does not change any pointer, so the list stays the same.

Trace

In `main.cpp`, the list is `1->2->3`. The call is `printReverse(list.head())`.

Call	curr points to	Branch taken	Prints
1	node 1	recursive case, calls call 2 first	nothing yet
2	node 2	recursive case, calls call 3 first	nothing yet
3	node 3	recursive case, calls call 4 first	nothing yet
4	`NULL`	base case, returns	nothing

The calls return in the order 4, 3, 2, 1. After each return, the caller runs its print line:

- Call 3 returns from call 4, then prints `3`.
- Call 2 returns from call 3, then prints `2`.
- Call 1 returns from call 2, then prints `1`.

The call `printReverse(list.head()->next())` starts at node 2. This call prints `3` and then `2`. The call on the empty list is a base case at once and prints nothing.

The output of `main.cpp` is:

```
1->2->3
Reverse:
3
2
1
Reverse from the second node:
3
2
Reverse of an empty list:
(end)
```

Common mistakes

- **Printing before the recursive call.** If `cout << curr->getData() << endl;` comes first, the values appear in forward order: `1`, `2`, `3`. The output is wrong, and the edge tests with a list of strings and a list of 10 numbers show this error.
- **No check for `NULL`.** Without the `if`, every call makes another call, so the recursion can never end. g++ 13 reports `infinite recursion detected`, and under `-Wall -Werror` the build fails. The mark is 0.
- **Printing without `endl`.** The values appear on one line, for example `321`. The expected output has one value on each line.
- **Calling `printReverse(curr)` instead of `printReverse(curr->next())`.** The argument never changes, so the recursion never ends. The program compiles, but at run time valgrind reports `Stack overflow` and the program stops with a segmentation fault (`SIGSEGV`). Nothing after `Reverse:` is printed.
- **Changing the list.** QUESTION.md says not to change the list. The last edge test prints the list again after the call and expects the same list and the same size.
- **A loop or a helper function.** QUESTION.md states that either one scores 0.

B1-07. Recursion: `List<T>::contains` (100)

File: B_1.cpp

```
#ifndef B_1_CPP
#define B_1_CPP

#include "List.h"

template <class T>
bool List<T>::contains(Node<T>* curr, T value) {
    // Base case: past the last node, so the value is not in the list.
    if (curr == NULL) {
        return false;
    }
    // Base case: this node holds the value.
    if (curr->getData() == value) {
        return true;
    }
    // Recursive case: search the nodes after this one.
    return contains(curr->next(), value);
}

#endif
```

Expected output of the given main.cpp

```
5->10->15
Contains 10: true
Contains 7: false
Contains 15 (the last node): true
Contains 5, from the second node: false
Empty list contains 5: false
```

What the function must do

The function `List<T>::contains(curr, value)` searches a linked list for a value. The function looks at the node `curr` and at every node after it. The function returns `true` if one of these nodes holds a value equal to `value`. Otherwise the function returns `false`.

Nodes before `curr` are not searched. If `curr` is `NULL`, the answer is `false`. This also covers an empty list. The function compares values with `==`, so strings must match exactly: `"hr"` is not equal to `"HR"`.

How the solution works

```
template <class T>
bool List<T>::contains(Node<T>* curr, T value) {
```

The line `template <class T>` makes `T` a placeholder for the type of the stored values. The parameter `curr` is a pointer to a `Node<T>`. A pointer holds the address of a node, and `NULL` means it points to nothing. The parameter `value` is a copy of the value to find. The function allocates no memory and frees none. The code sits between `#define B_1_CPP` and `#endif`, because `List.h` includes `B_1.cpp`.

```
if (curr == NULL) {
    return false;
}
```

This is the first base case. The value `NULL` appears after the last node, and it also appears as the head of an empty list. The function has then looked at every node without a match, so the value is not in the list. This `if` must come before any use of `curr`. Without it, the next line would call `getData()` through a `NULL` pointer when the search reaches the end of the list. This is undefined behaviour.

```
if (curr->getData() == value) {
    return true;
}
```

This is the second base case. The arrow `->` calls `getData()` on the node that `curr` points to. The call returns the stored value. If that value equals `value`, the search is finished, so the function returns `true`. The function makes no further call. The search stops at the first match, and the later nodes are not visited.

```
return contains(curr->next(), value);
```

This is the recursive case. The current node does not hold the value, so the answer depends on the rest of the list. The call `curr->next()` gives the pointer to the following node. Each call moves one node forward, so the calls move toward `NULL`, which is the first base case. The function returns the answer of this call without a change.

Trace

In `main.cpp`, the list is `5->10->15`. The first call is `contains(list.head(), 10)`.

Call	curr points to	Value in node	Branch taken	Result
1	node 5	5, not equal to 10	recursive case	waits for call 2
2	node 10	10, equal to 10	second base case	returns `true`

Call 1 returns the `true` that it receives, so the answer is `true`.

The second call is `contains(list.head(), 7)`. No node holds 7.

Call	curr points to	Value in node	Branch taken	Result
1	node 5	5, not equal to 7	recursive case	waits for call 2
2	node 10	10, not equal to 7	recursive case	waits for call 3
3	node 15	15, not equal to 7	recursive case	waits for call 4
4	`NULL`	none	first base case	returns `false`

The calls return in the order 4, 3, 2, 1. Each call returns the `false` that it receives, so the answer is `false`.

The output of `main.cpp` is:

```
5->10->15
Contains 10: true
Contains 7: false
Contains 15 (the last node): true
Contains 5, from the second node: false
Empty list contains 5: false
```

Common mistakes

- **Testing `getData()` before `curr == NULL`.** For an empty list or for a missing value, the function calls `getData()` through a null pointer. This is undefined behaviour. In the test run, valgrind reported `Invalid read of size 8`, and the program stopped with a segmentation fault (`SIGSEGV`) at the main test `Contains 7`, because 7 is not in the list. The main test also uses an empty list, and the edge test calls `contains(NULL, 7)`.
- **Returning `false` when the first node does not match.** Code such as `else { return false; }` checks only the first node. The main test `Contains 15 (the last node)` then prints `false`.
- **Returning `true` in the `NULL` base case.** Every search then ends with `true`, so `Contains 7` prints `true`.
- **Writing `=` instead of `==`.** The function `getData()` returns a `const` reference, so an assignment to the result is a compile error: `assignment of read-only location`. The mark is 0.
- **Calling `contains(curr->next(), value);` without `return`.** The compiler warns that control reaches the end of a non-void function. Under `-Wall -Werror` this stops the build.
- **A loop or a helper function.** QUESTION.md states that either one scores 0.

B1-08. Recursion: `Company::totalCost` (100)

File: B_1.cpp

```
#include "Company.h"

float Company::totalCost(Node<Department*>* curr) {
    // Base case: past the last node, so there is nothing to add.
    if (curr == NULL) {
        return 0;
    }
    // Cost of the department in this node (a NULL department adds 0).
    float here = 0;
    if (curr->getData() != NULL) {
        here = curr->getData()->costToCompany();
    }
    // Recursive case: add the cost of the departments after this node.
    return here + totalCost(curr->next());
}
```

Expected output of the given main.cpp

```
Company with 3 departments
HR of size 1:
Slot 0: Team of size 1:
  0: Lebo Nkosi, 25, F, ID: none
Sales of size 2:
Slot 0: Team of size 3:
  0: Anna Botha, 41, F, ID: none
  1: empty
  2: empty
Slot 1: Team of size 1:
  0: Ravi Naidoo, 38, M, ID: none
Development of size 1:
Slot 0: Team of size 2:
  0: Thandi Mokoena, 29, F, ID: none
  1: Sipho Dlamini, 34, M, ID: none
Total cost: R92000
Cost from the second department: R80000
Cost from the third department: R50000
```

What the function must do

The function `Company::totalCost(curr)` adds up the cost of departments in a linked list. The list is `List<Department*>`, so each node stores a `Department*`. The function starts at the node `curr` and adds the value of `costToCompany()` for each department until the end of the list.

A node can store `NULL` instead of a department. That node adds `0`. If `curr` is `NULL`, there are no nodes, so the answer is `0`. This also covers an empty company. The given `Company::costToCompany()` calls `totalCost(departments.head())`.

How the solution works

The function reads the list and the departments. The function allocates no memory and frees none. The `Company` destructor already deletes the departments, so `totalCost` must not delete them.

```
if (curr == NULL) {
    return 0;
}
```

This is the base case. The value `NULL` appears after the last node, and it also appears as the head of an empty list. In both situations there are no departments left to add, so the answer is `0`. Without this `if`, every call would make another call, so the recursion could never end. g++ 13 sees this and reports `infinite recursion detected`. Under `-Wall -Werror` the build fails.

```
float here = 0;
if (curr->getData() != NULL) {
    here = curr->getData()->costToCompany();
}
```

The variable `here` holds the cost of the department in the current node. The arrow `->` calls a function through a pointer. The call `curr->getData()` returns the stored `Department*`. The `if` checks that this pointer is not `NULL`. A node can store `NULL`, and calling `costToCompany()` through a null pointer is undefined behaviour. For a `NULL` department, `here` stays `0`.

The function `costToCompany()` is `virtual` in `Department`. When the pointer points to a `Sales`, `Marketing`, `HR` or `Development` object, the call runs the version of that class. Each version multiplies the number of employees by a rate. The call is legal here because `costToCompany()` is `public` in `Department`, and the pointer has the type `Department*`.

```
return here + totalCost(curr->next());
```

This is the recursive case. The call `curr->next()` returns the pointer to the following node. The call `totalCost(curr->next())` adds the costs of all later departments. Each call moves one node forward, so the calls move toward `NULL`, which is the base case. The addition runs after the recursive call returns.

Trace

In `main.cpp`, the program adds `dev`, then `sales`, then `hr` with `addDepartment`. The function `addDepartment` adds at the front of the list, so the list order is `HR`, `Sales`, `Development`.

- `HR` has 1 employee, so its cost is `1 * 12000 = 12000`.
- `Sales` has 2 employees (one in each of its two teams), so its cost is `2 * 15000 = 30000`.
- `Development` has 2 employees, so its cost is `2 * 25000 = 50000`.

The call is `yyra.costToCompany()`, which calls `totalCost(departments.head())`.

Call	curr points to	Branch taken	here	Result
1	node with `HR`	recursive case	12000	waits for call 2
2	node with `Sales`	recursive case	30000	waits for call 3
3	node with `Development`	recursive case	50000	waits for call 4
4	`NULL`	base case	not set	returns `0`

The calls return in the order 4, 3, 2, 1:

- Call 3 returns `50000 + 0 = 50000`.
- Call 2 returns `30000 + 50000 = 80000`.
- Call 1 returns `12000 + 80000 = 92000`.

The call `totalCost(first->next())` starts at the `Sales` node and returns `80000`. The call `totalCost(first->next()->next())` starts at the `Development` node and returns `50000`.

The output of `main.cpp` is:

```
Company with 3 departments
HR of size 1:
Slot 0: Team of size 1:
  0: Lebo Nkosi, 25, F, ID: none
Sales of size 2:
Slot 0: Team of size 3:
  0: Anna Botha, 41, F, ID: none
  1: empty
  2: empty
Slot 1: Team of size 1:
  0: Ravi Naidoo, 38, M, ID: none
Development of size 1:
Slot 0: Team of size 2:
  0: Thandi Mokoena, 29, F, ID: none
  1: Sipho Dlamini, 34, M, ID: none
Total cost: R92000
Cost from the second department: R80000
Cost from the third department: R50000
```

Common mistakes

- **No check of `curr->getData()` for `NULL`.** The main test still passes, because it has no `NULL` department. The edge test adds `NULL` with `addDepartment(NULL)`. The call `costToCompany()` through a null pointer is undefined behaviour. In the test

run, valgrind reported `Invalid read of size 8`, and the program stopped with a segmentation fault (`SIGSEGV`) before the line `2. total:`.

- **No check of `curr` for `NULL`.** Without the base case, every call makes another call, so the recursion can never end. g++ 13 reports `infinite recursion detected`, and under `-Wall -Werror` the build fails. The mark is 0.
- **Calling `countEmployees()` instead of `costToCompany()`.** The function `countEmployees()` is `protected` in `Department`, so `Company` cannot call it. This is a compile error: `'int Department::countEmployees()' is protected within this context`. This function would also leave out the rate of each department.
- **Deleting a department inside `totalCost`.** The `Company` destructor deletes the same department again. A second delete of one object is undefined behaviour. In the test run, the first `Total cost` line was still `R92000`, but the next two lines showed `R0`, and valgrind reported `Invalid read`, `Invalid write` and `Invalid free() / delete / delete[] / realloc()` errors.
- **A loop or a helper function.** QUESTION.md states that either one scores 0. The call to the given `costToCompany()` of a department is allowed.

B1-09. Recursion: `Employee::checksum` (100)

File: B_1.cpp

```
#include "Employee.h"

int Employee::checksum(string id, int index) {
    // Base case: an invalid index, or no digits left. Nothing to add.
    if (index < 0 || index >= (int)id.length()) {
        return 0;
    }
    int digit = id[index] - '0';

    // Double this digit if the number of characters after it is odd.
    // A doubled value of more than 9 becomes that value minus 9.
    if (((int)id.length() - 1 - index) % 2 == 1) {
        digit = digit * 2;
        if (digit > 9) {
            digit = digit - 9;
        }
    }

    // Recursive case: add the sum of the digits after this one.
    return digit + checksum(id, index + 1);
}
```

Expected output of the given main.cpp

```
Luhn sum of 9702140123086: 50
Valid: yes
Luhn sum of 9702140123081: 45
Valid: no
Sum from index 6 of 9702140123086: 23
Caught: ID number has a wrong check digit
Thandi Mokoena, 29, F, ID: 9702140123086
```

What the function must do

The function `Employee::checksum(id, index)` returns the Luhn sum of the digits of `id`, from position `index` to the end. The Luhn rule treats each digit by its distance from the end of the whole string. A digit is doubled when the number of characters after it is odd. If the doubled value is more than 9, the function subtracts 9. All other digits keep their value.

The call `checksum(id, 0)` gives the sum for the whole ID number. An ID number is valid when this sum is a multiple of 10. If `index` equals the length of `id`, no digits are left, so the answer is `0`. If `index` is negative or larger than the length, the answer is also `0`. The function may assume that `id` holds only digits.

How the solution works

The function handles one digit, then asks itself about the digits after it. The parameter `id` is a `string` passed by value, so the function works on its own copy. The function allocates no memory and frees none.

```
if (index < 0 || index >= (int)id.length()) {
    return 0;
}
```

This is the base case. `index < 0` protects against a negative position. `index >= (int)id.length()` covers the position after the last digit, where no digits are left, and any larger invalid position. Both give `0`. The cast `(int)` is needed because `length()` is unsigned, and `-Wall -Werror` stops a comparison between an unsigned value and an `int`. Without this base case, every call would make another call, so the recursion could never end. g++ 13 sees this and reports `infinite recursion detected`. Under `-Wall -Werror` the build fails.

```
int digit = id[index] - '0';
```

The expression `id[index]` is a character. The codes of `'0'` to `'9'` are next to each other, so subtracting `'0'` turns a digit character into its number. For example, `'7' - '0'` is `7`.

```
if (((int)id.length() - 1 - index) % 2 == 1) {
    digit = digit * 2;
    if (digit > 9) {
        digit = digit - 9;
    }
}
```

The value `(int)id.length() - 1 - index` is the number of characters after the current digit, counted to the end of the whole string. The base case guarantees that `index` is less than the length, so this number is never negative. The operator `%` gives the remainder after division, so `% 2 == 1` is true when the number is odd. In that case the function doubles the digit. The inner `if` applies the "more than 9" rule: a doubled value such as `14` becomes `14 - 9 = 5`. A digit in an even position keeps its value. The count must use the length of the whole string, not the part that remains, so the recursive calls give the same answer as the first call.

```
return digit + checksum(id, index + 1);
```

This is the recursive case. The call `checksum(id, index + 1)` returns the sum for the digits after the current one. Each call raises `index` by 1, so the calls move toward `index == id.length()`, which is the base case. The addition runs after the recursive call returns. The function returns the sum itself. The caller applies `% 10` to test the sum.

Trace

The call is `checksum("9702140123086", 6)` from `main.cpp`. The string has 13 characters, so the number of characters after index `i` is `12 - i`.

Call	index	Digit	Characters after	Rule	Value of `digit`
1	6	0	6, even	keep	0
2	7	1	5, odd	double: 2	2
3	8	2	4, even	keep	2
4	9	3	3, odd	double: 6	6
5	10	0	2, even	keep	0
6	11	8	1, odd	double: 16, then 16 - 9	7
7	12	6	0, even	keep	6
8	13	none	not tested	base case, `13 >= 13`	returns `0`

The calls return in the order 8, 7, 6, 5, 4, 3, 2, 1:

- Call 7 returns `6 + 0 = 6`.
- Call 6 returns `7 + 6 = 13`.
- Call 5 returns `0 + 13 = 13`.
- Call 4 returns `6 + 13 = 19`.
- Call 3 returns `2 + 19 = 21`.
- Call 2 returns `2 + 21 = 23`.
- Call 1 returns `0 + 23 = 23`.

The output of `main.cpp` is:

```
Luhn sum of 9702140123086: 50
Valid: yes
Luhn sum of 9702140123081: 45
Valid: no
Sum from index 6 of 9702140123086: 23
Caught: ID number has a wrong check digit
Thandi Mokoena, 29, F, ID: 9702140123086
```

Common mistakes

- **Counting the doubling position from the front.** A test such as `index % 2 == 0` gives the wrong digits to double when the string has an odd length. The edge test `"12345"` expects `21`, but this version gives `15`. The rule must use the number of characters after the digit.

- **Counting from `index` instead of from the end of `id`.** QUESTION.md states that `checksum("7918", 2)` is `10`, because the `1` is still doubled. A function that restarts the count at each call gives a different sum.
- **Leaving out the subtraction of 9.** For `"59"`, the digit `5` doubles to `10`. The correct value is `1`, so the sum is `10`. Without the subtraction, the sum is `19`. For `"99"`, the correct sum is `18`, and without the subtraction it is `27`. Both are edge tests.
- **Returning the sum modulo 10.** The function must return the sum. The call `checksum("79927398713", 0)` must return `70`, not `0`.
- **Using `index > (int)id.length()` in the base case.** When `index` equals the length, the function reads `id[index]` past the last character. In C++98 this is undefined behaviour, so the result cannot be trusted. With g++ 13 the read gave `'\0'`, so `digit` was `0 - 48 = -48`, and every sum was wrong. For example, the edge line `4. index equal to the length:` showed `-48` instead of `0`.
- **A loop or a helper function.** QUESTION.md states that either one scores 0.

B1-10. Recursion: `Team::oldest` (100)

File: B_1.cpp

```
#include "Team.h"

int Team::oldest(int index) {
    // Base case: an invalid index, or past the last slot. No employees, so 0.
    if (index < 0 || index >= size) {
        return 0;
    }
    // Recursive case: the oldest age in the slots after this one.
    int restOldest = oldest(index + 1);

    // Compare this slot (if it holds an employee) with the rest.
    if (members[index] != NULL && members[index]->getAge() > restOldest) {
        return members[index]->getAge();
    }
    return restOldest;
}
```

Expected output of the given main.cpp

```
Team of size 5:
  0: Thandi Mokoena, 29, F, ID: none
  1: empty
  2: Anna Botha, 41, F, ID: none
  3: Ravi Naidoo, 38, M, ID: none
  4: empty
Oldest from index 0: 41
Oldest from index 3: 38
Oldest from index 4: 0
After removing member 2, from index 0: 38
```

What the function must do

The function `Team::oldest(index)` returns the greatest age among the employees in the slots from `index` to the last slot. Each slot of `members` holds an `Employee*` or `NULL`. The function skips `NULL` slots.

If no employee is in those slots, the answer is `0`. This is safe because all ages are more than `0`. If `index` equals `size`, no slots are left, so the answer is `0`. If `index` is negative or larger than `size`, the answer is also `0`. If two employees have the same age, the answer is that age.

How the solution works

The array `members` is an array of pointers (`Employee**`). A pointer holds the address of an object, and `NULL` means it points to nothing. The function first asks about the later slots, then compares the current slot with that answer. The function allocates no memory and frees none.

```
if (index < 0 || index >= size) {
    return 0;
}
```

This is the base case. `index < 0` protects against a negative index, because `members[-1]` is outside the array. `index >= size` covers the value `size`, which means the function has passed the last slot, and any larger invalid value. Both give `0`, the "no employee" answer. The same line handles a team of size 0. Without a base case, every call would make another call, so the recursion could never end. g++ 13 sees this and reports `infinite recursion detected`. Under `-Wall -Werror` the build fails.

```
int restOldest = oldest(index + 1);
```

This is the recursive call. The variable `restOldest` holds the greatest age in the slots after the current slot. Each call raises `index` by 1, so the calls move toward `index == size`, which is the base case. The call comes before the comparison. The function needs `restOldest` to compare with the current slot.

```
if (members[index] != NULL && members[index]->getAge() > restOldest) {
    return members[index]->getAge();
```

```
}
return restOldest;
```

This `if` has two conditions joined by `&&`. The operator `&&` checks the left condition first. If it is false, the right condition is not evaluated. The left condition `members[index] != NULL` checks that the slot holds an employee. The right condition uses the arrow `->`, which calls `getAge()` on the employee that the pointer points to. The `NULL` test must come first. Otherwise, the function would call `getAge()` through a null pointer, which is undefined behaviour.

The right condition `members[index]->getAge() > restOldest` checks that the current employee is older than every employee after this slot. If both conditions are true, the function returns the age of the current employee. In every other case, the function returns `restOldest`. This covers an empty slot, a younger employee and an employee of the same age. For the same age, the two values are equal, so the answer is correct.

Trace

In `main.cpp`, the team has size 5. The slots are `Thandi` (29), empty, `Anna` (41), `Ravi` (38), empty. The call is `oldest(0)`.

Call	index	Slot	Branch taken	`restOldest`	Result
1	0	Thandi, 29	recursive case	set when call 2 returns	waits for call 2
2	1	empty (`NULL`)	recursive case	set when call 3 returns	waits for call 3
3	2	Anna, 41	recursive case	set when call 4 returns	waits for call 4
4	3	Ravi, 38	recursive case	set when call 5 returns	waits for call 5
5	4	empty (`NULL`)	recursive case	set when call 6 returns	waits for call 6
6	5	none	base case, `5 >= 5`	not set	returns `0`

The calls return in the order 6, 5, 4, 3, 2, 1:

- Call 5: `restOldest` is `0`. The slot is `NULL`, so the function returns `0`.
- Call 4: `restOldest` is `0`. Since `38 > 0`, the function returns `38`.
- Call 3: `restOldest` is `38`. Since `41 > 38`, the function returns `41`.
- Call 2: `restOldest` is `41`. The slot is `NULL`, so the function returns `41`.
- Call 1: `restOldest` is `41`. Since `29 > 41` is false, the function returns `41`.

The output of `main.cpp` is:

```
Team of size 5:
  0: Thandi Mokoena, 29, F, ID: none
  1: empty
  2: Anna Botha, 41, F, ID: none
  3: Ravi Naidoo, 38, M, ID: none
  4: empty
Oldest from index 0: 41
Oldest from index 3: 38
Oldest from index 4: 0
After removing member 2, from index 0: 38
```

Common mistakes

- **Returning the first age found.** A version that returns the age of the first employee that it meets gives a wrong answer. For the team above, `oldest(0)` returns `29` instead of `41`. The output of the main test then differs from the expected output.
- **Testing the age before `NULL`.** If the condition is `members[index]->getAge() > restOldest && members[index] != NULL`, the first part runs on a `NULL` slot. The call goes through a null pointer. This is undefined behaviour. In the test run, valgrind reported `Invalid read of size 4`, and the program stopped with a segmentation fault (`SIGSEGV`) in the first call `oldest(0)`.
- **No `NULL` check at all.** The same thing happens: `Invalid read of size 4`, then `SIGSEGV`. The main test has empty slots at index 1 and index 4, and the edge test starts with a team where every slot is empty.
- **Using `index > size` in the base case.** When `index` equals `size`, the function then reads `members[size]`, which is outside the array. This is undefined behaviour, so the result cannot be trusted. In the test run the output was still correct, but valgrind reported `Invalid read of size 8`, so the valgrind check fails.
- **Calling `oldest(index)` instead of `oldest(index + 1)`.** The argument never changes, so the recursion never ends. The program compiles, but at run time valgrind reports `Stack overflow` and the program stops with a segmentation fault (`SIGSEGV`).

- **A loop or a helper function.** QUESTION.md states that either one scores 0. The call to the given `Employee::getAge()` is allowed.

B_2 Linked lists: solutions

B2-01. Linked lists: `List<T>::append` (100)

File: B_2.cpp

```
#ifndef B_2_CPP
#define B_2_CPP

#include "List.h"

// Adds value at the end of the list.
template <class T>
void List<T>::append(T value) {
    Node<T>* newNode = new Node<T>(new T(value));
    // Case 1: the list is empty, so the new node becomes the head.
    if (headNode == NULL) {
        headNode = newNode;
        size++;
        return;
    }
    // Case 2: walk to the last node (the node whose next is NULL).
    Node<T>* curr = headNode;
    while (curr->next() != NULL) {
        curr = curr->next();
    }
    curr->next() = newNode;
    size++;
}

#endif
```

Expected output of the given main.cpp

```
Empty List (size 0)
10 (size 1)
10->20->30 (size 3)
5->10->20->30->40 (size 5)
```

What the function must do

The function `append` adds one value at the end of a singly linked list. A singly linked list is a chain of nodes. Each node stores a value and a pointer to the next node. The new node must come after the current last node. The order of the other nodes stays the same, and `size` increases by 1.

How the solution works

A pointer is a variable that holds the address of an object. The operator `->` uses a pointer to reach a member of the object. `NULL` is the value of a pointer that points to nothing. In `List<T>`, `T` is a template type. The compiler replaces `T` with a real type such as `int` or `string`.

The function starts here:

```
template <class T>
void List<T>::append(T value) {
```

The function takes `value` by copy. The next line builds the new node:

```
    Node<T>* newNode = new Node<T>(new T(value));
```

- `new T(value)` allocates a copy of the value on the heap.
- `new Node<T>(...)` allocates the node. The node stores the pointer to that copy.
- The node owns the copy. `~Node()` deletes it, so the function never calls `delete`.
- The given `Node` constructor sets `nextNode` to `NULL`. The new node is therefore a valid last node already.

The first branch handles the empty list:

```
if (headNode == NULL) {
    headNode = newNode;
    size++;
    return;
}
```

- The condition `headNode == NULL` is true only when the list has no nodes.
- An empty list has no last node, so the walk below would dereference `NULL`. Dereference means to use `->` on the pointer. In the tests, the program then stops with a segmentation fault (SIGSEGV).
- The assignment `headNode = newNode` makes the new node the first node and the last node.
- `size++` keeps `size` equal to the number of nodes.
- `return` leaves the function, so the walk below does not run.

If the list has at least one node, the function finds the last node:

```
Node<T>* curr = headNode;
while (curr->next() != NULL) {
    curr = curr->next();
}
```

- `curr` starts at the first node. This is safe, because the `if` above ended the function for an empty list.
- `curr->next()` returns the link to the next node. The condition `curr->next() != NULL` asks if another node follows `curr`.
- If a node follows, `curr = curr->next()` moves one step forward.
- The loop stops when `curr->next()` is `NULL`. At that point `curr` is the last node.
- The loop tests `curr->next()` and not `curr`. If the loop tested `curr != NULL`, it would end after the last node. The function would then have no node to link from.
- A list with one node skips the loop body. `curr` stays at the head, which is also the last node.

The last two lines attach the node:

```
curr->next() = newNode;
size++;
```

- `next()` returns a reference to the link (`Node<T>*&`). A reference is another name for an existing variable. The assignment therefore changes the real link inside the last node.
- The old last node now points to the new node. The new node still has `NULL` as its link, so it is the new last node.
- `head` does not change. The other nodes do not change.
- `size++` adds 1 for the new node.

Memory: each call allocates two blocks, the node and its value. The destructor of `List` frees both when the list ends. valgrind reports no leak.

Trace

Call from `main.cpp`: `numbers.append(30);`. Before the call, the list is `10->20` and `size` is 2.

Step	`curr` points to	`curr->next()`	Action
Setup	node 10	node 20	The `if` is false. `newNode` holds 30. `curr = headNode`.
Loop test 1	node 10	node 20 (not `NULL`)	`curr = curr->next()`, so `curr` is node 20.
Loop test 2	node 20	`NULL`	The loop stops.
After loop	node 20	`NULL`	`curr->next() = newNode`. The chain is `10->20->30`.
End	node 20	node 30	`size++`, so `size` is 3.

The earlier calls in `main.cpp` use the other branch. `numbers.append(10)` runs on an empty list. The `if` is true, so `headNode` becomes the new node and the function returns. The call `numbers.append(20)` then walks from node 10, which is the last node already, and links node 20 after it.

The program output is:

```
Empty List (size 0)
10 (size 1)
10->20->30 (size 3)
5->10->20->30->40 (size 5)
```

Common mistakes

- Walking with `while (curr != NULL)` and then writing to `curr->next()`. After the loop, `curr` is `NULL`. The write dereferences `NULL`. In the main test, the second call (`append(20)`) does this. valgrind reports "Invalid write of size 8" and the program stops with a segmentation fault (SIGSEGV).
- Leaving out the check `headNode == NULL`. The first call on an empty list reads `curr->next()` while `curr` is `NULL`. valgrind reports "Invalid read of size 8" and the program stops with SIGSEGV. Both the main test and the edge test start with an empty list, so both fail.
- Forgetting `size++` on one of the two paths. The list prints correctly, but the text `(size N)` shows the wrong number.
- Building the node with `new Node<T>(&value)`. `value` is a local variable, so the node keeps the address of a variable that stops existing when `append` returns. That is undefined behaviour. In the test, the printed values are wrong (every appended node showed the same number), and `~Node()` deletes a stack address, so valgrind reports "Invalid free() / delete / delete[] / realloc()".
- Setting `headNode = newNode` in the non-empty case. The old nodes become unreachable, so the list loses its values. In the main test, the list prints only `30` and then `40`, and valgrind reports "definitely lost: 48 bytes in 3 blocks". In the edge test, `names.head()->next()` is `NULL`, so the test reads through `NULL` and stops with SIGSEGV.

B2-02. Linked lists: `List<T>::insertAt` (100)

File: B_2.cpp

```
#ifndef B_2_CPP
#define B_2_CPP

#include "List.h"

// Inserts value so that it ends up at index pos (the first node is index 0).
// pos <= 0 adds at the front. pos >= size adds at the back.
template <class T>
void List<T>::insertAt(T value, int pos) {
    Node<T>* newNode = new Node<T>(new T(value));
    // Front: an empty list, or a position of 0 or less.
    if (headNode == NULL || pos <= 0) {
        newNode->next() = headNode;
        headNode = newNode;
        size++;
        return;
    }
    // Move prev to the node just before index pos.
    // If the list runs out first, prev stops at the last node (append).
    Node<T>* prev = headNode;
    int count = 1;
    while (prev->next() != NULL && count < pos) {
        prev = prev->next();
        count++;
    }
    newNode->next() = prev->next();
    prev->next() = newNode;
    size++;
}

#endif
```

Expected output of the given main.cpp

```
20 (size 1)
20->40 (size 2)
20->30->40 (size 3)
10->20->30->40 (size 4)
10->20->30->35->40 (size 5)
10->20->30->35->40->50 (size 6)
```

What the function must do

The function `insertAt` puts a new value into the list so that the value sits at index `pos`. Index 0 is the first node. The nodes that were at `pos` and later move one place to the right. If `pos` is 0 or less, the value goes at the front. If `pos` is as large as the list or larger, the value goes at the end. `size` increases by 1.

How the solution works

A pointer is a variable that holds the address of an object. The operator `->` uses a pointer to reach a member. `NULL` means the pointer points to nothing. `T` is the template type, such as `int` or `string`.

The function builds the new node first:

```
    Node<T>* newNode = new Node<T>(new T(value));
```

- `new T(value)` allocates a copy of the value on the heap.
- `new Node<T>(...)` allocates the node. The node owns the copy, and `~Node()` deletes it later.
- The given constructor sets the link of the new node to `NULL`.

The first branch handles the front of the list:

```
    if (headNode == NULL || pos <= 0) {
        newNode->next() = headNode;
        headNode = newNode;
```

```
        size++;
        return;
    }
```

- The test `headNode == NULL` covers an empty list. An empty list has no node to walk from, so the new node must become the head.
- The test `pos <= 0` covers index 0 and every negative index. The question says these insert at the head.
- The operator `||` is true if either test is true.
- `newNode->next() = headNode` links the new node to the old first node. This line must run before the next line. If the order changes, the old head pointer is overwritten and the old nodes are lost.
- `headNode = newNode` makes the new node the first node.
- For an empty list, `headNode` is `NULL`. The new node then points to `NULL` and is the only node.
- `return` ends the function, so the walk below does not run.

The rest of the function handles `pos >= 1` on a list with at least one node:

```
    Node<T>* prev = headNode;
    int count = 1;
    while (prev->next() != NULL && count < pos) {
        prev = prev->next();
        count++;
    }
```

- `prev` must end at the node directly before index `pos`. The function needs this node to change its link.
- `count` is the index of the node after `prev`. `prev` is the head at index 0, so `count` starts at 1.
- The test `count < pos` stops the walk at the right place. It is false when `count` equals `pos`. At that moment `prev` is at index `pos - 1`, the node directly before the new position.
- The test `prev->next() != NULL` protects against a `pos` larger than the list. Without it, `prev` would become `NULL`, and `prev->next()` would read through `NULL`. The program stops with a segmentation fault (SIGSEGV). With it, `prev` stops at the last node, and the new node goes at the end.
- The operator `&&` needs both tests to be true. The test on `prev->next()` runs first, so `NULL` is never dereferenced.

The last lines insert the node:

```
    newNode->next() = prev->next();
    prev->next() = newNode;
    size++;
```

- `newNode->next() = prev->next()` points the new node at the node that follows `prev`. At the end of the list, this value is `NULL`, so the new node becomes the last node.
- `prev->next() = newNode` then points `prev` at the new node.
- The order matters. If `prev->next() = newNode` runs first, the original follower of `prev` is lost. Those nodes can never be reached or freed.
- `size++` keeps `size` equal to the number of nodes.

Memory: the function allocates one node and one value. The destructor of `List` frees them when the list ends. valgrind shows no leak.

Trace

Call from `main.cpp`: `numbers.insertAt(35, 3);`. Before the call, the list is `10->20->30->40` and `size` is 4.

Step	prev points to	count	Test: prev->next() != NULL and count < pos	Action
Setup	node 10	1	not tested yet	The first `if` is false (`pos` is 3).
Loop test 1	node 10	1	true and `1 < 3` is true	`prev` moves to node 20, `count` becomes 2.
Loop test 2	node 20	2	true and `2 < 3` is true	`prev` moves to node 30, `count` becomes 3.
Loop test 3	node 30	3	`3 < 3` is false	The loop stops.

Step	prev points to	count	Test: prev->next() != NULL and count < pos	Action
Link	node 30	3		newNode->next() becomes node 40.
Link	node 30	3		prev->next() becomes the new node.

The chain is now `10->20->30->35->40`. The call sets `size` to 5.

The earlier call `numbers.insertAt(40, 5)` runs on the one-node list `20`. The loop test fails at once because `prev->next()` is `NULL`. The new node links after node 20 and the list is `20->40`.

The program output is:

```
20 (size 1)
20->40 (size 2)
20->30->40 (size 3)
10->20->30->40 (size 4)
10->20->30->35->40 (size 5)
10->20->30->35->40->50 (size 6)
```

Common mistakes

- Checking only `pos == 0` and not `pos <= 0`. A negative `pos` then reaches the walk. The test `count < pos` is false at once, so the loop does not run. The code inserts the node after the head, at index 1, and not at the front. In the main test, `insertAt(10, -3)` gives `20->10->30->40` and not `10->20->30->40`. The edge test calls `insertAt(-1000, -1000)` and fails in the same way.
- Leaving out `prev->next() != NULL` in the loop condition. A `pos` larger than the list moves `prev` to `NULL`. The next `prev->next()` dereferences `NULL`. In the main test, `insertAt(40, 5)` does this: valgrind reports "Invalid read of size 8" and the program stops with a segmentation fault (SIGSEGV).
- Leaving out `headNode == NULL` in the first test. For an empty list with `pos` greater than 0, `prev` starts as `NULL`, and the loop test `prev->next()` reads through `NULL`. The main test does not do this, so it still passes. The edge test inserts at position 3 into an empty list: valgrind reports "Invalid read of size 8" and the program stops with SIGSEGV.
- Writing `prev->next() = newNode` before `newNode->next() = prev->next()`. The new node then points to itself, and the original follower nodes are lost. The print loop never ends. In the main test, the second line prints `20->40->40->40...` without stopping.
- Forgetting `size++` on one path. The list prints correctly, but the printed size is wrong.

B2-03. Linked lists: `List<T>::remove` by value (100)

File: B_2.cpp

```
#ifndef B_2_CPP
#define B_2_CPP

#include "List.h"

// Removes the first node that holds value. Returns true if a node was removed.
template <class T>
bool List<T>::remove(T value) {
    // Case 1: empty list, nothing to remove.
    if (headNode == NULL) {
        return false;
    }
    // Case 2: the head holds the value.
    if (headNode->getData() == value) {
        Node<T>* temp = headNode;
        headNode = headNode->next();
        delete temp;    // ~Node() deletes the data
        size--;
        return true;
    }
    // Case 3: look further, keeping prev one node behind curr.
    // The loop checks curr != NULL first, so a one-node list is safe.
    Node<T>* prev = headNode;
    Node<T>* curr = headNode->next();
    while (curr != NULL) {
        if (curr->getData() == value) {
            prev->next() = curr->next();
            delete curr;
            size--;
            return true;
        }
        prev = curr;
        curr = curr->next();
    }
    // Case 4: not found.
    return false;
}

#endif
```

Expected output of the given main.cpp

```
1->2->3->4->5 (size 5)
remove(3): true -> 1->2->4->5 (size 4)
remove(1): true -> 2->4->5 (size 3)
remove(5): true -> 2->4 (size 2)
remove(9): false -> 2->4 (size 2)
```

What the function must do

The function `remove` searches the list from the head for the first node whose value equals `value`. If the function finds the node, it removes the node from the chain, deletes it, subtracts 1 from `size` and returns `true`. If the list is empty or no node matches, the function changes nothing and returns `false`. Only the first match is removed.

How the solution works

A pointer is a variable that holds the address of an object. The operator `->` uses a pointer to reach a member. `NULL` means the pointer points to nothing. `T` is the template type, so the function compares values with `==` and `T` must support that operator.

The function has four cases. The first case handles the empty list:

```
    if (headNode == NULL) {
        return false;
    }
```

- The condition checks for an empty list. The next case calls `headNode->getData()`. If `headNode` is `NULL`, that call reads through `NULL`, and the program stops with a segmentation fault (SIGSEGV).
- An empty list has nothing to remove, so the function returns `false`.

The second case handles a match at the head:

```
if (headNode->getData() == value) {
    Node<T>* temp = headNode;
    headNode = headNode->next();
    delete temp;    // ~Node() deletes the data
    size--;
    return true;
}
```

- `getData()` returns the stored value. The condition asks if the first node holds `value`.
- The head needs its own case because no node comes before it. The usual rule "make the previous node skip this node" has no previous node here. The function must change `headNode` itself.
- `temp` remembers the node that will be deleted. A local pointer variable holds this address.
- `headNode = headNode->next()` moves the head to the second node. For a one-node list, `next()` is `NULL`, so the list becomes empty.
- `delete temp` frees the old head. The order matters. If `delete temp` ran first, `headNode->next()` would read freed memory.
- `~Node()` also deletes the value that the node owns. The function does not delete the value by hand, because that would cause a double delete.
- `size--` and `return true` finish the removal.

The third case searches the rest of the list:

```
Node<T>* prev = headNode;
Node<T>* curr = headNode->next();
while (curr != NULL) {
```

- `prev` points to the node before `curr`. The function needs `prev` to link around the removed node.
- `curr` starts at the second node. The head was already checked in case 2.
- The condition `curr != NULL` stops the loop after the last node. For a one-node list, `curr` is `NULL` at the start, so the loop does not run and the function reaches `return false`. The loop never reads `curr->getData()` through `NULL`.

Inside the loop:

```
    if (curr->getData() == value) {
        prev->next() = curr->next();
        delete curr;
        size--;
        return true;
    }
    prev = curr;
    curr = curr->next();
```

- The `if` checks if the current node holds `value`.
- `prev->next() = curr->next()` links the node before `curr` to the node after `curr`. The chain now skips `curr`. If `curr` is the last node, `curr->next()` is `NULL`, so `prev` becomes the last node.
- `delete curr` then frees the node and its value. The link must change first. After `delete`, the program cannot read `curr->next()` safely.
- If the node does not match, `prev = curr` and `curr = curr->next()` move both pointers one node forward. `prev` is assigned first, so it takes the old `curr` before `curr` changes.

The fourth case runs when the loop ends without a match:

```
return false;
```

The list is unchanged, `size` is unchanged, and the function reports that nothing was removed.

Memory: the function frees exactly one node and its value for each successful removal. The destructor of `List` frees any node that remains.

Trace

Call from `main.cpp`: `tryRemove(numbers, 3);`. Before the call, the list is `1->2->3->4->5` and `size` is 5.

Step	`prev`	`curr`	Check	Result
Case 1			`headNode == NULL` is false	Continue.
Case 2			head holds 1, and `1 == 3` is false	Continue.
Setup	node 1	node 2	`curr != NULL` is true	Enter the loop.
Iteration 1	node 1	node 2	`2 == 3` is false	`prev` becomes node 2, `curr` becomes node 3.
Iteration 2	node 2	node 3	`3 == 3` is true	`prev->next()` becomes node 4. The chain is `1->2->4->5`.
End	node 2	node 3		`delete curr` frees node 3 and its value. `size` becomes 4. Return `true`.

The later calls use other cases. The call `remove(1)` takes case 2 and moves `headNode` to node 2. The call `remove(5)` finds the last node in the loop, and `prev->next()` becomes `NULL`. The call `remove(9)` runs the loop to the end and returns `false`.

The program output is:

```
1->2->3->4->5 (size 5)
remove(3): true -> 1->2->4->5 (size 4)
remove(1): true -> 2->4->5 (size 3)
remove(5): true -> 2->4 (size 2)
remove(9): false -> 2->4 (size 2)
```

Common mistakes

- Skipping the head case. A match at the head has no node before it. If the code starts with `prev = NULL` and `curr = headNode`, a match at the head runs `prev->next()` with `prev` equal to `NULL`. In the main test, `remove(1)` does this. valgrind reports "Invalid write of size 8" and the program stops with a segmentation fault (SIGSEGV). If the loop starts at the second node and there is no head case, `remove(1)` returns `false` and the output is wrong.
- Writing a search loop that reads `curr->getData()` or `curr->next()` without testing `curr` for `NULL`. The edge test catches this. In that test, a one-node list searches for a value that is not there. `curr` starts as `headNode->next()`, which is `NULL`, and the loop reads `curr->getData()` through `NULL`. valgrind reports "Invalid read of size 8" and the program stops with SIGSEGV.
- Calling `delete curr` before `prev->next() = curr->next()`. The program then reads a freed node, which is undefined behaviour. valgrind reports "Invalid read of size 8". In the test, the nodes after the removed node were cut off the list (`remove(3)` printed `1->2`), and valgrind also reported a leak.
- Forgetting `size--`. The list prints correctly but shows the wrong size.
- Removing every match instead of the first. The duplicate test `7->8->7` expects `8->7` after the first call, and the output differs.

B2-04. Linked lists: `List<T>::removeAt` (100)

File: B_2.cpp

```
#ifndef B_2_CPP
#define B_2_CPP

#include "List.h"

// Removes the node at index pos (the first node is index 0).
// Returns false and changes nothing when pos is out of range.
template <class T>
bool List<T>::removeAt(int pos) {
    // Out of range (this also covers an empty list).
    if (pos < 0 || pos >= size) {
        return false;
    }
    // Remove the head.
    if (pos == 0) {
        Node<T>* temp = headNode;
        headNode = headNode->next();
        delete temp;
        size--;
        return true;
    }
    // Move prev to the node just before index pos.
    Node<T>* prev = headNode;
    for (int i = 0; i < pos - 1; i++) {
        prev = prev->next();
    }
    Node<T>* curr = prev->next();
    prev->next() = curr->next();
    delete curr;
    size--;
    return true;
}

#endif
```

Expected output of the given main.cpp

```
10->20->30->40->50 (size 5)
removeAt(2): true -> 10->20->40->50 (size 4)
removeAt(0): true -> 20->40->50 (size 3)
removeAt(2): true -> 20->40 (size 2)
removeAt(5): false -> 20->40 (size 2)
removeAt(-1): false -> 20->40 (size 2)
```

What the function must do

The function `removeAt` removes the node at index `pos`. Index 0 is the first node. If `pos` is negative or not smaller than the number of nodes, the function changes nothing and returns `false`. Otherwise the function unlinks the node, deletes it, subtracts 1 from `size` and returns `true`.

How the solution works

A pointer is a variable that holds the address of an object. The operator `->` uses a pointer to reach a member. `NULL` means the pointer points to nothing. `T` is the template type, such as `int` or `string`.

The function starts with the range check:

```
    if (pos < 0 || pos >= size) {
        return false;
    }
```

- `pos < 0` rejects negative indexes. A negative index has no node.
- `pos >= size` rejects an index at or past the end. The valid indexes are 0 to `size - 1`.
- The operator `||` is true if either test is true.
- An empty list has `size` equal to 0. Every `pos` then fails one of the two tests, so the function returns `false`.

- After this check, the list has at least one node and `pos` names an existing node. The code below does not need to test for `NULL` again.

The second branch removes the head:

```
if (pos == 0) {
    Node<T>* temp = headNode;
    headNode = headNode->next();
    delete temp;
    size--;
    return true;
}
```

- The condition `pos == 0` selects the first node. The first node has no node before it, so the walk below cannot handle it.
- `temp` saves the address of the first node.
- `headNode = headNode->next()` moves the head to the second node. For a one-node list, `next()` is `NULL`, so the list becomes empty.
- `delete temp` frees the old first node. This line comes after the head moves. If `delete` ran first, `headNode->next()` would read a freed node.
- `~Node()` deletes the value that the node owns. The function does not delete the value itself, because that would delete it twice.
- `size--` keeps `size` equal to the number of nodes.

The remaining case has `pos` of at least 1. The function finds the node before index `pos`:

```
Node<T>* prev = headNode;
for (int i = 0; i < pos - 1; i++) {
    prev = prev->next();
}
```

- `prev` starts at index 0.
- The loop condition `i < pos - 1` makes the loop run `pos - 1` times. After that, `prev` sits at index `pos - 1`, the node directly before the target.
- The function needs this node to change its link.
- The loop does not test `prev->next()` for `NULL`. The range check at the top guarantees that index `pos` exists, so each step stays inside the list.

The last lines remove the node:

```
Node<T>* curr = prev->next();
prev->next() = curr->next();
delete curr;
size--;
return true;
```

- `curr` is the node at index `pos`. This node will be removed.
- `prev->next() = curr->next()` links the node before `curr` to the node after `curr`. The chain now skips `curr`. If `curr` is the last node, `curr->next()` is `NULL`, so `prev` becomes the last node.
- `delete curr` frees the node and its value. The link change must come first, because the function reads `curr->next()` before the node is freed.
- `size--` and `return true` finish the removal.

Memory: each successful call frees one node and its value. The destructor of `List` frees the nodes that remain.

Trace

Call from `main.cpp`: `tryRemoveAt(numbers, 2);`. Before the call, the list is `10->20->30->40->50` and `size` is 5.

Step	`i`	prev points to	Action
Range check			`2 < 0` is false and `2 >= 5` is false. Continue.
Head check			`pos == 0` is false. Continue.
Setup		node 10	`prev = headNode`.
Loop test	0	node 10	`0 < 1` is true. `prev` becomes node 20.

Step	i	prev points to	Action
Loop test	1	node 20	`1 < 1` is false. The loop ends.
After loop		node 20	`curr` is node 30.
Link		node 20	`prev->next()` becomes node 40. The chain is `10->20->40->50`.
End		node 20	`delete curr` frees node 30. `size` becomes 4. Return `true`.

The next call, `removeAt(0)`, uses the head branch and gives `20->40->50`. The call `removeAt(5)` fails the test `pos >= size` and returns `false`. The call `removeAt(-1)` fails the test `pos < 0` and returns `false`.

The program output is:

```
10->20->30->40->50 (size 5)
removeAt(2): true -> 10->20->40->50 (size 4)
removeAt(0): true -> 20->40->50 (size 3)
removeAt(2): true -> 20->40 (size 2)
removeAt(5): false -> 20->40 (size 2)
removeAt(-1): false -> 20->40 (size 2)
```

Common mistakes

- Writing the range check as `pos > size`. The value `pos == size` then passes the check. On an empty list, `removeAt(0)` reaches the head branch and reads `headNode->next()` through `NULL`. On a list with nodes, `curr` becomes `NULL` and `curr->next()` reads through `NULL`. The main test still passes. The first edge call, `removeAt(0)` on an empty list, makes valgrind report "Invalid read of size 8", and the program stops with a segmentation fault (SIGSEGV).
- Leaving out `pos < 0`. A negative `pos` skips the head branch, and the loop does not run. The function then removes the second node. In the main test, `removeAt(-1)` returns `true` and the list becomes `20`. On a one-node list, `curr` is `NULL`. The edge test calls `removeAt(-1)` on a one-node list: valgrind reports "Invalid read of size 8" and the program stops with SIGSEGV.
- Not handling `pos == 0` separately. The loop limit `pos - 1` is then -1, so the loop does not run and `prev` stays at the head. The code removes the second node, not the first. In the main test, `removeAt(0)` gives `10->40->50`. On a one-node list, `curr` is `NULL`, and the edge test stops with SIGSEGV after valgrind reports "Invalid read of size 8".
- Calling `delete curr` before `prev->next() = curr->next()`. The function then reads a freed node, which is undefined behaviour. valgrind reports "Invalid read of size 8". In the main test, the nodes after the removed node were cut off (`removeAt(2)` printed `10->20`), valgrind reported a leak, and the program then stopped with SIGSEGV.
- Deleting the value as well as the node, for example `delete &curr->getData();` before `delete curr;`. `~Node()` deletes the value too, so the value is deleted twice. valgrind reports "Invalid free() / delete / delete[] / realloc()".
- Forgetting `size--`. The list prints correctly, but the printed size is wrong. If `size--` is missing in the head branch, the edge test removes the only node of a one-node list, and `size` stays 1. The next `removeAt(0)` passes the range check and reads `headNode->next()` while `headNode` is `NULL`. valgrind reports "Invalid read of size 8" and the program stops with SIGSEGV.

B2-05. Linked lists: `List<T>::reverse` (100)

File: B_2.cpp

```
#ifndef B_2_CPP
#define B_2_CPP

#include "List.h"

// Reverses the order of the nodes by turning every link around.
// No node is created or deleted.
template <class T>
void List<T>::reverse() {
    Node<T>* prev = NULL;
    Node<T>* curr = headNode;
    while (curr != NULL) {
        Node<T>* temp = curr->next();   // remember the rest of the list
        curr->next() = prev;            // turn this link around
        prev = curr;
        curr = temp;
    }
    headNode = prev;                    // the old last node is the new head
}

#endif
```

Expected output of the given main.cpp

```
1->2->3->4->5 (size 5)
5->4->3->2->1 (size 5)
1->2->3->4->5 (size 5)
```

What the function must do

The function `reverse` changes the order of the nodes so that the last node becomes the first node. The function does this by changing the links only. It creates no node and deletes no node. `size` does not change. An empty list and a one-node list stay as they are.

How the solution works

A pointer is a variable that holds the address of an object. The operator `->` uses a pointer to reach a member. `NULL` means the pointer points to nothing. `next()` returns a reference to the link of a node, so an assignment to `curr->next()` changes the real link.

The function uses two walking pointers:

```
    Node<T>* prev = NULL;
    Node<T>* curr = headNode;
```

- `curr` is the node that the loop works on. It starts at the first node.
- `prev` is the node that came before `curr` in the original list. At the start there is no such node, so `prev` is `NULL`.
- The first node will become the last node. A last node must have `NULL` as its link. The start value of `prev` supplies that `NULL`.

The loop turns each link around:

```
    while (curr != NULL) {
```

- The condition `curr != NULL` is true while a node remains to process. The loop ends when `curr` moves past the last node.
- For an empty list, `headNode` is `NULL`, so the loop body never runs. For a one-node list, the loop body runs once.

Each pass has four lines:

```
        Node<T>* temp = curr->next();   // remember the rest of the list
        curr->next() = prev;            // turn this link around
        prev = curr;
        curr = temp;
```

1. `temp = curr->next()` saves the address of the node after `curr`. The next line overwrites this link, so the function must save it first. Without `temp`, the rest of the list is lost.
2. `curr->next() = prev` points `curr` back at the node before it. This reverses one link.
3. `prev = curr` moves `prev` forward. `curr` is now the front of the reversed part.
4. `curr = temp` moves `curr` to the next unprocessed node.

The order of these four lines matters. Line 2 destroys the old forward link, and line 1 keeps a copy of it. Lines 3 and 4 must run after line 2, because line 2 uses the old `prev` and the old `curr`. Line 3 must run before line 4, because line 3 copies the old `curr` and line 4 changes `curr`.

After the loop:

```
    headNode = prev;                  // the old last node is the new head
```

- When the loop ends, `curr` is `NULL` and `prev` is the old last node.
- `headNode` still points to the old first node, which is now the last node. The assignment moves `headNode` to the old last node.
- For an empty list, `prev` is `NULL`, so `headNode` stays `NULL`.
- The function does not change `size`, because the number of nodes is the same.

Memory: the function calls neither `new` nor `delete`. Every node stays in use, so valgrind shows no leak.

Trace

Call from `main.cpp`: the first `numbers.reverse();`. Before the call, the list is `1->2->3->4->5`.

Iteration	`curr`	`temp`	After `curr->next() = prev`	`prev` after	`curr` after
Start	node 1			`NULL`	node 1
1	node 1	node 2	`1->NULL`	node 1	node 2
2	node 2	node 3	`2->1->NULL`	node 2	node 3
3	node 3	node 4	`3->2->1->NULL`	node 3	node 4
4	node 4	node 5	`4->3->2->1->NULL`	node 4	node 5
5	node 5	`NULL`	`5->4->3->2->1->NULL`	node 5	`NULL`

The loop test `curr != NULL` is false after iteration 5. The line `headNode = prev` then makes node 5 the head. The list is `5->4->3->2->1`.

The second `numbers.reverse();` repeats the same steps on this list and restores `1->2->3->4->5`.

The program output is:

```
1->2->3->4->5 (size 5)
5->4->3->2->1 (size 5)
1->2->3->4->5 (size 5)
```

Common mistakes

- Leaving out `temp`, for example by writing `curr = curr->next();` as the last line. After `curr->next() = prev`, that link already points back, so the function cannot find the next node. In the main test, the loop stops after one pass. The list prints `1 (size 5)`, the other nodes are lost, and valgrind reports "definitely lost".
- Leaving out `headNode = prev`. The head still points to the old first node. That node is now the last node and its link is `NULL`. In the main test, the program prints `1 (size 5)`. The other nodes are never freed, so valgrind reports "definitely lost".
- Starting with `prev` set to `headNode` and not to `NULL`. The old first node then points to itself. The list has a cycle, and the print loop never ends. In the main test, the second line prints `5->4->3->2->1->1->1...` without stopping.
- Building a new list with `new` and then deleting the old nodes. The question forbids this because the marking program checks that the same nodes are used. The edge test compares node addresses, so new nodes give `no` in the lines "Same head node", "Old tail is the new head" and "Old head is the new tail". The edge test also reads a deleted node through its saved pointer, so valgrind reports "Invalid read of size 8".
- Looping with `while (curr->next() != NULL)`. The loop skips the last node, so the last node is not linked into the reversed list. In the main test, the list prints `4->3->2->1`, and valgrind reports a leak for the lost node. For an empty list, `curr` is `NULL` and `curr->next()` reads through `NULL`. The edge test starts with an empty list: valgrind reports "Invalid read of size 8" and the program stops with a segmentation fault (SIGSEGV).

B2-06. Linked lists: `List<T>::sort` by relinking (100)

File: B_2.cpp

```
#ifndef B_2_CPP
#define B_2_CPP

#include "List.h"

// Sorts the list in ascending order by relinking the nodes (insertion sort).
// The data cannot be changed, so whole nodes are moved instead.
template <class T>
void List<T>::sort() {
    Node<T>* sorted = NULL;           // head of the sorted part
    Node<T>* curr = headNode;
    while (curr != NULL) {
        Node<T>* temp = curr->next(); // remember the rest of the unsorted part
        // Case 1: curr goes in front of the sorted part.
        if (sorted == NULL || curr->getData() < sorted->getData()) {
            curr->next() = sorted;
            sorted = curr;
        } else {
            // Case 2: find the last sorted node that is <= curr, and link curr after it.
            Node<T>* prev = sorted;
            while (prev->next() != NULL && prev->next()->getData() <= curr->getData()) {
                prev = prev->next();
            }
            curr->next() = prev->next();
            prev->next() = curr;
        }
        curr = temp;
    }
    headNode = sorted;
}

#endif
```

Expected output of the given main.cpp

```
0->5->3->6->-1 (size 5)
-1->0->3->5->6 (size 5)
```

What the function must do

The function `sort` puts the values in ascending order, with the smallest value at the head. The node data is read-only, because `getData()` returns a `const T&` and `Node` has no setter. The function therefore moves whole nodes by changing links. It creates no node and deletes no node. `size` does not change.

How the solution works

A pointer is a variable that holds the address of an object. The operator `->` uses a pointer to reach a member. `NULL` means the pointer points to nothing. `T` is the template type. The function compares values with `<` and `<=`, so `T` must support them.

The method is insertion sort. The function builds a second chain called the sorted part. It takes the nodes of the original list one at a time and links each node into the right place in the sorted part. The original list is the unsorted part.

```
    Node<T>* sorted = NULL;           // head of the sorted part
    Node<T>* curr = headNode;
```

- `sorted` points to the first node of the sorted part. The sorted part is empty at the start, so `sorted` is `NULL`.
- `curr` points to the node that the function moves next. It starts at the first node of the original list.

The outer loop runs once for each node:

```
    while (curr != NULL) {
        Node<T>* temp = curr->next(); // remember the rest of the unsorted part
```

- The condition `curr != NULL` is false for an empty list, so the function does nothing and `headNode` stays `NULL`.

- `temp` saves the node after `curr`. The code below changes `curr->next()`. Without `temp`, the unsorted part would be lost.

The first case puts `curr` at the front of the sorted part:

```
        if (sorted == NULL || curr->getData() < sorted->getData()) {
            curr->next() = sorted;
            sorted = curr;
        } else {
```

- `sorted == NULL` is true for the first node. The sorted part is empty, so `curr` becomes its only node.
- `curr->getData() < sorted->getData()` is true when `curr` holds a value smaller than the current first sorted value. The new node must then come before all sorted nodes.
- The operator `||` stops at the first true test. When `sorted` is `NULL`, the second test does not run, so the program never reads `sorted->getData()` through `NULL`.
- `curr->next() = sorted` points `curr` at the old front of the sorted part.
- `sorted = curr` makes `curr` the new front. The order matters. If the two lines swap, the old front is lost.

The second case finds the place inside or at the end of the sorted part:

```
            Node<T>* prev = sorted;
            while (prev->next() != NULL && prev->next()->getData() <= curr->getData()) {
                prev = prev->next();
            }
            curr->next() = prev->next();
            prev->next() = curr;
        }
```

- `prev` starts at the first sorted node. The first sorted node is known to be not larger than `curr`, because case 1 was false.
- The test `prev->next() != NULL` stops the walk at the last sorted node. Without it, `prev->next()->getData()` would dereference `NULL`.
- The test `prev->next()->getData() <= curr->getData()` asks if the node after `prev` still belongs before `curr`. If so, `prev` moves forward.
- The walk stops when the next sorted node is larger than `curr`, or when the sorted part ends. The walk uses `<=`, so `curr` goes after sorted nodes that hold an equal value.
- `curr->next() = prev->next()` points `curr` at the node that follows `prev`. At the end of the sorted part, that value is `NULL`, so `curr` becomes the last node with a `NULL` link.
- `prev->next() = curr` then links `prev` to `curr`. The order matters. If this line runs first, the nodes after `prev` are lost.

The end of the outer loop and the function:

```
        curr = temp;
    }
    headNode = sorted;
```

- `curr = temp` moves to the next node of the unsorted part.
- After the loop, every node is in the sorted part. `headNode = sorted` makes the sorted part the list.
- The last node of the sorted part keeps a `NULL` link. A node placed at the end copies `NULL` from `prev->next()`, and no later step changes it.

Memory: the function calls neither `new` nor `delete`. Every node is reused, so valgrind shows no leak.

Trace

Call from `main.cpp`: `numbers.sort();`. Before the call, the list is `0->5->3->6->-1`.

Iteration	`curr`	`temp`	Case	Sorted part after the iteration
1	0	5	`sorted == NULL`, case 1	`0`
2	5	3	`5 < 0` is false. `prev` is 0 and `prev->next()` is `NULL`.	`0->5`
3	3	6	`3 < 0` is false. `prev` is 0. `5 <= 3` is false, so `prev` stays.	`0->3->5`
4	6	-1	`6 < 0` is false. `prev` moves 0, 3, 5. Then `prev->next()` is `NULL`.	`0->3->5->6`
5	-1	`NULL`	`-1 < 0` is true, case 1	`-1->0->3->5->6`

After iteration 5, `curr` becomes `NULL` and the loop ends. The assignment `headNode = sorted` makes node -1 the head.

The program output is:

```
0->5->3->6->-1 (size 5)
-1->0->3->5->6 (size 5)
```

Common mistakes

- Skipping `temp`. After `curr->next() = sorted` or `curr->next() = prev->next()`, the function cannot find the rest of the unsorted part. The outer loop ends early and nodes are lost. If the last line of the loop is `curr = curr->next();`, the main test prints `0 (size 5)` after the sort, and valgrind reports "definitely lost".
- Creating new nodes for the sorted part, for example `new Node<T>(new T(curr->getData()))`. The sorted list may print correctly. The edge test checks that the same nodes are reused, so it prints `Same nodes: no`. If the old nodes are not deleted, valgrind also reports "definitely lost".
- Leaving out `headNode = sorted`. The head still points to the old first node, whose link now leads into the sorted part. The printed list is wrong and the nodes before it in the sorted order are lost. The main test prints `0->3->5->6`, and valgrind reports "definitely lost" for node -1.
- Leaving out `prev->next() != NULL` in the inner loop. When `curr` is larger than every sorted value, `prev->next()` is `NULL` and the test `prev->next()->getData()` reads through `NULL`. In the main test, the second node (5) does this: valgrind reports "Invalid read of size 8" and the program stops with a segmentation fault (SIGSEGV).
- Leaving out `sorted == NULL` in case 1. The first node then reads `sorted->getData()` through `NULL`. valgrind reports "Invalid read of size 8" and the program stops with SIGSEGV.
- Trying to swap the data between nodes. `getData()` returns a `const T&` and `Node` has no setter. A line such as `a->getData() = b->getData();` does not compile: g++ reports "assignment of read-only location".

B2-07. Linked lists: `List<T>::indexOf` (100)

File: B_2.cpp

```
#ifndef B_2_CPP
#define B_2_CPP

#include "List.h"

// Returns the index of the first node that holds value, or -1 if there is none.
template <class T>
int List<T>::indexOf(T value) {
    Node<T>* curr = headNode;
    int index = 0;
    while (curr != NULL) {
        if (curr->getData() == value) {
            return index;   // early exit: the first match wins
        }
        curr = curr->next();
        index++;
    }
    return -1;              // empty list, or not found
}

#endif
```

Expected output of the given main.cpp

```
Sales->Marketing->HR->Development
indexOf(Sales): 0
indexOf(HR): 2
indexOf(Development): 3
indexOf(Finance): -1
```

What the function must do

The function `indexOf` finds the first node, counting from the head, whose value equals `value`. The function returns the index of that node. The first node has index 0. If the list is empty or no node holds the value, the function returns -1. The function does not change the list.

How the solution works

A pointer is a variable that holds the address of an object. The operator `->` uses a pointer to reach a member. `NULL` means the pointer points to nothing. `T` is the template type, such as `int` or `string`. The function compares values with `==`, so `T` must support that operator.

The function starts with two variables:

```
    Node<T>* curr = headNode;
    int index = 0;
```

- `curr` is a pointer to the node that the function checks now. It starts at the first node.
- `index` is the position of `curr`. The first node has position 0, so `index` starts at 0.
- The function only reads the nodes. It does not call `new` or `delete`, and it does not change any link.

The loop visits the nodes one at a time:

```
    while (curr != NULL) {
```

- The condition `curr != NULL` is true while `curr` points to a real node.
- For an empty list, `headNode` is `NULL`, so the loop body never runs. The function goes straight to `return -1`.
- The loop ends when `curr` moves past the last node and becomes `NULL`. The program never calls `getData()` through `NULL`.

The check inside the loop:

```
        if (curr->getData() == value) {
            return index;   // early exit: the first match wins
```

```
        }
```

- `getData()` returns the value that the node stores. The condition asks if that value equals `value`.
- If the condition is true, the function returns `index` at once. The function leaves the loop, so later nodes are not checked.
- This early `return` is the reason the function reports the first match. If a list has duplicates, the first match has the smallest index. The edge test uses `HR->Sales->HR->Sales`, and the answer for `Sales` is 1.
- Comparison with `==` is exact. For `string`, `"hr"` does not equal `"HR"`. The edge test shows `indexOf(hr): -1`.

The step to the next node:

```
        curr = curr->next();
        index++;
```

- `curr->next()` is the link to the following node. The assignment moves `curr` one node forward.
- `index++` adds 1, so `index` keeps matching the position of `curr`.
- Both lines run only when the current node does not match.

The last line:

```
    return -1;              // empty list, or not found
```

- The line runs only if the loop ended without a match. This happens for an empty list and for a value that is not in the list.
- The value -1 is not a valid index, so the caller can tell "not found" from a real position.

Memory: the function allocates nothing, so valgrind shows no leak.

Trace

Call from `main.cpp`: `find(departments, string("HR"));`, which calls `list.indexOf(value)`. The list is `Sales->Marketing->HR->Development`.

Iteration	`curr` points to	`index`	`curr->getData() == "HR"`	Action
1	Sales	0	false	`curr` moves to Marketing, `index` becomes 1.
2	Marketing	1	false	`curr` moves to HR, `index` becomes 2.
3	HR	2	true	The function returns 2.

The loop stops at iteration 3. The node Development is never checked.

For `Finance`, no node matches. Four iterations run, `curr` becomes `NULL` after Development, the loop ends and the function returns -1.

The program output is:

```
Sales->Marketing->HR->Development
indexOf(Sales): 0
indexOf(HR): 2
indexOf(Development): 3
indexOf(Finance): -1
```

Common mistakes

- Leaving out the final `return -1`. The function then reaches the end without a return value for the not-found case. With `-Wall`, g++ warns about the missing return, and `-Werror` turns the warning into an error: "control reaches end of non-void function [-Werror=return-type]". The program does not compile.
- Returning 0 or `size` for "not found". The expected output shows -1, so the output is wrong.
- Writing `while (curr->next() != NULL)`. The loop never checks the last node, so `indexOf(Development)` returns -1 instead of 3. For an empty list, `curr` is `NULL`, and the loop test reads `curr->next()` through `NULL`. The edge test starts with an empty list: valgrind reports "Invalid read of size 8" and the program stops with a segmentation fault (SIGSEGV).
- Not stopping at the first match, for example by saving the index and continuing the loop. The function then returns the last match. The main test has no duplicates, so it still passes. The edge test expects 1 for `Sales` in `HR->Sales->HR->Sales`, and this version returns 3.
- Starting `index` at 1. Every index that is found is then one too large. The answer for "not found" stays -1.

B2-08. Linked lists: `List<T>::operator[]` (100)

File: B_2.cpp

```
#ifndef B_2_CPP
#define B_2_CPP

#include "List.h"

// Returns the data in the node at position index (the head is position 0).
// Throws an Exception when index is out of range.
template <class T>
const T& List<T>::operator[](int index) {
    if (index < 0 || index >= size) {
        throw Exception("Index out of bounds");
    }
    Node<T>* curr = headNode;
    for (int i = 0; i < index; i++) {
        curr = curr->next();
    }
    return curr->getData();
}

#endif
```

Expected output of the given main.cpp

```
10->20->30
list[0]: 10
list[2]: 30
list[1]: 20
list[3]: Caught: Index out of bounds
list[-1]: Caught: Index out of bounds
```

What the function must do

The function `operator[]` lets the caller write `list[index]` to read the value at position `index`. Position 0 is the head. If `index` is negative, or equal to or larger than the number of nodes, the function throws an `Exception` with the message `Index out of bounds`. Otherwise, the function returns a reference to the value that is stored inside the node. The function does not change the list.

How the solution works

A pointer is a variable that holds the address of an object. The operator `->` uses a pointer to reach a member. `NULL` means the pointer points to nothing. `T` is the template type. A reference is another name for an existing object. A return type of `const T&` returns the stored object itself, not a copy, and the caller cannot change it through the reference.

The function starts here:

```
template <class T>
const T& List<T>::operator[](int index) {
```

Calling `list[2]` runs `list.operator[](2)`. The function takes the index as an `int`.

The range check:

```
    if (index < 0 || index >= size) {
        throw Exception("Index out of bounds");
    }
```

- `index < 0` rejects negative positions. No node has a negative position.
- `index >= size` rejects positions at or past the end. The valid positions are 0 to `size - 1`.
- The operator `||` is true if either test is true.
- An empty list has `size` 0, so every index fails `index >= size`. The function throws and the code below never reads `headNode`, which is `NULL`.
- `throw Exception("Index out of bounds");` creates an `Exception` object and throws it by value. The function stops at this line. No value is returned.
- The check comes before any `new`. The function allocates nothing, so a throw cannot leak memory.

- The caller must catch the exception. The given `tryIndex` function uses `catch (Exception& e)` and prints `e.what()`, which returns the message. An uncaught exception ends the program, and the mark is 0.

The walk to the node:

```
Node<T>* curr = headNode;
for (int i = 0; i < index; i++) {
    curr = curr->next();
}
```

- `curr` starts at the head, which is position 0.
- The condition `i < index` makes the loop run `index` times. Each pass moves `curr` one node forward with `curr->next()`.
- For `index` equal to 0, the loop body does not run, and `curr` stays at the head.
- The loop does not test `curr` for `NULL`. The range check guarantees that position `index` exists, so each step stays inside the list.

The return line:

```
return curr->getData();
```

- `getData()` returns a `const T&` to the value that the node owns.
- The function returns that same reference. The reference stays valid as long as the node exists.
- The function must not copy the value into a local variable and return that variable. A local variable is destroyed when the function ends, so the reference would point to a variable that no longer exists. g++ warns about this, and `-Werror` makes the warning an error.

Memory: the function allocates nothing and frees nothing. The list is unchanged, so valgrind shows no leak.

Trace

Call from `main.cpp`: `tryIndex(numbers, 2);`, which runs `list[index]`. The list is `10->20->30` and `size` is 3.

Step	`i`	`curr` points to	Action
Range check			`2 < 0` is false and `2 >= 3` is false. No throw.
Setup		node 10	`curr = headNode`.
Loop test	0	node 10	`0 < 2` is true. `curr` moves to node 20.
Loop test	1	node 20	`1 < 2` is true. `curr` moves to node 30.
Loop test	2	node 30	`2 < 2` is false. The loop ends.
Return		node 30	The function returns the reference to 30.

The function `tryIndex` copies the value into `T value` and prints it. For `tryIndex(numbers, 3)`, the test `3 >= 3` is true. The function throws, `catch` runs, and the program prints the message.

The program output is:

```
10->20->30
list[0]: 10
list[2]: 30
list[1]: 20
list[3]: Caught: Index out of bounds
list[-1]: Caught: Index out of bounds
```

Common mistakes

- Writing `index > size` in the range check. The value `index == size` then passes. The walk ends with `curr` equal to `NULL`, and `curr->getData()` reads through `NULL`. The main test calls `list[3]` on a three-node list: valgrind reports "Invalid read of size 8" and the program stops with a segmentation fault (SIGSEGV).
- Leaving out `index < 0`. A negative index does not throw. The loop does not run, and the function returns the head value. In the main test, `list[-1]` prints `10` and not `Caught: Index out of bounds`. On an empty list, `curr` would be `NULL`, so `curr->getData()` would read through `NULL`.

- Returning a value such as -1 or a default value instead of throwing. The expected output shows `Caught: Index out of bounds`, so the output is wrong. For example, a function that returns a default `T()` prints `list[3]: 0` in the main test.
- Throwing a different type, such as `throw "Index out of bounds";`. The `catch (Exception& e)` does not match, so the exception is not caught. The program prints "terminate called after throwing an instance of 'char const*'" and stops with SIGABRT at `list[3]`. The mark is 0.
- Returning a copy, for example `T value = curr->getData(); return value;`. The return type is `const T&`, so the function returns a reference to a local variable that no longer exists. The code does not compile: g++ reports "reference to local variable 'value' returned [-Werror=return-local-addr]". The mark is 0.

B2-09. Linked lists: `List<T>::clear` (100)

File: B_2.cpp

```
#ifndef B_2_CPP
#define B_2_CPP

#include "List.h"

// Deletes every node. The list is empty afterwards and can be used again.
template <class T>
void List<T>::clear() {
    Node<T>* curr = headNode;
    while (curr != NULL) {
        Node<T>* temp = curr;      // never delete the loop variable itself
        curr = curr->next();
        delete temp;               // ~Node() deletes the data
    }
    headNode = NULL;
    size = 0;
}

#endif
```

Expected output of the given main.cpp

```
1->2->3->4 (size 4)
Empty List (size 0)
8->7 (size 2)
```

What the function must do

The function `clear` deletes every node of the list. After the call, `headNode` is `NULL` and `size` is 0. The list must still work afterwards. The caller can add values again, and the caller can call `clear` again, also on an empty list. In this question, the given destructor `~List()` calls `clear()`, so a wrong `clear` also breaks the destructor.

How the solution works

A pointer is a variable that holds the address of an object. The operator `->` uses a pointer to reach a member. `NULL` means the pointer points to nothing. `delete` frees a heap object that `new` created. `T` is the template type.

Each node was created with `new`, and each node owns its value. `~Node()` deletes the value of that node. `~Node()` does not delete the next node. The function must therefore visit every node and call `delete` on it.

The loop starts here:

```
    Node<T>* curr = headNode;
    while (curr != NULL) {
```

- `curr` points to the node that the loop handles. It starts at the first node.
- The condition `curr != NULL` is true while a node remains.
- For an empty list, `headNode` is `NULL`, so the loop body does not run. Calling `clear` on an empty list is safe.

The loop body has three lines:

```
        Node<T>* temp = curr;      // never delete the loop variable itself
        curr = curr->next();
        delete temp;               // ~Node() deletes the data
```

1. `temp = curr` remembers the node that will be deleted.
2. `curr = curr->next()` moves `curr` to the following node. The function reads the link while the node still exists.
3. `delete temp` frees the old node. `~Node()` also frees the value inside it.

The order matters. If `delete` ran before `curr = curr->next()`, the function would read the link of a freed node. That is undefined behaviour, and valgrind reports an invalid read. The function also never deletes the value by hand, because `~Node()` does it, and a second delete would be a double free.

After the loop:

```
    headNode = NULL;
    size = 0;
```

- Every node is deleted, but `headNode` still holds the address of the first deleted node. The assignment `headNode = NULL` removes this dangling pointer. A dangling pointer is a pointer to memory that was freed.
- Without this line, the print function and the destructor would read freed nodes. The destructor calls `clear()` again, and the second call would delete the same nodes twice.
- `size = 0` makes `size` equal to the number of nodes again. The list is empty.
- These two lines let the list work again. `prepend` can add a node to a list with `headNode` `NULL`.

Memory: the function frees every node and every value. The destructor `~List()` calls `clear()`, so the destructor frees a full list with the same code. A list that was cleared has no nodes, so the destructor does nothing more.

Trace

Call from `main.cpp`: `numbers.clear();`. Before the call, the list is `1->2->3->4` and `size` is 4.

Iteration	`curr` before	`temp`	`curr` after `curr = curr->next()`	Node deleted
1	node 1	node 1	node 2	node 1 and its value
2	node 2	node 2	node 3	node 2 and its value
3	node 3	node 3	node 4	node 3 and its value
4	node 4	node 4	`NULL`	node 4 and its value

The loop test `curr != NULL` is false after iteration 4. The function then sets `headNode` to `NULL` and `size` to 0. The print function shows `Empty List`. The later calls `prepend(7)` and `prepend(8)` build the new list `8->7`.

The program output is:

```
1->2->3->4 (size 4)
Empty List (size 0)
8->7 (size 2)
```

Common mistakes

- Deleting only the head node, for example `delete headNode;`. The other nodes are never freed. The output is still correct, but valgrind reports "definitely lost" for the other nodes.
- Writing `delete curr;` and then `curr = curr->next();`. The function reads the link of a freed node. That is undefined behaviour, and valgrind reports "Invalid read of size 8". In the test, the given `~Node()` had already set that link to `NULL`, so the loop stopped after the first node, and valgrind also reported the other nodes as "definitely lost".
- Leaving out `headNode = NULL`. The list prints freed nodes after `clear()`. In the main test, the second line is `1 (size 0)` and not `Empty List (size 0)`. The destructor then calls `clear()` again on the same freed nodes. valgrind reports "Invalid read of size 8" and "Invalid free() / delete / delete[] / realloc()".
- Leaving out `size = 0`. The list prints `Empty List`, but `getSize()` still returns the old number. In the main test, the second line is `Empty List (size 4)`, and the expected output shows `(size 0)`.
- Deleting the value as well as the node. `~Node()` already deletes the value, so the value is deleted twice. valgrind reports "Invalid free() / delete / delete[] / realloc()".

B2-10. Linked lists: `Company::removeDepartment` (ownership when removing from a list) (100)

File: B_2.cpp

```
#include "Company.h"

// Removes the first department (in list order) whose name is name.
// The company owns its departments, so the Department object is deleted too.
bool Company::removeDepartment(string name) {
    Node<Department*>* curr = departments.head();
    while (curr != NULL) {
        Department* department = curr->getData();
        if (department->getName() == name) {
            // Unlink and delete the node first (curr is not used after this),
            // then delete the Department that the pointer points to.
            departments.remove(department);
            delete department;
            return true;
        }
        curr = curr->next();
    }
    return false;
}
```

Expected output of the given main.cpp

```
Company with 3 departments
Development (cost R25000)
Team:
  Lebo Nkosi, 25, F, ID: none
HR (cost R24000)
Team:
  Anna Botha, 41, F, ID: none
  Sipho Dlamini, 34, M, ID: none
Sales (cost R15000)
Team:
  Thandi Mokoena, 29, F, ID: none
Total cost: R64000
removeDepartment(HR): true
removeDepartment(Finance): false
Company with 2 departments
Development (cost R25000)
Team:
  Lebo Nkosi, 25, F, ID: none
Sales (cost R15000)
Team:
  Thandi Mokoena, 29, F, ID: none
Total cost: R40000
```

What the function must do

The function `removeDepartment` finds the first department whose name equals `name`. The function then removes its node from the list `departments` and also deletes the `Department` object. If a department is removed, the function returns `true`. If the company has no departments, or no department has that name, the function changes nothing and returns `false`.

Two objects are involved. The list node holds a pointer. The `Department` object is a separate object on the heap. The function must free both.

How the solution works

A pointer is a variable that holds the address of an object. The operator `->` uses a pointer to reach a member. `NULL` means the pointer points to nothing. `List<Department*>` is a list whose values are `Department*` pointers. For each node, `getData()` returns that pointer.

The question gives these ownership rules:

- `~Company()` deletes every `Department` that is still in the list.

- `List<Department*>` does not delete the `Department` objects. `~Node()` deletes only that stored pointer, not the object it points to.
- `List<T>::remove(T value)` is given. It unlinks and deletes the first node whose data equals `value`. For `Department*`, it compares the pointers.

The search loop starts here:

```
    Node<Department*>* curr = departments.head();
    while (curr != NULL) {
```

- `departments.head()` returns the first node, or `NULL` for an empty list.
- `curr` is the node that the loop checks.
- The condition `curr != NULL` stops the loop after the last node. For a company with no departments, the loop body never runs and the function returns `false`.

The body of the loop:

```
        Department* department = curr->getData();
        if (department->getName() == name) {
```

- `department` is a copy of the pointer in the node. It points to the `Department` object.
- `getName()` is a virtual function. It returns `"Sales"`, `"HR"`, `"Marketing"` or `"Development"` for the matching subclass.
- The test `department->getName() == name` compares two strings. The comparison is case sensitive, so `"hr"` does not match `"HR"`.
- The loop runs from the head, so the first match in list order is the department that the function removes.

The removal:

```
            departments.remove(department);
            delete department;
            return true;
```

1. `departments.remove(department)` finds the node that holds this pointer, links the previous node around it and deletes the node. The call also lowers `size`. The `Department` object is still alive after this call.
2. `delete department` then frees the `Department` object. The destructor is virtual, so the destructor of the subclass runs. The department deletes its teams, and each team deletes its employees.
3. `return true` ends the function at once.

The order is intentional. The list is fixed first, so no node holds a pointer to a freed `Department`, not even for a moment.

The `return true` is needed for a second reason. The node `curr` was deleted by `remove`. After that call, the loop must not run `curr = curr->next()`, because that would read a freed node. The comment in the code says the same: "curr is not used after this".

The loop ends without a match when the next lines run:

```
        curr = curr->next();
    }
    return false;
```

- `curr = curr->next()` moves to the following node. It runs only if the current department did not match.
- `return false` runs after the last node, or for an empty list. The function changed nothing.

Memory: a successful call frees the list node, the `Department`, its teams and its employees. When the company is deleted later, `~Company()` deletes only the departments that remain in the list. The removed department is no longer in the list, so `~Company()` does not delete it a second time.

Trace

Call from `main.cpp`: `tryRemove(yyra, "HR");`, which runs `company.removeDepartment(name)`. Departments were added with `prepend`, so the list order is the reverse of the add order: `Development->HR->Sales`.

Step	curr points to	department->getName()	Test == "HR"	Action
Setup	Development node			`curr = departments.head()`.
Iteration 1	Development node	`Development`	false	`curr` moves to the HR node.
Iteration 2	HR node	`HR`	true	`departments.remove(department)` links the Development node to the Sales node and deletes the HR node.
After `remove`	(deleted)			`delete department` frees the HR `Department`, its team and its two employees.
End				The function returns `true`.

The call `tryRemove(yyra, "Finance");` checks three nodes, finds no match, reaches `NULL` and returns `false`.

The part of the program output for the removal is:

```
removeDepartment(HR): true
removeDepartment(Finance): false
Company with 2 departments
Development (cost R25000)
Team:
  Lebo Nkosi, 25, F, ID: none
Sales (cost R15000)
Team:
  Thandi Mokoena, 29, F, ID: none
Total cost: R40000
```

Common mistakes

- Calling `delete department;` without `departments.remove(department)`. The list keeps a node that holds a pointer to freed memory. Printing the company then reads a freed object. `~Company()` later deletes the same pointer again, which is a double delete. In the main test, the output still says `Company with 3 departments` and prints the freed HR department. valgrind reports "Invalid read of size 8" and "Invalid free() / delete / delete[] / realloc()".
- Calling `departments.remove(department)` without `delete department`. The node is gone, but the `Department` object, its teams and its employees are never freed. The output is still correct, but valgrind reports "definitely lost". The question says `List<Department*>` does not delete the departments.
- Not returning after the removal, so that the loop runs `curr = curr->next()`. The node `curr` was deleted by `remove`. The program reads a freed node, and valgrind reports "Invalid read of size 8". The output is still correct, so only valgrind shows this mistake.
- Using `while (curr->next() != NULL)` as the loop condition. The last node is never checked, so `removeDepartment("Sales")` fails when `Sales` is the tail. For an empty company, `curr` is `NULL`, and the loop test reads `curr->next()` through `NULL`. The main test still passes. The edge test starts with an empty company: valgrind reports "Invalid read of size 8" and the program stops with a segmentation fault (SIGSEGV).
- Deleting the node with `delete curr;` and then fixing nothing else. The previous node still links to the freed node, and `size` is not lowered. The list is broken. In the main test, the company still prints 3 departments, valgrind reports "Invalid read of size 8" when the freed node is printed, and "Invalid free() / delete / delete[] / realloc()" when the list deletes the freed node again. The given `remove` already does the unlinking and the `size` change, so the function should call it.

B2-11. Linked lists: `List<T>` copy constructor (100)

File: B_2.cpp

```
#ifndef B_2_CPP
#define B_2_CPP

#include "List.h"

// Copy constructor: builds a deep copy of other, in the same order.
template <class T>
List<T>::List(const List<T>& other) : headNode(NULL), size(0) {
    Node<T>* tail = NULL;              // last node of the new list
    Node<T>* curr = other.headNode;
    while (curr != NULL) {
        // A new node with a new copy of the data.
        Node<T>* newNode = new Node<T>(new T(curr->getData()));
        if (tail == NULL) {
            headNode = newNode;        // first node of the copy
        } else {
            tail->next() = newNode;    // link after the current last node
        }
        tail = newNode;
        size++;
        curr = curr->next();
    }
}

#endif
```

Expected output of the given main.cpp

```
original: 1->2->3 (size 3)
copy:     1->2->3 (size 3)
original: 0->1->2->3 (size 4)
copy:     9->1->2->3 (size 4)
```

What the function must do

The copy constructor builds a new list from an existing list `other`. The new list is a deep copy. A deep copy has its own new nodes, and each new node holds its own new copy of the value. The new list has the same values in the same order and the same `size`. The two lists share nothing, so a change to one list does not change the other. Copying an empty list gives an empty list.

How the solution works

A pointer is a variable that holds the address of an object. The operator `->` uses a pointer to reach a member. `NULL` means the pointer points to nothing. `T` is the template type. A constructor runs when an object is created. In `List<int> copy(original);`, the compiler calls the copy constructor with `original` as `other`. The compiler also calls it when a list is passed to a function by value. The type `const List<T>&` is a reference to a list that the constructor must not change.

The constructor starts here:

```
template <class T>
List<T>::List(const List<T>& other) : headNode(NULL), size(0) {
```

- The part after the colon is the initializer list. It sets `headNode` to `NULL` and `size` to 0 before the body runs.
- A new object has no values in its members yet. Without these initial values, `headNode` and `size` hold unknown data. The body then adds to a wrong `size`, and an empty copy has an unknown head.
- With these values, the new list is a valid empty list. The body only needs to add nodes.

Two pointers support the loop:

```
    Node<T>* tail = NULL;              // last node of the new list
    Node<T>* curr = other.headNode;
```

- `tail` points to the last node of the new list. The new list is empty, so `tail` is `NULL`.
- `curr` points to the node of `other` that the loop copies. It starts at the first node of `other`.

- The constructor may read `other.headNode` directly, because `other` has the same class. The constructor does not change `other`.
- The function keeps `tail` so that it does not need to walk the new list again for each new node.

The loop:

```
    while (curr != NULL) {
```

- The condition `curr != NULL` is true while `other` has a node left to copy.
- For an empty `other`, `curr` is `NULL` at the start. The body never runs, and the new list stays empty with `size` 0.

The new node:

```
        Node<T>* newNode = new Node<T>(new T(curr->getData()));
```

- `curr->getData()` returns the value of the old node.
- `new T(curr->getData())` allocates a new value and copies the old value into it.
- `new Node<T>(...)` allocates a new node that owns this new value.
- The old node and the new node therefore share no memory. This is the deep copy.

The link step has two cases:

```
        if (tail == NULL) {
            headNode = newNode;        // first node of the copy
        } else {
            tail->next() = newNode;    // link after the current last node
        }
```

- The test `tail == NULL` is true only for the first node. The copy has no last node yet, so `headNode` must point to the new node.
- For later nodes, `tail` is the current last node. `tail->next() = newNode` links the new node after it. The reference returned by `next()` lets the assignment change the real link.
- Appending in this way keeps the order of `other`. The new node's own link is `NULL` from the `Node` constructor, so the new last node ends the chain.

The updates at the end of each pass:

```
        tail = newNode;
        size++;
        curr = curr->next();
```

- `tail = newNode` makes the new node the last node. The next pass links after it.
- `size++` counts the new node. The copy ends with the same `size` as `other`.
- `curr = curr->next()` moves to the next node of `other`.

Memory: each pass allocates one node and one value. The destructor of the new list frees them. The destructor of `other` frees the old ones. The two destructors free different memory, so valgrind shows no error.

Trace

Call from `main.cpp`: `List<int> copy(original);`. The list `original` is `1->2->3`.

Step	curr (in original)	tail (in copy)	Action	copy after the step	size
Setup	node 1	`NULL`	`headNode` is `NULL`, `size` is 0.	empty	0
Pass 1	node 1	`NULL`	New node with 1. `tail == NULL`, so `headNode` points to it.	`1`	1
Pass 2	node 2	node 1 (copy)	New node with 2. `tail->next()` points to it.	`1->2`	2
Pass 3	node 3	node 2 (copy)	New node with 3. `tail->next()` points to it.	`1->2->3`	3

Step	`curr` (in `original`)	`tail` (in `copy`)	Action	`copy` after the step	`size`
End	`NULL`	node 3 (copy)	The loop test is false.	`1->2->3`	3

After the copy, `original.prepend(0)` changes only `original`, and `copy.prepend(9)` changes only `copy`. The two lists use different nodes.

The program output is:

```
original: 1->2->3 (size 3)
copy:     1->2->3 (size 3)
original: 0->1->2->3 (size 4)
copy:     9->1->2->3 (size 4)
```

Common mistakes

- Copying only the pointer, for example `headNode = other.headNode; size = other.size;`. This is a shallow copy. Both lists use the same nodes. When both destructors run, the same nodes are deleted twice. The main test still prints the expected lines, but valgrind reports "Invalid read of size 8" and "Invalid free() / delete / delete[] / realloc()". The edge test also checks `one.head() != oneCopy.head()` and prints `no`, but it expects `yes`.
- Building the copy with `prepend` inside the loop. The copy is then in reverse order, for example `3->2->1`. The output is wrong.
- Forgetting `size++`. The values are copied, but the printed size is wrong. The main test prints `copy: 1->2->3 (size 0)`.
- Leaving out the initializer list `headNode(NULL), size(0)`. The values of `headNode` and `size` are then unknown. Using them is undefined behaviour. In the main test, the copy printed a wrong size (a large number such as 73731), and valgrind reported "Conditional jump or move depends on uninitialised value(s)". A copy of an empty list also keeps the unknown `headNode`, so printing or destroying it uses an unknown address. The edge test copies an empty list.
- Not updating `tail`. Then `tail` stays `NULL`, so every pass sets `headNode = newNode`. The copy keeps only the last node, and the earlier nodes are lost. The main test prints `copy: 3 (size 3)`, and valgrind reports "definitely lost".
- Looping with `while (curr->next() != NULL)`. The loop skips the last value of `other`, so the copy is one node short. For an empty `other`, `curr` is `NULL`, and the loop test reads `curr->next()` through `NULL`. The edge test starts by copying an empty list: valgrind reports "Invalid read of size 8" and the program stops with a segmentation fault (SIGSEGV).

B_3 Template specialization: solutions

B3-01. Template specialization: `printItem<Department*>` (100)

File: B_3.cpp

```
#include "Templates.h"

// For a Department pointer, print the Department it points to, not the address.
template <>
void printItem<Department*>(Department* const& item) {
    if (item == NULL) {
        cout << "No department" << endl;
        return;
    }
    cout << *item;    // Department's operator<< already ends its lines
}
```

Expected output of the given main.cpp

```
42
3.5
Yyra HRMS
Sales (cost R15000)
Team:
  Thandi Mokoena, 29, F, ID: none
No department
```

What the function must do

The general template `printItem` prints a value with `cout << item << endl;`. For a `Department*`, that statement prints an address. The specialization is a special version of the template for the type `Department*`. It prints the department that the pointer points to. If the pointer is `NULL`, it prints `No department`.

How the solution works

A **function template** is a recipe for functions. In `template <class T> void printItem(const T& item)`, the name `T` is a template parameter: a placeholder for a type. When the program calls `printItem(42)`, the compiler sets `T` to `int` and builds a version for `int`.

A **specialization** is a version of the template for one exact type. When the program calls `printItem` with a `Department*`, the compiler uses the specialization. For all other types, it uses the general template.

```
template <>
void printItem<Department*>(Department* const& item) {
```

The line `template <>` tells the compiler that the function is a specialization and that no type placeholder remains. The text `<Department*>` after the function name gives the concrete type for `T`.

The parameter type comes from `const T&` with `T` equal to `Department*`. A `const` that is written after the star applies to the pointer. The result is `Department* const&`: a reference to a pointer that cannot change. The type is not `const Department*&`. The definition must match the declaration in `Templates.h` exactly. A different parameter type does not match any template declaration, and the compiler reports an error.

The declaration in `Templates.h` comes after the general template and before any call. For this reason, `main.cpp` calls the specialization and not the general template. `B_3.cpp` holds the definition.

```
    if (item == NULL) {
        cout << "No department" << endl;
        return;
    }
```

`item` is a pointer. The condition `item == NULL` checks whether the pointer points at no object. A `NULL` pointer has no department to print. The next line would use `*item`. Using `*item` on a `NULL` pointer is undefined behaviour: the C++ rules do not say what

happens. In this program, it stops with a segmentation fault. This check must come before the print line. If the condition is true, the function prints `No department` with a newline and leaves the function with `return`. The `return` prevents the function from reaching the print line below.

```
    cout << *item;    // Department's operator<< already ends its lines
}
```

`*item` is the department that the pointer points to. The operator `<<` is **overloaded** for `Department`: the class defines its own version of `<<`. This version prints `<name> (cost R<cost>)` and a newline, and then each team. The functions `getName()` and `costToCompany()` are virtual, so a `Sales` object prints `Sales`, even though the pointer type is `Department*`.

The function does not add `endl`. The `Department` operator already ends every line, so an extra `endl` would print a blank line.

The function allocates and frees no memory. The caller owns the department and deletes it.

Trace

The calls come from `main.cpp`. The first three calls use the general template. The last two calls use the specialization.

Call	Type `T`	Template used	Check `item == NULL`	Printed
`printItem(42)`	`int`	general	not applicable	`42`
`printItem(3.5)`	`double`	general	not applicable	`3.5`
`printItem(string("Yyra HRMS"))`	`string`	general	not applicable	`Yyra HRMS`
`printItem(sales)`	`Department*`	specialization	false, because `sales` points at a `Sales` object	the department, with its team
`printItem(none)`	`Department*`	specialization	true, because `none` is `NULL`	`No department`

In the fourth call, `cout << *item;` calls the `Department` operator. `getName()` gives `Sales`, and `costToCompany()` gives 1 employee times 15000. Then the team operator prints `Team:` and one line for each employee.

The output of `main.cpp` is:

```
42
3.5
Yyra HRMS
Sales (cost R15000)
Team:
  Thandi Mokoena, 29, F, ID: none
No department
```

Common mistakes

- **Write `const Department*& item`.** The parameter type does not match the declaration in `Templates.h`. The compiler reports that the template-id does not match any template declaration, and the mark is 0.
- **Omit the `NULL` check.** The call `printItem(none)` then runs `*item` on a `NULL` pointer. This is undefined behaviour. In this program, valgrind reports `Invalid read of size 8`, and the program stops with a segmentation fault (signal 11) before it prints `No department`. The edge test also passes a `NULL` entry from an array.
- **Print the pointer, as in `cout << item << endl;`.** The program prints an address, and the output differs from the expected output.
- **Add `endl` after `cout << *item;`.** The `Department` operator already ends every line. The extra `endl` prints a blank line, and the output differs from the expected output.
- **Write the first line by hand with a fixed name.** For example, `cout << "Department (cost R" << item->costToCompany() << ")" << endl;`. In `main.cpp`, this prints `Department (cost R15000)` and not `Sales (cost R15000)`, and the team lines are missing, so the output differs from the expected output. Use `cout << *item;`. The `Department` operator calls the virtual functions, which choose the name and the cost, and it also prints the teams. The edge test prints `HR`, `Department` and `Development` objects through the same function.

B3-02. Template specialization: `largest<Employee>` (100)

File: B_3.cpp

```
#include "Templates.h"

// For employees, the "largest" one is the older one. Same age: return b,
// like the general template does when the values are equal.
template <>
Employee largest<Employee>(Employee a, Employee b) {
    if (a.getAge() > b.getAge()) {
        return a;
    }
    return b;
}
```

Expected output of the given main.cpp

```
7
2.5
pear
Sipho Dlamini, 34, M, ID: none
Sipho Dlamini, 34, M, ID: none
```

What the function must do

The general template `largest` returns the larger of two values, found with the `>` operator. The class `Employee` has no `>` operator, so the general template cannot compile for employees. The specialization for `Employee` must compare the ages of the two employees. It returns the older employee, and it returns `b` when the ages are equal.

How the solution works

A **function template** is a recipe for functions. In `template <class T> T largest(T a, T b)`, the name `T` is a template parameter: a placeholder for a type. For `largest(3, 7)`, the compiler sets `T` to `int` and builds a version for `int`. In that version, the line `a > b` compares two integers.

For `Employee`, the line `a > b` has no meaning, because the class does not define `>`. A **specialization** is a version of the template that you write for one exact type. When the program calls `largest` with two `Employee` objects, the compiler uses the specialization and not the general template.

```
template <>
Employee largest<Employee>(Employee a, Employee b) {
```

The line `template <>` tells the compiler that this function is a specialization. The text `<Employee>` after the function name gives the type for `T`. The definition must match the declaration in `Templates.h` exactly: the same return type, the same name and the same parameter types. The declaration appears in `Templates.h` after the general template and before any call, so `main.cpp` calls this specialization.

The parameters `a` and `b` are passed by value. C++ makes a copy of each argument with the `Employee` copy constructor. The function returns an `Employee` by value, which makes another copy. The copy keeps every attribute, including `idNum`. The function does not use `new`, so it allocates nothing and frees nothing.

```
    if (a.getAge() > b.getAge()) {
        return a;
    }
    return b;
```

The condition `a.getAge() > b.getAge()` compares the two ages, which are `int` values. The function uses `getAge()` because the attribute `age` is private. The comparison `>` works on `int`, so it compiles.

If the age of `a` is greater, the function returns `a`. In every other case, the function reaches `return b;`. This includes the case where the ages are equal. The general template has the same rule: it returns `b` when the values are equal.

The comparison must use `>` and not `>=`. With `>=`, equal ages would return `a`, and the result would differ from the general template.

Trace

The calls come from `main.cpp`. The first three calls use the general template. The last two calls use the specialization.

Call	Template used	Comparison	Result
`largest(3, 7)`	general, `T` is `int`	`3 > 7` is false	`7`
`largest(2.5, 1.5)`	general, `T` is `double`	`2.5 > 1.5` is true	`2.5`
`largest(string("apple"), string("pear"))`	general, `T` is `string`	`"apple" > "pear"` is false	`pear`
`largest(thandi, sipho)`	specialization	`29 > 34` is false, so the function returns `b`	Sipho
`largest(sipho, thandi)`	specialization	`34 > 29` is true, so the function returns `a`	Sipho

In both of the last two calls, the older employee is Sipho. The function returns a copy of Sipho.

The output of `main.cpp` is:

```
7
2.5
pear
Sipho Dlamini, 34, M, ID: none
Sipho Dlamini, 34, M, ID: none
```

Common mistakes

- **Write `a > b` for the employees.** The class `Employee` has no `operator>`. The compiler reports an error, and the mark is 0. The function must compare `a.getAge()` and `b.getAge()`.
- **Use `>=` instead of `>`.** When the ages are equal, the function returns `a`. The first edge test (`Anna` and `Piet`, both aged 41) expects `Piet` for `largest(anna, piet)`, so the output differs.
- **Return the younger employee.** If the function uses `<`, or swaps the two returns, it returns the younger employee whenever the ages differ. In `main.cpp`, both lines then show `Thandi Mokoena, 29, F, ID: none` and not Sipho. With swapped returns, equal ages also give `a`, so the first two edge lines are wrong too.
- **Change the header line.** If the definition does not match the declaration in `Templates.h` (for example, `Employee& a` instead of `Employee a`), the compiler reports that the specialization does not match any template declaration, and the mark is 0.

B3-03. Template specialization: `isEqual<Department*>` (100)

File: B_3.cpp

```
#include "Templates.h"

// Two departments are equal when they cost the company the same amount.
template <>
bool isEqual<Department*>(Department* a, Department* b) {
    if (a == NULL && b == NULL) {
        return true;
    }
    if (a == NULL || b == NULL) {
        return false;
    }
    return a->costToCompany() == b->costToCompany();
}
```

Expected output of the given main.cpp

```
5 and 5: equal
5 and 6: not equal
HR and HR: equal
Marketing R36000, HR R36000, Sales R15000
Marketing and HR: equal
Sales and HR: not equal
```

What the function must do

The general template `isEqual` compares two values with `==`. For two `Department*` pointers, `==` compares addresses, so two different objects are never equal. The specialization for `Department*` must treat two departments as equal when their `costToCompany()` values are equal. Two `NULL` pointers are equal. One `NULL` pointer and one department are not equal.

How the solution works

A **function template** is a recipe for functions. In `template <class T> bool isEqual(T a, T b)`, the name `T` is a template parameter: a placeholder for a type. For `isEqual(5, 6)`, the compiler sets `T` to `int` and builds a version for `int`.

A **specialization** is a version of the template that you write for one exact type. When the program calls `isEqual` with two `Department*` values, the compiler uses the specialization and not the general template.

```
template <>
bool isEqual<Department*>(Department* a, Department* b) {
```

The line `template <>` tells the compiler that the function is a specialization. The text `<Department*>` after the function name gives the type for `T`. The parameters `a` and `b` are pointers. A **pointer** holds the address of an object, or `NULL` when it points at no object. The definition must match the declaration in `Templates.h` exactly. That declaration comes after the general template and before any call, so `main.cpp` calls this specialization.

```
    if (a == NULL && b == NULL) {
        return true;
    }
```

The operator `&&` means "and". The condition is true only when both pointers are `NULL`. The function returns `true` for this case, because two missing departments are equal by the rule of the question. This check must come first. The next check uses `||` ("or"), which is true when one pointer is `NULL`. If the order were reversed, two `NULL` pointers would reach the "or" check and return `false`.

```
    if (a == NULL || b == NULL) {
        return false;
    }
```

At this line, the first check has already handled the case where both pointers are `NULL`. If the condition is true here, exactly one pointer is `NULL`. A missing department and a real department are not equal, so the function returns `false`. This check also protects the next line. The next line calls a function through both pointers. A call through a `NULL` pointer is undefined behaviour: the C++ rules do not say what happens. In this program, it stops with a segmentation fault.

```
    return a->costToCompany() == b->costToCompany();
```

At this line, both pointers point at real departments. The operator `->` calls a member function through a pointer. The function `costToCompany()` is virtual. C++ therefore calls the version that belongs to the real type of the object, even though the pointer type is `Department*`. A `Sales` object uses the rate of 15000 per employee, an `HR` object uses 12000, a `Marketing` object uses 18000 and a `Development` object uses 25000. The base `Department` returns 0.

The operator `==` compares the two costs. The result of the comparison (`true` or `false`) is the result of the function. The type of department does not matter, only the cost. The function does not delete or change the departments, so it allocates and frees no memory.

Trace

The calls come from `main.cpp`. The first three calls use the general template. The last two calls use the specialization.

Call	Template used	Check `a == NULL && b == NULL`	Check `a == NULL || b == NULL`	Costs compared	Result
`isEqual(5, 5)`	general	not applicable	not applicable	`5 == 5`	true
`isEqual(5, 6)`	general	not applicable	not applicable	`5 == 6`	false
`isEqual(string("HR"), string("HR"))`	general	not applicable	not applicable	`"HR" == "HR"`	true
`isEqual(marketing, hr)`	specialization	false	false	`2 * 18000 = 36000` and `3 * 12000 = 36000`	true
`isEqual(sales, hr)`	specialization	false	false	`1 * 15000 = 15000` and `36000`	false

The function `compare` prints `equal` for `true` and `not equal` for `false`.

The output of `main.cpp` is:

```
5 and 5: equal
5 and 6: not equal
HR and HR: equal
Marketing R36000, HR R36000, Sales R15000
Marketing and HR: equal
Sales and HR: not equal
```

Common mistakes

- **Compare the pointers, as in `return a == b;`.** This compares addresses. `Marketing and HR` are two different objects, so the output shows `not equal`, and the expected output shows `equal`.
- **Omit the `NULL` checks.** The edge tests pass `NULL` pointers. A call such as `a->costToCompany()` on a `NULL` pointer is undefined behaviour. In the edge test, the first call `isEqual(none, none)` already fails: valgrind reports `Invalid read of size 8`, and the program stops with a segmentation fault (signal 11) before it prints any line.
- **Check for one `NULL` before both `NULL`.** The call `isEqual(none, none)` then returns `false`, and the expected output is `equal`.
- **Compare names or types.** The question compares costs only. A function that compares `getName()` returns `false` for `Marketing and HR`, and the expected output is `equal`.

B3-04. Template specialization: `List<Department*>::printAll` (100)

File: B_3.cpp

```
#include "Department.h"
#include "List.h"

// For a list of Department pointers, print each Department, not its address.
template <>
void List<Department*>::printAll() {
    if (headNode == NULL) {
        cout << "Empty List" << endl;
        return;
    }
    Node<Department*>* curr = headNode;
    int index = 0;
    while (curr != NULL) {
        Department* department = curr->getData();
        if (department == NULL) {
            cout << index << ": No department" << endl;
        } else {
            cout << index << ": " << *department;   // operator<< ends its own lines
        }
        curr = curr->next();
        index++;
    }
}
```

Expected output of the given main.cpp

```
0: 10
1: 20
2: 30
0: HR (cost R24000)
Team:
  Anna Botha, 41, F, ID: none
  Sipho Dlamini, 34, M, ID: none
1: Sales (cost R15000)
Team:
  Thandi Mokoena, 29, F, ID: none
```

What the function must do

The general `List<T>::printAll()` prints each value as `<index>: <value>`. For `List<Department*>`, the value is a pointer, and the general version would print an address. The specialization must print the department that each pointer points to. It must also handle an empty list and a `NULL` entry.

How the solution works

`List<T>` is a **class template**. The name `T` is a template parameter: a placeholder for a type such as `int`, `string` or `Department*`. The compiler builds a separate class for each type that the program uses.

A **member specialization** replaces one member function of the template for one exact type. Here, `printAll()` has its own version for `T = Department*`. For all other types, the program uses the general `printAll()`.

```
template <>
void List<Department*>::printAll() {
```

The line `template <>` tells the compiler that this function is a specialization. The concrete type goes after the class name: `List<Department*>`. The type does not go after the function name. The text `template <class T>` does not appear here, because `T` is already fixed to `Department*`. Inside this function, `headNode` has the type `Node<Department*>*`. The definition must match the declaration at the bottom of `List.h`.

The file starts with `#include "Department.h"` and `#include "List.h"`. `List.h` only names the class `Department`. The function uses `*department`, which needs the full class, so the file includes `Department.h`.

```
    if (headNode == NULL) {
        cout << "Empty List" << endl;
```

```
        return;
    }
```

`headNode` is a pointer to the first node. It is `NULL` when the list is empty. If the condition is true, the function prints `Empty List` and leaves the function with `return`. Without this check, the loop below would run zero times and print nothing. The question requires the text `Empty List`, as in the general version.

```
    Node<Department*>* curr = headNode;
    int index = 0;
```

`curr` points at the node that the loop visits. It starts at the first node. `index` counts the position of the node, from 0. Each node holds one `Department*`.

```
    while (curr != NULL) {
```

The loop continues while `curr` points at a node. After the last node, `curr` becomes `NULL`, and the loop ends. The loop does not change the list and does not delete a node.

```
        Department* department = curr->getData();
```

`getData()` returns the stored value, which is a `Department*`. The line copies the pointer into the local variable `department`. This copies the address only. It does not copy the department and does not take ownership. The list does not own the departments in this question. `main` deletes them.

```
        if (department == NULL) {
            cout << index << ": No department" << endl;
        } else {
            cout << index << ": " << *department;   // operator<< ends its own lines
        }
```

The condition `department == NULL` checks for a `NULL` entry. The list can hold `NULL` pointers, because `prepend(NULL)` is allowed. For a `NULL` entry, the expression `*department` would follow a `NULL` pointer. This is undefined behaviour: the C++ rules do not say what happens. In this program, it stops with a segmentation fault. So the check must come before it. For a `NULL` entry, the function prints `<index>: No department` and a newline.

For a real department, `*department` is the department that the pointer points to. The operator `<<` is **overloaded** for `Department`: the class defines its own version of `<<`. This version prints `<name> (cost R<cost>)` and a newline, and then each team. The functions `getName()` and `costToCompany()` are virtual, so a `Sales` object prints `Sales`. The Department operator already ends every line, so the function does not add `endl`.

```
        curr = curr->next();
        index++;
    }
}
```

`next()` returns the next node. The assignment moves `curr` forward. `index++` adds 1 to the position. If the function did not move `curr`, the loop would never end.

Trace

The call is `departments.printAll();` from `main.cpp`. The list was built with `departments.prepend(sales)` and then `departments.prepend(hr)`. The function `prepend` adds at the front, so the list is `hr` then `sales`. HR has 2 employees (2 times 12000 is 24000). Sales has 1 employee (1 times 15000 is 15000).

Step	`curr`	`index`	Check	Printed
Start	first node	0	`headNode == NULL` is false	nothing
Round 1	node with `hr`	0	`department == NULL` is false	`0:` and the HR department, with its team
Move	node with `sales`	1	none	nothing
Round 2	node with `sales`	1	`department == NULL` is false	`1:` and the Sales department, with its team
Move	`NULL`	2	the loop condition `curr != NULL` is false	nothing

The call `numbers.printAll()` on `List<int>` uses the general version. It prints `0: 10`, `1: 20` and `2: 30`.

The output of `main.cpp` is:

```
0: 10
1: 20
2: 30
0: HR (cost R24000)
Team:
  Anna Botha, 41, F, ID: none
  Sipho Dlamini, 34, M, ID: none
1: Sales (cost R15000)
Team:
  Thandi Mokoena, 29, F, ID: none
```

Common mistakes

- **Write the type after the function name**, as in `List<Department*>::printAll<Department*>()`. The compiler rejects this form with `error: expected initializer before '<' token`. The program does not compile, and the mark is 0.
- **Write `template <class T>` instead of `template <>`.** The definition does not match any declaration. The compiler reports `error: no declaration matches 'void List<Department*>::printAll()'`.
- **Use `Node<T>*` inside the specialization.** The name `T` is not defined in a full specialization. The compiler reports `error: 'T' was not declared in this scope`, and the code does not compile. The function must use `Node<Department*>*`.
- **Omit the `NULL` check.** The edge test has a `NULL` entry. The code `*department` on a `NULL` pointer is undefined behaviour. In the edge test, valgrind reports `Invalid read of size 8` at the `NULL` entry, and the program stops with a segmentation fault (signal 11). The lines after `0: Development ...` are never printed.
- **Print the pointer, as in `cout << curr->getData()`.** The program prints an address, and the output differs from the expected output.
- **Delete the department inside the function.** `main` deletes each department, so each department is deleted twice. In `main.cpp`, valgrind reports `Invalid read` and `Invalid free() / delete / delete[] / realloc()` errors when `main` deletes the departments again. In the edge test, `dev` is printed a second time after it was deleted, and the output shows `0: Department (cost R0)` instead of `0: Development (cost R25000)`.
- **Add `endl` after `*department`.** The Department operator already ends every line, so the output gets an extra blank line.

B3-05. Template specialization: `List<Department*>::clear` (100)

File: B_3.cpp

```
#include "Department.h"
#include "List.h"

// For a list of Department pointers, the list owns the Departments:
// delete each Department, then its node.
template <>
void List<Department*>::clear() {
    Node<Department*>* curr = headNode;
    while (curr != NULL) {
        Node<Department*>* temp = curr;
        curr = curr->next();
        delete temp->getData();   // the Department (deleting NULL does nothing)
        delete temp;              // the node, which deletes its copy of the pointer
    }
    headNode = NULL;
    size = 0;
}
```

Expected output of the given main.cpp

```
1->2->3
Empty List (size 0)
Company with 2 departments
HR (cost R24000)
Team:
  Anna Botha, 41, F, ID: none
  Sipho Dlamini, 34, M, ID: none
Sales (cost R15000)
Team:
  Thandi Mokoena, 29, F, ID: none
Total cost: R39000
Spare departments: 2
Spare departments after clear: 0
```

What the function must do

The general `List<T>::clear()` deletes every node. For `List<Department*>`, the list must also delete the `Department` objects, because `Company` has no destructor and nothing else frees them. After the call, `headNode` is `NULL` and `size` is `0`. The destructor `~List()` calls `clear()`, so the specialization also runs when a `Company` is destroyed.

How the solution works

`List<T>` is a **class template**. The name `T` is a template parameter: a placeholder for a type. A **member specialization** replaces one member function of the template for one exact type. Here, `clear()` has its own version for `T = Department*`.

Each `Node<T>` holds `T* data`. For `List<Department*>`, `data` has the type `Department**`: a pointer to a `Department*`. `~Node()` deletes `data`. This frees only the pointer that the node allocated. It does not delete the `Department` that the pointer points to. The specialization must delete the `Department` itself.

```
template <>
void List<Department*>::clear() {
```

The line `template <>` tells the compiler that this function is a specialization. The concrete type goes after the class name: `List<Department*>`. The text `template <class T>` does not appear here, because `T` is fixed to `Department*`. The definition must match the declaration at the bottom of `List.h`. The file also includes `Department.h`, because `delete` on a `Department*` needs the full class.

```
    Node<Department*>* curr = headNode;
    while (curr != NULL) {
```

`curr` points at the node that the loop visits. It starts at the first node. The condition `curr != NULL` stops the loop after the last node. For an empty list, `headNode` is `NULL`, so the loop does not run, and the function does not use a `NULL` pointer. This makes `clear()` safe to call on an empty list and safe to call twice.

```
        Node<Department*>* temp = curr;
        curr = curr->next();
```

`temp` saves the current node. `next()` returns the next node, and the assignment moves `curr` forward. The function must move `curr` before it deletes the node. After `delete temp`, the function cannot read `next()` from that node.

```
        delete temp->getData();   // the Department (deleting NULL does nothing)
```

`getData()` returns the `Department*` that the node holds. The statement `delete` on this pointer runs the destructor `~Department()`, which is virtual. The destructor that runs belongs to the real type of the object, even through a `Department*`. The department deletes its teams, and each team deletes its employees. A `NULL` entry in the list is safe, because `delete` on a `NULL` pointer does nothing. The function needs no extra `if` for `NULL`.

This line must come before `delete temp;`. The function reads the data through the node. After the node is deleted, the function cannot call `getData()` on it.

```
        delete temp;              // the node, which deletes its copy of the pointer
    }
```

This line deletes the node. `~Node()` deletes `data`, which is the node's own copy of the pointer. Each node and each department is deleted exactly once.

```
    headNode = NULL;
    size = 0;
}
```

After the loop, `headNode` still holds the address of the first deleted node. The line `headNode = NULL;` removes this pointer to freed memory. The line `size = 0;` makes the size match the empty list. These two lines are necessary for two reasons. First, the list can be used again after `clear()`. Second, `~List()` calls `clear()` again when the list is destroyed. A `headNode` that still points at freed memory would make that second call delete the same nodes again.

Trace

The call is `spare.clear();` from `main.cpp`. The list `spare` holds `Development(2)` and then `Marketing(1)`, because `prepend` adds at the front. Neither department has teams. `D` is the `Development` department and `M` is the `Marketing` department.

Step	`curr`	`temp`	Action	Freed
Start	node 1 (holds `D`)	none	`curr != NULL` is true	nothing
Round 1	node 2 (holds `M`)	node 1	`curr = curr->next()`	nothing
	node 2	node 1	`delete temp->getData();`	`D`
	node 2	node 1	`delete temp;`	node 1 and its pointer copy
Round 2	`NULL`	node 2	`curr = curr->next()`	nothing
	`NULL`	node 2	`delete temp->getData();`	`M`
	`NULL`	node 2	`delete temp;`	node 2 and its pointer copy
End	`NULL`	node 2	`curr != NULL` is false; the loop ends	nothing
After	none	none	`headNode = NULL; size = 0;`	nothing

The same steps run for the `Company` object `yyra` when `main` ends. `~List()` calls `clear()`. The list holds `HR` and then `Sales`. Each `delete` on a department also frees its team and its employees.

The output of `main.cpp` is:

```
1->2->3
Empty List (size 0)
Company with 2 departments
HR (cost R24000)
Team:
  Anna Botha, 41, F, ID: none
  Sipho Dlamini, 34, M, ID: none
```

```
Sales (cost R15000)
Team:
  Thandi Mokoena, 29, F, ID: none
Total cost: R39000
Spare departments: 2
Spare departments after clear: 0
```

Common mistakes

- **Delete only the nodes.** If the function deletes `temp` but not `temp->getData()`, `~Node()` frees only the pointer copy. Every `Department`, `Team` and `Employee` stays in memory. The output is still correct, but valgrind reports `definitely lost` and `indirectly lost` blocks for the four departments and everything they own.
- **Read `next()` after `delete temp`.** The code `delete temp; curr = temp->next();` reads a deleted node. This is undefined behaviour. In `main.cpp`, valgrind reports `Invalid read of size 8`. The loop also stops after the first node, so valgrind reports leaks for the other nodes and departments.
- **Call `temp->getData()` after `delete temp`.** The node is gone, so the call reads freed memory. This is undefined behaviour. In `main.cpp`, valgrind reports `Invalid read of size 8`. Delete the department first, then the node.
- **Forget `headNode = NULL;` and `size = 0;`.** After `clear()`, `headNode` points at freed nodes. The call `clear()` in `~List()` then deletes the nodes and the departments a second time. Valgrind reports `Invalid read` and `Invalid free() / delete / delete[] / realloc()` errors. The size also stays wrong: the output shows `Spare departments after clear: 2` instead of `0`.
- **Write `template <class T>` or use `Node<T>*`.** With `template <class T>`, the compiler reports `error: no declaration matches 'void List<Department*>::clear()'`. With `Node<T>*` after `template <>`, it reports `error: 'T' was not declared in this scope`, because `T` is not defined in a full specialization. In both cases the code does not compile. The function must use `Node<Department*>*`.

B3-06. Template specialization: `totalCost<Department*>` (100)

File: B_3.cpp

```
#include "Templates.h"

// The total monthly cost of every department in the list. NULL entries are skipped.
template <>
float totalCost<Department*>(List<Department*>& list) {
    float total = 0;
    Node<Department*>* curr = list.head();
    while (curr != NULL) {
        Department* department = curr->getData();
        if (department != NULL) {
            total += department->costToCompany();   // virtual: the subclass rule is used
        }
        curr = curr->next();
    }
    return total;
}
```

Expected output of the given main.cpp

```
totalCost(numbers): 0
Company cost: R76000
totalCost(empty): 0
```

What the function must do

The general template `totalCost` takes a list and returns 0. The specialization for `Department*` must walk through a `List<Department*>` and add up the `costToCompany()` of each department. The function skips `NULL` entries. An empty list gives 0. The function changes nothing and deletes nothing.

How the solution works

A **function template** is a recipe for functions. In `template <class T> float totalCost(List<T>& list)`, the name `T` is a template parameter: a placeholder for a type. For a `List<int>`, the compiler sets `T` to `int` and builds a version that returns 0.

A **specialization** is a version of the template that you write for one exact type. When the program calls `totalCost` with a `List<Department*>`, the compiler uses the specialization. `Company::costToCompany()` calls `totalCost(departments)`, so it also uses the specialization.

```
template <>
float totalCost<Department*>(List<Department*>& list) {
```

The line `template <>` tells the compiler that the function is a specialization. The text `<Department*>` after the function name gives the type for `T`. The parameter `list` is a reference to a list (`&` after the type). A reference names the real list and does not copy it. The copy constructor of `List` is private, so the function cannot take the list by value. The definition must match the declaration in `Templates.h` exactly.

```
    float total = 0;
```

`total` holds the running sum. It starts at 0, so an empty list returns 0. Without this value, the sum would start from an undefined value.

```
    Node<Department*>* curr = list.head();
```

`totalCost` is not a member of `List`, so the function cannot use the private member `headNode`. The member function `head()` returns the first node, or `NULL` for an empty list. `curr` points at the node that the loop visits.

```
    while (curr != NULL) {
```

The loop continues while `curr` points at a node. After the last node, `curr` becomes `NULL`, and the loop ends. For an empty list, `head()` returns `NULL`, so the loop does not run, and the function returns 0.

```
        Department* department = curr->getData();
```

`getData()` returns the stored `Department*`. The line copies the pointer into the local variable `department`. This copies the address only. The function does not take ownership of the department.

```
        if (department != NULL) {
            total += department->costToCompany();   // virtual: the subclass rule is used
        }
```

The list can hold `NULL` pointers, because `prepend(NULL)` is allowed. The condition `department != NULL` skips these entries. A call through a `NULL` pointer is undefined behaviour: the C++ rules do not say what happens. In this program, it stops with a segmentation fault. So the check must come before the call. A `NULL` entry adds nothing to the total.

The operator `->` calls a member function through a pointer. The function `costToCompany()` is virtual. C++ therefore calls the version that belongs to the real type of the object, even though the pointer type is `Department*`. A `Sales` object uses 15000 per employee, an `HR` object uses 12000, a `Marketing` object uses 18000 and a `Development` object uses 25000. The base `Department` returns 0. The operator `+=` adds the cost to `total`.

```
        curr = curr->next();
    }
    return total;
}
```

`next()` returns the next node. The assignment moves `curr` forward. The assignment must run for every node, also for a node with a `NULL` entry, so it is outside the `if`. If the assignment were inside the `if`, a `NULL` entry would stop `curr` from moving, and the loop would never end. After the loop, the function returns the sum.

Trace

The call is `yyra.costToCompany();` from `main.cpp`, which calls `totalCost(departments)`. The company holds `Sales` with 1 employee, `Marketing` with 2 employees and `Development` with 1 employee. The function `addDepartment` uses `prepend`, so the list order is `Development`, `Marketing`, `Sales`.

Step	`curr` holds	`department != NULL`	`costToCompany()`	`total`
Start	none	not applicable	not applicable	0
Round 1	`Development`	true	1 * 25000 = 25000	25000
Round 2	`Marketing`	true	2 * 18000 = 36000	61000
Round 3	`Sales`	true	1 * 15000 = 15000	76000
End	`NULL`	the loop condition `curr != NULL` is false	not applicable	76000

The function returns 76000.

The call `totalCost(numbers)` on a `List<int>` uses the general template and returns 0. The call `totalCost(empty)` on an empty `List<Department*>` uses the specialization, and the loop does not run.

The output of `main.cpp` is:

```
totalCost(numbers): 0
Company cost: R76000
totalCost(empty): 0
```

Common mistakes

- **Omit the `NULL` check.** The edge test has `NULL` entries. The call `department->costToCompany()` on a `NULL` pointer is undefined behaviour. In the edge test, valgrind reports `Invalid read of size 8` in the `Mixed` call, and the program stops with a segmentation fault (signal 11).
- **Do not set `total` to 0.** With `float total;`, the sum starts from an undefined value. The compiler gives no warning with the course flags, so the code builds. In `main.cpp`, the printed values happened to be correct, but valgrind reports `Conditional jump or move depends on uninitialised value(s)` and `Use of uninitialised value of size 8`. The result cannot be trusted, also for an empty list, and the valgrind errors cost marks.

- **Place `curr = curr->next();` inside the `if`.** A `NULL` entry then leaves `curr` unchanged, and the loop does not end. In the edge test, the program hangs at the `Mixed` call and prints nothing more.
- **Delete the departments in `totalCost`.** The company deletes them in `~Company()`, and `main` deletes the others. Each department is then deleted twice. In `main.cpp`, the output is still correct, but valgrind reports `Invalid read` and `Invalid free() / delete / delete[] / realloc()` errors when `~Company()` deletes the departments again. The function must not change the list.
- **Use `list.headNode`.** The member is private, and `totalCost` is not a member of `List`. The compiler reports that `headNode` is private within this context, and the code does not compile. The function must use `list.head()`.
- **Take the list by value**, as in `List<Department*> list`. The definition does not match the declaration in `Templates.h`. The compiler reports that the template-id `totalCost<Department*>` does not match any template declaration, and the code does not compile. The copy constructor of `List` is also private, so a list cannot be passed by value.

B3-07. Template specialization: `addition<Employee, Employee>` throws (100)

File: B_3.cpp

```
#include "Templates.h"

// Adding two employees makes no sense, so this specialization throws.
template <>
Employee addition<Employee, Employee>(Employee v1, Employee v2) {
    throw Exception("Cannot add two employees");
}
```

Expected output of the given main.cpp

```
7
5.5
Human Resources
Caught: Cannot add two employees
Program continues
```

What the function must do

The general template `addition` returns `v1 + v2`. Two `Employee` objects cannot be added, because the class has no `+` operator. The specialization for two employees must not add anything. The function must throw an `Exception` with the message `Cannot add two employees`, and it must not print anything.

How the solution works

A **function template** is a recipe for functions. In `template <class T, class U> T addition(T v1, U v2)`, the names `T` and `U` are template parameters: placeholders for two types. The function returns a value of type `T`. For `addition(3, 4.5)`, the compiler sets `T` to `int` and `U` to `double`, and builds a version for those types.

A **specialization** is a version of the template that you write for exact types. When the program calls `addition` with two `Employee` objects, the compiler uses the specialization and not the general template.

```
template <>
Employee addition<Employee, Employee>(Employee v1, Employee v2) {
```

The line `template <>` tells the compiler that this function is a specialization. The text `<Employee, Employee>` after the function name gives the types for `T` and `U`. The parameters `v1` and `v2` are passed by value, so C++ makes a copy of each argument with the `Employee` copy constructor. The definition must match the declaration in `Templates.h` exactly: the same return type, the same name and the same parameter types. The declaration appears after the general template and before any call, so `main.cpp` calls this specialization.

```
    throw Exception("Cannot add two employees");
}
```

The keyword `throw` stops the function and sends an exception to the calling code. The expression `Exception("Cannot add two employees")` creates an `Exception` object that holds the message. The function throws the object by value. It does not throw a pointer and does not throw a C-string.

The function has no `return` statement. The function ends with `throw`, so the code after it never runs, and the compiler does not report a missing return value. The return type `Employee` is still required, because the declaration says so.

The calling code uses a `try` block and a `catch` block. A `catch` block with the parameter `Exception& e` catches an `Exception` thrown by value. The `&` means that `e` is a reference to the thrown object and not a copy. The function `what()` returns the message. A catch of another type, such as `catch (char*)`, does not match an `Exception` object.

The function does not use `new`, so it allocates nothing and frees nothing. When the function leaves through `throw`, C++ destroys the copies `v1` and `v2` by itself, so no memory is lost. The caller's heap objects are not affected, and the caller can still delete them.

The function prints nothing. The `catch` block in the calling code prints the message.

Trace

The calls come from `main.cpp`. The first three calls use the general template. The call with two employees uses the specialization.

Call	Template used	Computation	Result
`addition(3, 4.5)`	general, `T` is `int`, `U` is `double`	`3 + 4.5` is `7.5`, and the function converts it to `int`	`7`
`addition(2.5, 3)`	general, `T` is `double`, `U` is `int`	`2.5 + 3`	`5.5`
`addition(string("Human "), string("Resources"))`	general, `T` and `U` are `string`	joins the two strings	`Human Resources`
`addition(thandi, sipho)`	specialization	throws	see the steps below

The steps of the last call:

Step	Event
1	`main` enters the `try` block.
2	The call `addition(thandi, sipho)` copies `thandi` and `sipho` into `v1` and `v2`.
3	The function runs `throw Exception("Cannot add two employees");`.
4	The function ends at once. C++ destroys `v1` and `v2`. No value is returned.
5	The statement `Employee both = ...` does not finish, so `both` is not created. The line `cout << both << endl;` does not run.
6	The `catch (Exception& e)` block matches the thrown type. `e.what()` gives `Cannot add two employees`.
7	The block prints `Caught: Cannot add two employees`.
8	The program continues after the `catch` block and prints `Program continues`.

The output of `main.cpp` is:

```
7
5.5
Human Resources
Caught: Cannot add two employees
Program continues
```

Common mistakes

- **Throw a pointer, as in `throw new Exception("...")`.** The type of the thrown value is `Exception*`. The block `catch (Exception& e)` does not match it, so the exception is uncaught. The program prints `terminate called after throwing an instance of 'Exception*'` and stops with signal 6 (SIGABRT). The lines `Caught: ...` and `Program continues` are never printed, and the mark is 0.
- **Throw a C-string, as in `throw "Cannot add two employees";`.** The lecture example for `char` throws a C-string, but here `main.cpp` catches `Exception&`. The type does not match, so the exception is uncaught. The program prints `terminate called after throwing an instance of 'char const*'` and stops with signal 6 (SIGABRT). The mark is 0.
- **Print the message instead of throwing.** The function then reaches its end without a return value. With `-Wall -Werror`, the compiler reports `error: no return statement in function returning non-void [-Werror=return-type]`, and the program does not compile.
- **Return one of the employees, as in `return v1;`.** No exception is thrown. In `main.cpp`, the program prints `Thandi Mokoena, 29, F, ID: none` instead of `Caught: Cannot add two employees`, and the output differs. The edge test prints `No exception` and `Caught 0 times` for this case.
- **Use a different message.** The text must be exactly `Cannot add two employees`. A different text changes the line that `what()` gives, and the output differs.